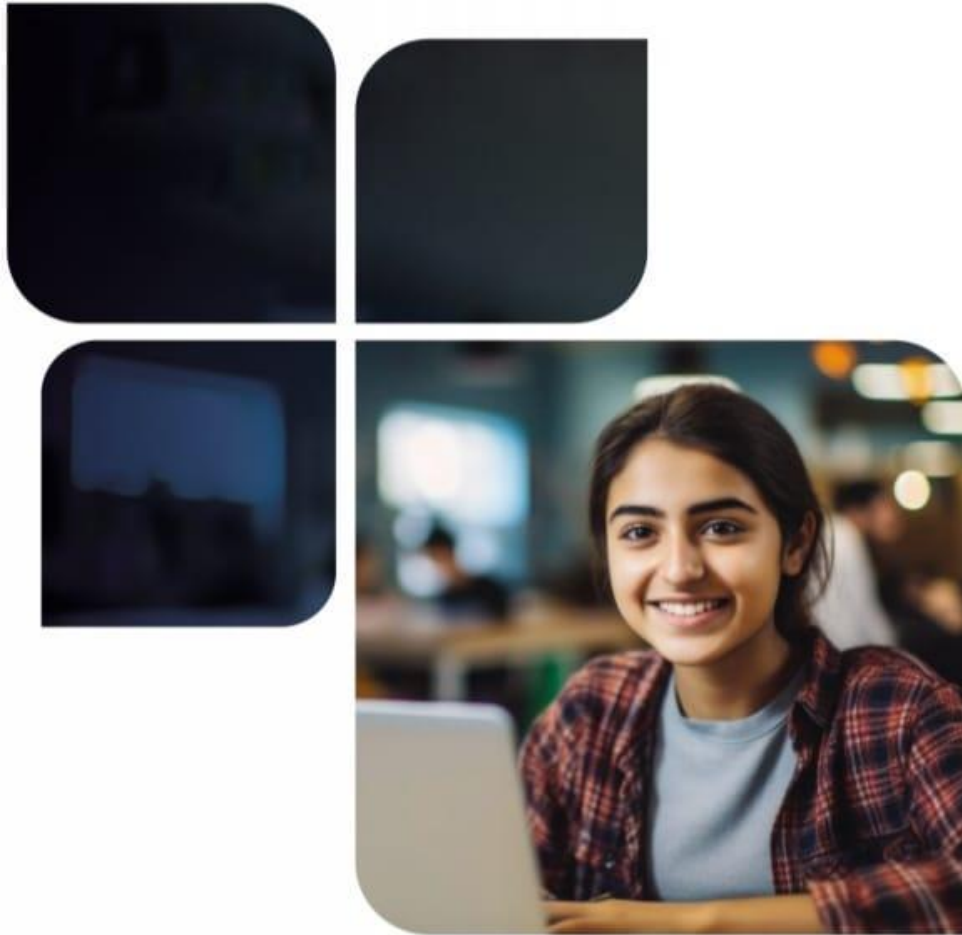


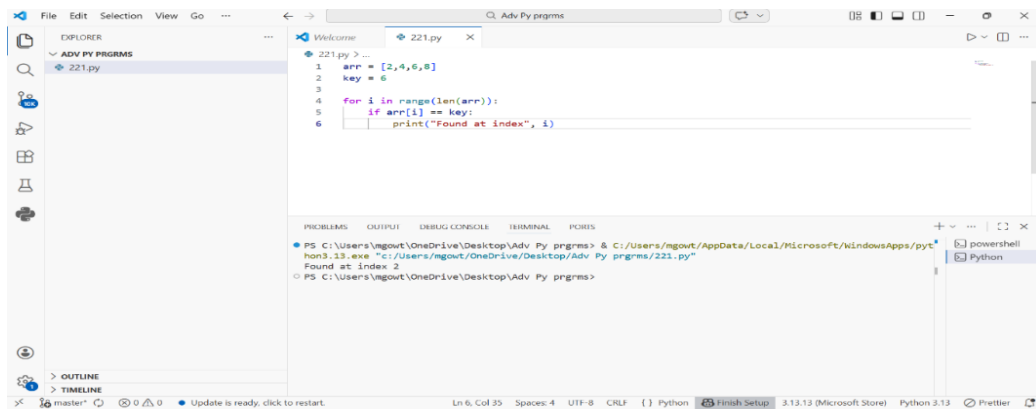
An ISO 9001 & ISO 21001
Certified Organization



The Quest
for your Dream Job
Ends Here!!

www.gqtech.in

221. Write a Python Program to implement linear search.

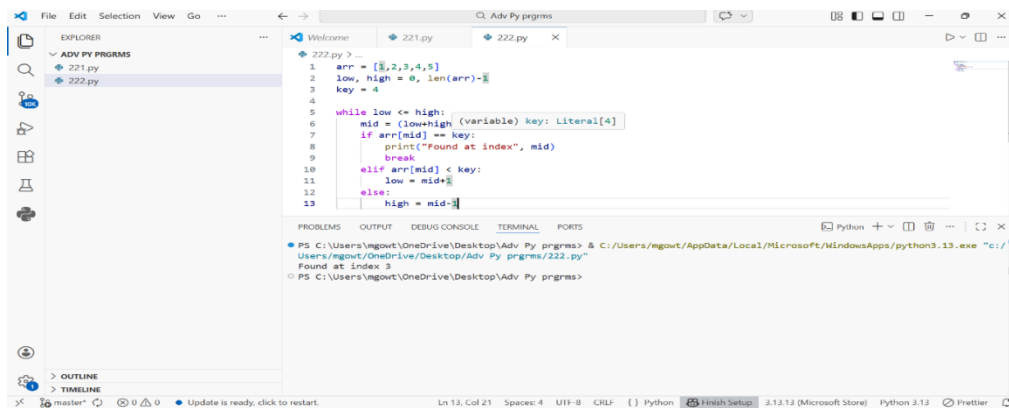


```
221.py > ...
1 arr = [2,4,6,8]
2 key = 6
3
4 for i in range(len(arr)):
5     if arr[i] == key:
6         print("Found at index", i)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgownt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgownt/OneDrive/Desktop/Adv Py prgrms/221.py"
Found at index 2
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms>
```

222. Write a Python Program to implement binary search.

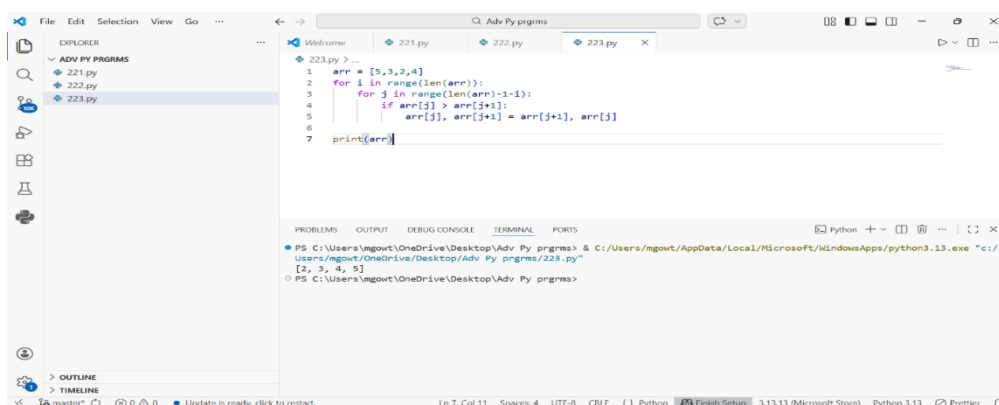


```
222.py > ...
1 arr = [5,2,3,4,5]
2 low, high = 0, len(arr)-1
3 key = 4
4
5 while low <= high:
6     mid = (low+high) // 2
7     if arr[mid] == key:
8         print("Found at index", mid)
9         break
10    elif arr[mid] < key:
11        low = mid+1
12    else:
13        high = mid-1
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgownt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgownt/OneDrive/Desktop/Adv Py prgrms/222.py"
Found at index 3
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms>
```

223. Write a Python Program to implement bubble sort.



```
223.py > ...
1 arr = [5,3,2,4]
2 for i in range(len(arr)):
3     for j in range(len(arr)-1-i):
4         if arr[j] > arr[j+1]:
5             arr[j], arr[j+1] = arr[j+1], arr[j]
6
7 print(arr)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgownt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgownt/OneDrive/Desktop/Adv Py prgrms/223.py"
[2, 3, 4, 5]
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms>
```

224. Write a Python Program to implement insertion sort.

The screenshot shows the Visual Studio Code editor with a file explorer on the left containing files 221.py through 224.py. The main editor window displays 224.py with the following code:

```
1 arr = [4,3,1,2]
2 for i in range(1,len(arr)):
3     key = arr[i]
4     j = i-1
5     while j>=0 and arr[j]>key:
6         arr[j+1] = arr[j]
7         j -=1
8     arr[j+1] = key
9
10 print(arr)
```

The bottom panel shows the terminal output:

```
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgowl/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgowl/OneDrive/Desktop/Adv Py prgrms/224.py"
[1, 2, 3, 4]
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms>
```

225. Write a Python Program to implement selection sort

The screenshot shows the Visual Studio Code editor with a file explorer on the left containing files 221.py through 225.py. The main editor window displays 225.py with the following code:

```
1 arr = [10,2,3,4,5,6]
2 for i in range(len(arr)):
3     min_idx = i
4     for j in range(i+1,len(arr)):
5         if arr[j] < arr[min_idx]:
6             min_idx = j
7     arr[i], arr[min_idx] = arr[min_idx], arr[i]
8
9 print(arr)
```

The bottom panel shows the terminal output:

```
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgowl/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgowl/OneDrive/Desktop/Adv Py prgrms/225.py"
[2, 3, 4, 5, 6, 10]
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms>
```

226. Write a Python Program to implement merge sort.

The screenshot shows the Visual Studio Code editor with a file explorer on the left containing files 221.py through 226.py. The main editor window displays 226.py with the following code:

```
1 def merge_sort(arr):
2     if len(arr)>1:
3         mid = len(arr)//2
4         L = arr[:mid]
5         R = arr[mid:]
6         merge_sort(L)
7         merge_sort(R)
8         i=j=k=0
9         while i<len(L) and j<len(R):
10             if L[i]<R[j]:
11                 arr[k]=L[i]; i+=1
12             else:
13                 arr[k]=R[j]; j+=1
14             k+=1
15         while i<len(L):
16             arr[k]=L[i]; i+=1; k+=1
17         while j<len(R):
18             arr[k]=R[j]; j+=1; k+=1
19         arr=[5,2,4,1]
20         merge_sort(arr)
21         print(arr)
```

The bottom panel shows the terminal output:

```
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgowl/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgowl/OneDrive/Desktop/Adv Py prgrms/226.py"
[1, 2, 4, 5]
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms>
```

227. Write a Python Program to implement quick sort.

```
1 def quick(arr):
2     if len(arr) <= 1:
3         return arr
4     pivot = arr[0]
5     left = [x for x in arr[1:] if x <= pivot]
6     right = [x for x in arr[1:] if x > pivot]
7     return quick(left) + [pivot] + quick(right)
8
9 print(quick([3, 6, 1, 2]))
```

PS C:\Users\mgovt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgovt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgovt/OneDrive/Desktop/Adv Py prgrms/227.py"

[1, 2, 3, 6]

PS C:\Users\mgovt\OneDrive\Desktop\Adv Py prgrms>

228. Write a Python Program to implement heap sort.

```
1 import heapq
2 arr=[4,1,3,2]
3 heapq.heapify(arr)
4
5 res=[]
6 while arr:
7     res.append(heapq.heappop(arr))
8
9 print(res)
```

PS C:\Users\mgovt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgovt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgovt/OneDrive/Desktop/Adv Py prgrms/228.py"

[1, 2, 3, 4]

PS C:\Users\mgovt\OneDrive\Desktop\Adv Py prgrms>

229. Write a Python Program to implement counting sort.

```
1 arr=[4,2,2,8,3]
2 count=[0]*(max(arr)+1)
3
4 for num in arr:
5     count[num] += 1
6
7 res=[]
8 for i in range(len(count)):
9     res += [i]*count[i]
10
11 print(res)
```

PS C:\Users\mgovt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgovt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgovt/OneDrive/Desktop/Adv Py prgrms/229.py"

[2, 2, 3, 4, 8]

PS C:\Users\mgovt\OneDrive\Desktop\Adv Py prgrms>

230. Write a Python Program to implement radix sort.

The screenshot shows the Visual Studio Code editor with a file explorer on the left containing a folder named 'ADV PY PRGRMS' with files 221.py through 230.py. The main editor window displays '230.py' with the following Python code:

```
1 def radix(arr):
2     exp=1
3     max1=max(arr)
4     while max1/exp>0:
5         buckets=[]
6         for num in arr:
7             buckets[(num//exp)%10].append(num)
8         arr=[num for bucket in buckets for num in bucket]
9         exp*=10
10    return arr
11
12 print(radix([170,45,75,90]))
```

The bottom panel shows the terminal with the command: `PS C:\Users\mgout\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgout/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgout/OneDrive/Desktop/Adv Py prgrms/230.py"` and the output: `[45, 75, 90, 170]`.

231. Write a Python Program to find the largest element using linear search.

The screenshot shows the Visual Studio Code editor with the file explorer on the left. The main editor window displays '231.py' with the following Python code:

```
1 arr=[2,9,4]
2 max_val=arr[0]
3
4 for x in arr:
5     if x>max_val:
6         max_val=x
7
8 print(max_val)
```

The bottom panel shows the terminal with the command: `PS C:\Users\mgout\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgout/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgout/OneDrive/Desktop/Adv Py prgrms/231.py"` and the output: `9`.

232. Write a Python Program to find the smallest element using linear search.

The screenshot shows the Visual Studio Code editor with the file explorer on the left. The main editor window displays '232.py' with the following Python code:

```
1 arr=[2,9,1]
2 min_val=arr[0]
3
4 for x in arr:
5     if x<min_val:
6         min_val=x
7
8 print(min_val)
```

The bottom panel shows the terminal with the command: `PS C:\Users\mgout\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgout/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgout/OneDrive/Desktop/Adv Py prgrms/232.py"` and the output: `1`.

233. Write a Python Program to find the position of an element using linear search.

The screenshot shows the Visual Studio Code interface with a file explorer on the left containing a folder named 'ADV PY PROGRAMS' with files 221.py through 233.py. The main editor window displays '233.py' with the following code:

```
1 arr=[5,3,7]
2 key=7
3
4 for i in range(len(arr)):
5     if arr[i]==key:
6         print(i)
```

The bottom panel shows the TERMINAL with the command `PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> C:/Users/mgowl/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgowl/OneDrive/Desktop/Adv Py prgrms/233.py"` and the output `2`.

234. Write a Python Program to find the position of an element using binary search.

The screenshot shows the Visual Studio Code interface with the file explorer on the left. The main editor window displays '234.py' with the following code:

```
1 arr=[3,2,3,4]
2 key=3
3 low,high=0,len(arr)-1
4
5 while low<high:
6     mid=(low+high)//2
7     if arr[mid]==key:
8         print(mid)
9         break
10    elif arr[mid]<key:
11        low=mid+1
12    else:
13        high=mid-1
```

The bottom panel shows the TERMINAL with the command `PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> C:/Users/mgowl/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgowl/OneDrive/Desktop/Adv Py prgrms/234.py"` and the output `2`.

235. Write a Python Program to count occurrences of an element in a list.

The screenshot shows the Visual Studio Code interface with the file explorer on the left. The main editor window displays '235.py' with the following code:

```
1 arr=[1,2,2,3,2]
2 print(arr.count(2))
```

The bottom panel shows the TERMINAL with the command `PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> C:/Users/mgowl/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgowl/OneDrive/Desktop/Adv Py prgrms/235.py"` and the output `3`.

236. Write a Python Program to remove an element from a list.

The screenshot shows the Visual Studio Code editor with a file explorer on the left containing a folder named 'ADV PY PROGRAMS' with files 221.py through 236.py. The active file is 236.py, which contains the following code:

```
1 arr=[1,2,3]
2 arr.remove(2)
3 print(arr)
```

The bottom panel shows the terminal with the command `python 236.py` and its output:

```
PS C:\Users\mgovt\OneDrive\Desktop\Adv Py prgrms> python 236.py
[1, 3]
```

237. Write a Python Program to insert an element at a specific position in a list.

The screenshot shows the Visual Studio Code editor with the file explorer on the left. The active file is 237.py, which contains the following code:

```
1 arr=[1,3,4]
2 arr.insert(1,2)
3 print(arr)
```

The bottom panel shows the terminal with the command `python 237.py` and its output:

```
PS C:\Users\mgovt\OneDrive\Desktop\Adv Py prgrms> python 237.py
[1, 2, 3, 4]
```

238. Write a Python Program to delete an element at a specific position in a list.

The screenshot shows the Visual Studio Code editor with the file explorer on the left. The active file is 238.py, which contains the following code:

```
1 arr=[1,2,3,4]
2 del arr[2]
3 print(arr)
```

The bottom panel shows the terminal with the command `python 238.py` and its output:

```
PS C:\Users\mgovt\OneDrive\Desktop\Adv Py prgrms> python 238.py
[1, 2, 4]
```

239. Write a Python Program to find the median of a list.

The screenshot shows the Visual Studio Code editor with a file explorer on the left containing a directory named 'ADV PY PROGRAMS' with files 221.py through 239.py. The main editor window displays a file named '239.py' with the following Python code:

```
1 arr = [1, 3, 2, 5, 4]
2 arr.sort()
3 n = len(arr)
4
5 if n % 2 == 0:
6     median = (arr[n//2] + arr[n//2 - 1]) / 2
7 else:
8     median = arr[n//2]
9
10 print(median)
```

The bottom panel shows the TERMINAL with the command `PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> C:\Users\mgowt\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:/Users/mgowt/OneDrive/Desktop/Adv Py prgrms/239.py"` and the output `3`.

240. Write a Python Program to find the mode of a list.

The screenshot shows the Visual Studio Code editor with the file explorer on the left. The main editor window displays a file named '240.py' with the following Python code:

```
1 from collections import Counter
2
3 arr = [1,2,2,3,3,4]
4 mode = Counter(arr).most_common(1)
5 print(mode[0][0])
```

The bottom panel shows the TERMINAL with the command `PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> C:\Users\mgowt\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:/Users/mgowt/OneDrive/Desktop/Adv Py prgrms/240.py"` and the output `3`.

241. Write a Python Program to find the mean of a list.

The screenshot shows the Visual Studio Code editor with the file explorer on the left. The main editor window displays a file named '241.py' with the following Python code:

```
1 arr = [1,2,3,4,5]
2 mean = sum(arr)/len(arr)
3 print(mean)
```

The bottom panel shows the TERMINAL with the command `PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> C:\Users\mgowt\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:/Users/mgowt/OneDrive/Desktop/Adv Py prgrms/241.py"` and the output `3.0`.

242. Write a Python Program to calculate standard deviation of a list.

The screenshot shows the Visual Studio Code interface with a file explorer on the left containing a directory named 'ADV PY PROGRAMS' with files 221.py through 242.py. The main editor window displays '242.py' with the following Python code:

```
1 import math
2
3 arr = [1,2,3,4,5]
4 mean = sum(arr)/len(arr)
5
6 var = sum((x-mean)**2 for x in arr)/len(arr)
7
8 std = math.sqrt(var)
9
10 print(std)
```

The bottom panel shows the 'TERMINAL' tab with the command prompt output:

```
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> C:/Users/mgown/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgown/OneDrive/Desktop/Adv Py prgrms/242.py"
1.4142135623730951
PS C:\Users\mgown\OneDrive\Desktop\Adv Py prgrms>
```

243. Write a Python Program to calculate variance of a list.

The screenshot shows the Visual Studio Code interface with a file explorer on the left containing a directory named 'ADV PY PROGRAMS' with files 221.py through 243.py. The main editor window displays '243.py' with the following Python code:

```
1 arr = [1,2,3,4,5]
2 mean = sum(arr)/len(arr)
3
4 var = sum((x-mean)**2 for x in arr)/len(arr)
5
6 print(var)
```

The bottom panel shows the 'TERMINAL' tab with the command prompt output:

```
PS C:\Users\mgown\OneDrive\Desktop\Adv Py prgrms> C:/Users/mgown/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgown/OneDrive/Desktop/Adv Py prgrms/243.py"
2.0
PS C:\Users\mgown\OneDrive\Desktop\Adv Py prgrms>
```

244. Write a Python Program to calculate correlation coefficient of two lists.

The screenshot shows the Visual Studio Code interface with a file explorer on the left containing a directory named 'ADV PY PROGRAMS' with files 223.py through 244.py. The main editor window displays '244.py' with the following Python code:

```
1 import numpy as np
2
3 x = [1,2,3,4]
4 y = [2,4,6,8]
5
6 print(np.corrcoef(x,y)[0][1])
```

The bottom panel shows the 'TERMINAL' tab with the command prompt output:

```
PS C:\Users\mgown\OneDrive\Desktop\Adv Py prgrms> C:/Users/mgown/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgown/OneDrive/Desktop/Adv Py prgrms/244.py"
2.0
PS C:\Users\mgown\OneDrive\Desktop\Adv Py prgrms>
```

245. Write a Python Program to calculate covariance of two lists.

The screenshot shows the VS Code editor with a file explorer on the left containing a directory named 'ADV PY PRGRMS'. The main editor window displays a file named '245.py' with the following code:

```
1 import numpy as np
2
3 x = [1,2,3]
4 y = [2,4,6]
5
6 print(np.cov(x,y)[0][1])
```

The bottom panel shows the 'TERMINAL' tab with the command prompt output:

```
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgowl/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/U
sers/mgowl/OneDrive/Desktop/Adv Py prgrms/231.py"
9
PS C:\Users\mgowl\OneDrive\Desktop\Adv Py prgrms>
```

246. Write a Python Program to implement stack using list.

The screenshot shows the VS Code editor with the file explorer on the left. The main editor window displays a file named '246.py' with the following code:

```
1 stack = []
2 stack.append(10)
3 stack.append(20)
4 stack.pop()
5
6 print(stack)
```

The bottom panel shows the 'TERMINAL' tab with the command prompt output:

```
PS C:\Users\mgowl\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgowl/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/U
sers/mgowl/OneDrive/Desktop/Adv Py prgrms/246.py"
[10]
PS C:\Users\mgowl\OneDrive\Desktop\Adv Py prgrms>
```

247. Write a Python Program to implement queue using list.

The screenshot shows the VS Code editor with the file explorer on the left. The main editor window displays a file named '247.py' with the following code:

```
1 queue = []
2 queue.append(10)
3 queue.append(20)
4 queue.pop(0)
5
6 print(queue)
```

The bottom panel shows the 'TERMINAL' tab with the command prompt output:

```
PS C:\Users\mgowl\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgowl/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/U
sers/mgowl/OneDrive/Desktop/Adv Py prgrms/247.py"
[20]
PS C:\Users\mgowl\OneDrive\Desktop\Adv Py prgrms>
```

248. Write a Python Program to implement circular queue using list.

```
1 size = 3
2 queue = [None]*size
3 front = rear = -1
4
5 # insert
6 rear = (rear+1)%size
7 queue[rear] = 10
8 front = 0
9
10 rear = (rear+1)%size
11 queue[rear] = 20
12
13 print(queue)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgowt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/U sers/mgowt/OneDrive/Desktop/Adv Py prgrms/248.py"
- [10, 20, None]
- PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgowt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/U sers/mgowt/OneDrive/Desktop/Adv Py prgrms/248.py"
- [10, 20, None]
- PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms>

249. Write a Python Program to implement deque using list.

```
1 dq = []
2 dq.append(10)
3 dq.insert(0,5)
4
5 print(dq)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgowt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/U sers/mgowt/OneDrive/Desktop/Adv Py prgrms/249.py"
- [5, 10]
- PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms>

250. Write a Python Program to implement priority queue using list.

```
1 import heapq
2
3 pq = []
4 heapq.heappush(pq, 3)
5 heapq.heappush(pq, 1)
6
7 print(heapq.heappop(pq))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgowt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/U sers/mgowt/OneDrive/Desktop/Adv Py prgrms/250.py"
- 1
- PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms>

251. Write a Python Program to implement stack using collections deque.

```
251.py
1 from collections import deque
2
3 stack = deque()
4 stack.append(10)
5 stack.append(20)
6 stack.pop()
7
8 print(stack)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgownt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgownt/OneDrive/Desktop/Adv Py prgrms/251.py"

deque([10])

PS C:\Users\mgownt\OneDrive\Desktop\Adv Py prgrms>

252. Write a Python Program to implement queue using collections deque.

```
252.py
1 from collections import deque
2
3 queue = deque()
4 queue.append(10)
5 queue.append(20)
6 queue.popleft()
7
8 print(queue)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\mgownt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgownt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgownt/OneDrive/Desktop/Adv Py prgrms/252.py"

deque([20])

PS C:\Users\mgownt\OneDrive\Desktop\Adv Py prgrms>

253. Write a Python Program to implement stack using class.

```
253.py
1 class Stack:
2     def __init__(self):
3         self.data = []
4
5     def push(self, x):
6         self.data.append(x)
7
8     def pop(self):
9         return self.data.pop()
10
11 s = Stack()
12 s.push(10)
13 s.push(20)
14 s.pop()
15 print(s.data)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\mgownt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgownt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgownt/OneDrive/Desktop/Adv Py prgrms/253.py"

[10]

PS C:\Users\mgownt\OneDrive\Desktop\Adv Py prgrms>

254. Write a Python Program to implement queue using class.

The screenshot shows the VS Code editor with a file explorer on the left containing a directory named 'ADV PY PRGRMS' with files 233.py through 254.py. The main editor window displays '254.py' with the following code:

```
1 class Queue:
2     def __init__(self):
3         self.data = []
4
5     def enqueue(self,x):
6         self.data.append(x)
7
8     def dequeue(self):
9         return self.data.pop(0)
10
11 q = Queue()
12 q.enqueue(10)
13 q.enqueue(20)
14 q.dequeue()
15 print(q.data)
```

The bottom panel shows the TERMINAL output:

```
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgownt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgownt/OneDrive/Desktop/Adv Py prgrms/254.py"
[20]
```

255. Write a Python Program to implement linked list.

The screenshot shows the VS Code editor with the file explorer on the left. The main editor window displays '255.py' with the following code:

```
1 class Node:
2     def __init__(self,data):
3         self.data=data
4         self.next=None
5
6 n1=Node(10)
7 n2=Node(20)
8 n1.next=n2
9
10 print(n1.data, n1.next.data)
```

The bottom panel shows the TERMINAL output:

```
PS C:\Users\mgowt\OneDrive\Desktop\Adv Py prgrms> & C:/Users/mgownt/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/mgownt/OneDrive/Desktop/Adv Py prgrms/255.py"
10 20
```

256. Write a Python Program to implement stack using array.

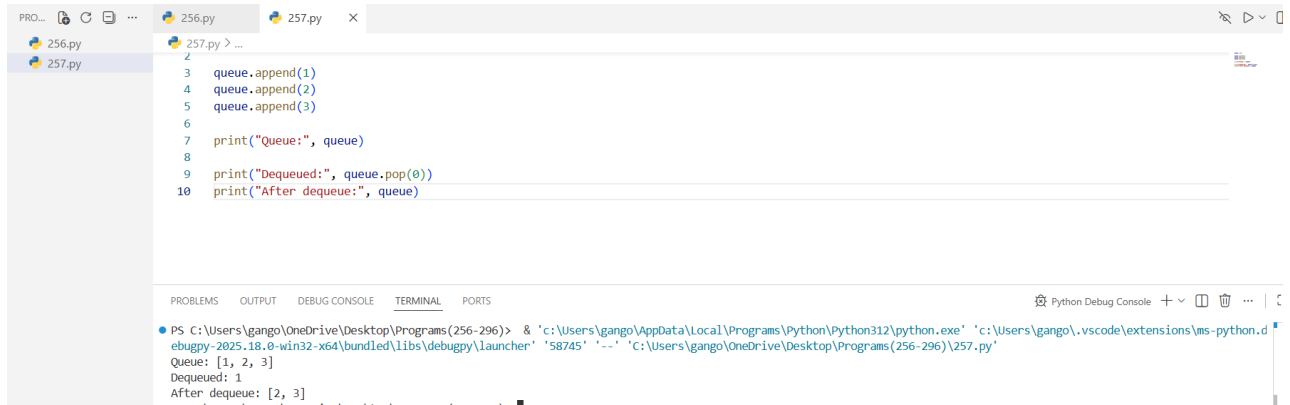
The screenshot shows the VS Code editor with the file explorer on the left. The main editor window displays '256.py' with the following code:

```
1 stack = []
2
3 stack.append(10)
4 stack.append(20)
5 stack.append(30)
6
7 print("Stack:", stack)
8
9 print("Popped:", stack.pop())
10 print("After pop:", stack)
```

The bottom panel shows the TERMINAL output:

```
PS C:\Users\gango\OneDrive\Desktop\Programs (256-296) > & 'c:\Users\gango\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\gango\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64167' '--' 'c:\Users\gango\OneDrive\Desktop\Programs (256-296)\256.py'
Stack: [10, 20, 30]
Popped: 30
After pop: [10, 20]
```

257. Write a Python Program to implement queue using array.



The screenshot shows a VS Code editor with a file named 257.py. The code implements a simple queue using a list. The terminal output shows the execution of the program, which appends 1, 2, and 3 to the queue, prints the queue, dequeues 1, and prints the queue after dequeuing.

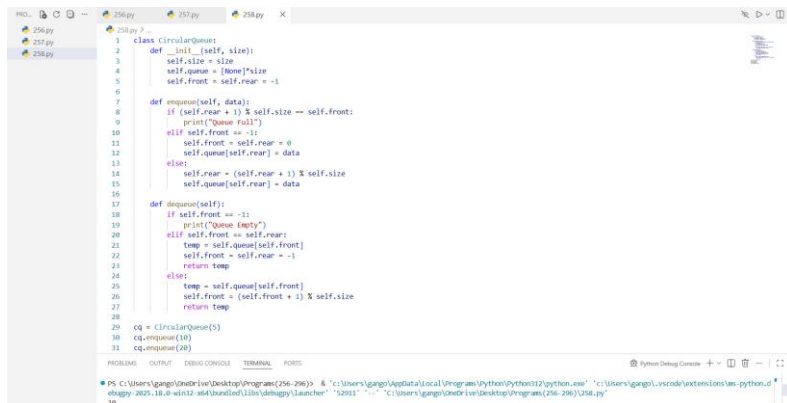
```
257.py > ...
2
3 queue.append(1)
4 queue.append(2)
5 queue.append(3)
6
7 print("Queue:", queue)
8
9 print("Dequeued:", queue.pop(0))
10 print("After dequeue:", queue)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python Debug Console

```
PS C:\Users\gango\OneDrive\Desktop\Programs (256-296)> & 'c:\Users\gango\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\gango\.vscode\extensions\ms-python.d
ebuggy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '58745' '-.' 'c:\Users\gango\OneDrive\Desktop\Programs (256-296)\257.py'
Queue: [1, 2, 3]
Dequeued: 1
After dequeue: [2, 3]
```

258. Write a Python Program to implement circular queue.



The screenshot shows a VS Code editor with a file named 258.py. The code implements a circular queue using a list. The terminal output shows the execution of the program, which enqueues 10 and 20, and dequeues 10.

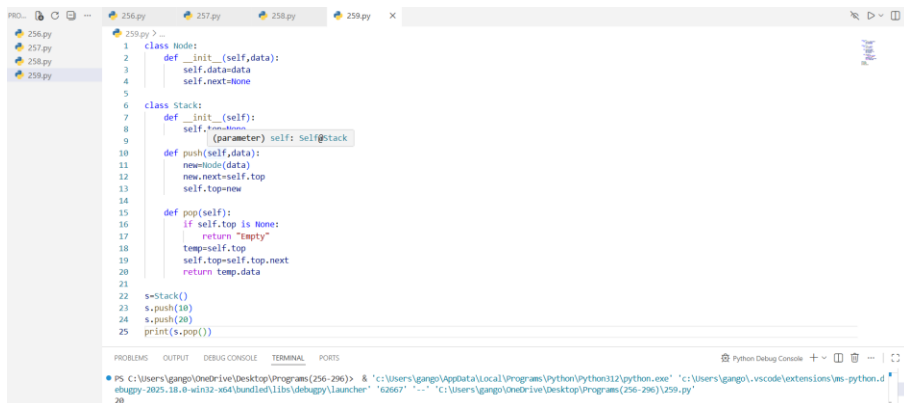
```
258.py > ...
1 class CircularQueue:
2     def __init__(self, size):
3         self.size = size
4         self.queue = [None]*size
5         self.front = self.rear = -1
6
7     def enqueue(self, data):
8         if (self.rear + 1) % self.size == self.front:
9             print("Queue full")
10        elif self.front == -1:
11            self.front = self.rear = 0
12            self.queue[self.rear] = data
13        else:
14            self.rear = (self.rear + 1) % self.size
15            self.queue[self.rear] = data
16
17    def dequeue(self):
18        if self.front == -1:
19            print("Queue empty")
20        elif self.front == self.rear:
21            temp = self.queue[self.front]
22            self.front = self.rear = -1
23            return temp
24        else:
25            temp = self.queue[self.front]
26            self.front = (self.front + 1) % self.size
27            return temp
28
29 cq = CircularQueue(5)
30 cq.enqueue(10)
31 cq.enqueue(20)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python Debug Console

```
PS C:\Users\gango\OneDrive\Desktop\Programs (256-296)> & 'c:\Users\gango\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\gango\.vscode\extensions\ms-python.d
ebuggy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52952' '-.' 'c:\Users\gango\OneDrive\Desktop\Programs (256-296)\258.py'
20
```

259. Write a Python Program to implement stack using linked list.



The screenshot shows a VS Code editor with a file named 259.py. The code implements a stack using a linked list. The terminal output shows the execution of the program, which pushes 10 and 20 onto the stack, and pops 20.

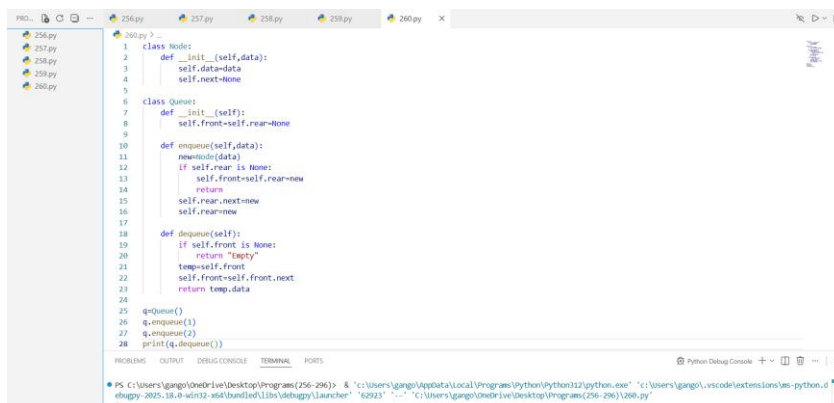
```
259.py > ...
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class Stack:
7     def __init__(self):
8         self.top = None
9         (parameter) self: Self@Stack
10
11    def push(self, data):
12        new_node = Node(data)
13        new_node.next = self.top
14        self.top = new_node
15
16    def pop(self):
17        if self.top is None:
18            return "Empty"
19        temp = self.top
20        self.top = self.top.next
21        return temp.data
22
23 s = Stack()
24 s.push(10)
25 s.push(20)
26 print(s.pop())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python Debug Console

```
PS C:\Users\gango\OneDrive\Desktop\Programs (256-296)> & 'c:\Users\gango\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\gango\.vscode\extensions\ms-python.d
ebuggy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '62667' '-.' 'c:\Users\gango\OneDrive\Desktop\Programs (256-296)\259.py'
20
```

260. Write a Python Program to implement queue using linked list.



The screenshot shows a VS Code editor with a file named 260.py. The code implements a queue using a linked list. The terminal output shows the execution of the program, which enqueues 1 and 2, and dequeues 1.

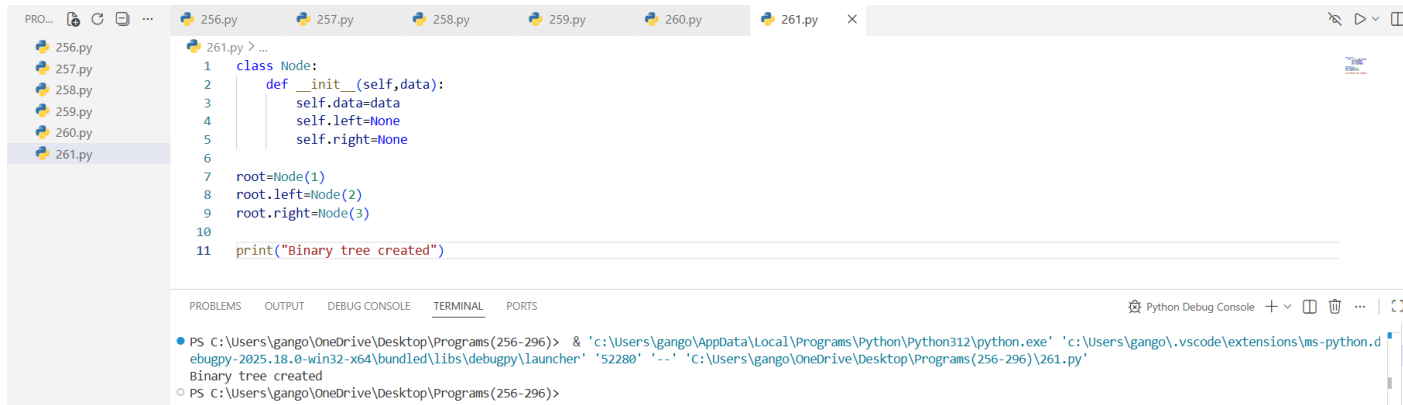
```
260.py > ...
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class Queue:
7     def __init__(self):
8         self.front = self.rear = None
9
10    def enqueue(self, data):
11        new_node = Node(data)
12        if self.rear is None:
13            self.front = self.rear = new_node
14            return
15        self.rear.next = new_node
16        self.rear = new_node
17
18    def dequeue(self):
19        if self.front is None:
20            return "Empty"
21        temp = self.front
22        self.front = self.front.next
23        return temp.data
24
25 q = Queue()
26 q.enqueue(1)
27 q.enqueue(2)
28 print(q.dequeue())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python Debug Console

```
PS C:\Users\gango\OneDrive\Desktop\Programs (256-296)> & 'c:\Users\gango\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\gango\.vscode\extensions\ms-python.d
ebuggy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '42923' '-.' 'c:\Users\gango\OneDrive\Desktop\Programs (256-296)\260.py'
1
```

261. Write a Python Program to implement binary tree.



```
1 class Node:
2     def __init__(self,data):
3         self.data=data
4         self.left=None
5         self.right=None
6
7 root=Node(1)
8 root.left=Node(2)
9 root.right=Node(3)
10
11 print("Binary tree created")
```

PS C:\Users\gango\OneDrive\Desktop\Programs(256-296)> & 'c:\Users\gango\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\gango\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '52280' '--' 'c:\Users\gango\OneDrive\Desktop\Programs(256-296)\261.py'

Binary tree created

PS C:\Users\gango\OneDrive\Desktop\Programs(256-296)>

262. Write a Python Program to implement binary search tree



```
1 class Node:
2     def __init__(self,data):
3         self.data=data
4         self.left=None
5         self.right=None
6
7 def insert(root,key):
8     if root is None:
9         return Node(key)
10    if key < root.data:
11        root.left=insert(root.left,key)
12    else:
13        root.right=insert(root.right,key)
14    return root
15
16 root=None
17 root=insert(root,50)
18 root=insert(root,30)
19 root=insert(root,70)
20 print("BST created")
```

PS C:\Users\gango\OneDrive\Desktop\Programs(256-296)> & 'c:\Users\gango\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\gango\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '51533' '--' 'c:\Users\gango\OneDrive\Desktop\Programs(256-296)\262.py'

BST created

263. Write a Python Program to implement inorder traversal of binary tree



```
1 class Node:
2     def __init__(self,data):
3         self.data=data
4         self.left=None
5         self.right=None
6
7 def inorder(root):
8     if root:
9         inorder(root.left)
10        print(root.data,end=" ")
11        inorder(root.right)
12
13 root=Node(1)
14 root.left=Node(2)
15 root.right=Node(3)
16 root.left.left=Node(4)
17 root.left.right=Node(5)
18
19 inorder(root)
```

PS C:\Users\gango\OneDrive\Desktop\Programs(256-296)> & 'c:\Users\gango\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\gango\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '55294' '--' 'c:\Users\gango\OneDrive\Desktop\Programs(256-296)\263.py'

4 2 5 1 3

264. Write a Python Program to implement preorder traversal of binary tree.

```

myproject > myproject > 264.py > ...
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6
7
8 def preorder(root):
9     if root:
10        print(root.data, end=" ")
11        preorder(root.left)
12        preorder(root.right)
13
14
15 # Example usage
16 root = Node(1)
17 root.left = Node(2)
18 root.right = Node(3)
19 root.left.left = Node(4)
20 root.left.right = Node(5)
21
22 print("Preorder Traversal:")
23 preorder(root)
24
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Percentage: 100.0%
Excellent!
PS C:\Users\reeth\OneDrive\Desktop\New folder> c:\cd 'c:\Users\reeth\OneDrive\Desktop\New folder\myproject\myproject\264.py'
Preorder Traversal:
1 2 4 5 3

```

265. Write a Python Program to implement postorder traversal of binary tree.

```

myproject > myproject > 264.py > ...
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6
7
8 def postorder(root):
9     if root:
10        postorder(root.left)
11        postorder(root.right)
12        print(root.data, end=" ")
13
14
15 # Example usage
16 root = Node(1)
17 root.left = Node(2)
18 root.right = Node(3)
19 root.left.left = Node(4)
20 root.left.right = Node(5)
21
22 print("Postorder Traversal:")
23 postorder(root)
24
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\reeth\OneDrive\Desktop\New folder> c:\cd 'c:\Users\reeth\OneDrive\Desktop\New folder\myproject\myproject\264.py'
Postorder Traversal:
4 5 2 3 1

```

266. Write a Python Program to implement level order traversal of binary tree.

```

myproject > myproject > 264.py > level_order
1 from collections import deque
2 class Node:
3     def __init__(self, data):
4         self.data = data
5         self.left = None
6         self.right = None
7 def level_order(root):
8     if not root:
9         return
10    queue = deque()
11    queue.append(root)
12
13    while queue:
14        node = queue.popleft()
15        print(node.data, end=" ")
16
17        if node.left:
18            queue.append(node.left)
19        if node.right:
20            queue.append(node.right)
21
22 # Example usage
23 root = Node(1)
24 root.left = Node(2)
25 root.right = Node(3)
26 root.left.left = Node(4)
27 root.left.right = Node(5)
28 level_order(root)
29
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\reeth\OneDrive\Desktop\New folder> c:\cd 'c:\Users\reeth\OneDrive\Desktop\New folder\myproject\myproject\264.py'
Level Order Traversal:
1 2 3 4 5

```

267. Write a Python Program to find height of a binary tree.

```

1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6
7
8 def height(root):
9     if root is None:
10        return 0
11    else:
12        left_height = height(root.left)
13        right_height = height(root.right)
14        return max(left_height, right_height) + 1
15
16
17 # Example usage
18 root = Node(1)
19 root.left = Node(2)
20 root.right = Node(3)
21 root.left.left = Node(4)
22 root.left.right = Node(5)
23
24 print("Height of Binary Tree:", height(root))
25
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
1 2 3 4 5
Height of Binary Tree: 3

```

268. Write a Python Program to count nodes in a binary tree.

```
myproject > myproject > 264.py > ...
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6
7
8     def count_nodes(root):
9         if root is None:
10            return 0
11        return 1 + count_nodes(root.left) + count_nodes(root.right)
12
13
14 # Example usage
15 root = Node(1)
16 root.left = Node(2)
17 root.right = Node(3)
18 root.left.left = Node(4)
19 root.left.right = Node(5)
20
21 print("Total Nodes:", count_nodes(root))
22
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\reeth\OneDrive\Desktop\New folder> c:: cd 'c:\Users\reeth\OneDrive\Desktop\New folder\eth\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56d

● Total Nodes: 5

○ PS C:\Users\reeth\OneDrive\Desktop\New folder> █

269. Write a Python Program to count leaf nodes in a binary tree.

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6
7
8     def count_leaf_nodes(root):
9         if root is None:
10            return 0
11
12        if root.left is None and root.right is None:
13            return 1
14
15        return count_leaf_nodes(root.left) + count_leaf_nodes(root.right)
16
17
18 # Example usage
19 root = Node(1)
20 root.left = Node(2)
21 root.right = Node(3)
22 root.left.left = Node(4)
23 root.left.right = Node(5)
24 root.right.right = Node(6)
25
26 print("Leaf Nodes Count:", count_leaf_nodes(root))
27
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\reeth\OneDrive\Desktop\New folder> c:: cd 'c:\Users\reeth\OneDrive\Desktop\New folder\eth\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56d

● Leaf Nodes Count: 3

○ PS C:\Users\reeth\OneDrive\Desktop\New folder> █

270. Write a Python Program to find diameter of a binary tree.

```
myproject > myproject > 264.py > diameter
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6
7     def height(root):
8         if root is None:
9             return 0
10        return 1 + max(height(root.left), height(root.right))
11
12     def diameter(root):
13         if root is None:
14             return 0
15
16        left_height = height(root.left)
17        right_height = height(root.right)
18
19        left_diameter = diameter(root.left)
20        right_diameter = diameter(root.right)
21        return max(left_height + right_height + 1, max(left_diameter, right_diameter))
22
23 # Example usage
24 root = Node(1)
25 root.left = Node(2)
26 root.right = Node(3)
27 root.left.left = Node(4)
28 root.left.right = Node(5)
29
30 print("Diameter of Binary Tree:", diameter(root))
31
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\reeth\OneDrive\Desktop\New folder> c:: cd 'c:\Users\reeth\OneDrive\Desktop\New folder\eth\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '54526' '...

● Diameter of Binary Tree: 4

○ PS C:\Users\reeth\OneDrive\Desktop\New folder> █

271. Write a Python Program to find maximum element in a binary tree.

```

myproject > myproject > 264.py > ...
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6
7
8 def find_max(root):
9     if root is None:
10        return float("-inf")
11
12    left_max = find_max(root.left)
13    right_max = find_max(root.right)
14
15    return max(root.data, left_max, right_max)
16
17
18 # Example usage
19 root = Node(10)
20 root.left = Node(20)
21 root.right = Node(5)
22 root.left.left = Node(40)
23 root.left.right = Node(15)
24
25 print("Maximum Element:", find_max(root))
26

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\reeth\OneDrive\Desktop\New folder> c:: cd 'c:\Users\reeth\OneDrive\Desktop\eth\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher'
● Maximum Element: 40

272. Write a Python Program to find minimum element in a binary tree.

```

1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6
7
8 def find_min(root):
9     if root is None:
10        return float("inf")
11
12    left_min = find_min(root.left)
13    right_min = find_min(root.right)
14
15    return min(root.data, left_min, right_min)
16
17
18 # Example usage
19 root = Node(10)
20 root.left = Node(20)
21 root.right = Node(5)
22 root.left.left = Node(40)
23 root.left.right = Node(15)
24
25 print("Minimum Element:", find_min(root))
26

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\reeth\OneDrive\Desktop\New folder> c:: cd 'c:\Users\reeth\OneDrive\Desktop\eth\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher'
● Minimum Element: 5

273. Write a Python Program to search an element in a binary search tree.

```

1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6
7 def search_bst(root, key):
8     if root is None:
9         return False
10
11    if root.data == key:
12        return True
13    elif key < root.data:
14        return search_bst(root.left, key)
15    else:
16        return search_bst(root.right, key)
17
18 # Example usage
19 root = Node(50)
20 root.left = Node(30)
21 root.right = Node(70)
22 root.left.left = Node(20)
23 root.left.right = Node(40)
24 root.right.left = Node(60)
25 root.right.right = Node(80)
26 key = 60
27
28 if search_bst(root, key):
29     print("Element Found")
30 else:
31     print("Element Not Found")
32

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\reeth\OneDrive\Desktop\New folder> c:: cd 'c:\Users\reeth\OneDrive\Desktop\eth\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '57070'
● Element Found

274. Write a Python Program to insert an element in a binary search tree.

```

1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6
7 def insert_bst(root, key):
8     if root is None:
9         return Node(key)
10
11    if key < root.data:
12        root.left = insert_bst(root.left, key)
13    else:
14        root.right = insert_bst(root.right, key)
15
16    return root
17
18 def inorder(root):
19     if root:
20         inorder(root.left)
21         print(root.data, end=" ")
22         inorder(root.right)
23
24 # Example usage
25 root = None
26 root = insert_bst(root, 50)
27 insert_bst(root, 30)
28 insert_bst(root, 70)
29 insert_bst(root, 20)
30 insert_bst(root, 40)
31 insert_bst(root, 60)
32 insert_bst(root, 80)
33

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\reeth\OneDrive\Desktop\New folder> c:: cd 'c:\Users\reeth\OneDrive\Desktop\eth\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher'
● Inorder Traversal after insertion:
20 30 40 50 60 70 80

275. Write a Python Program to delete an element from a binary search tree.

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6
7 def min_value_node(node):
8     current = node
9     while current.left is not None:
10         current = current.left
11     return current
12
13 def delete_bst(root, key):
14     if root is None:
15         return root
16     if key < root.data:
17         root.left = delete_bst(root.left, key)
18     elif key > root.data:
19         root.right = delete_bst(root.right, key)
20     else:
21         # Node with only one child or no child
22         if root.left is None:
23             return root.right
24         elif root.right is None:
25             return root.left
26         else:
27             # Node with two children
28             # Find the inorder successor (minimum node in the right subtree)
29             temp = min_value_node(root.right)
30             root.data = temp.data
31             root.right = delete_bst(root.right, temp.data)
32     return root
33
34 # Example usage
35 root = Node(20)
36 root.left = Node(10)
37 root.right = Node(30)
38 root.left.left = Node(5)
39 root.left.right = Node(15)
40 root.right.left = Node(25)
41 root.right.right = Node(35)
42
43 # Delete 20
44 root = delete_bst(root, 20)
45
46 # Inorder traversal
47 def inorder(root):
48     if root:
49         inorder(root.left)
50         print(root.data, end=" ")
51         inorder(root.right)
52
53 # Original Tree (Inorder):
54 # 5 10 15 20 25 30 35
55 # After Deleting 20:
56 # 5 10 15 25 30 35
```

276. Write a Python Program to implement graph using adjacency list.

```
1 class Graph:
2     def __init__(self):
3         self.graph = {}
4
5     def add_edge(self, u, v):
6         if u not in self.graph:
7             self.graph[u] = []
8         if v not in self.graph:
9             self.graph[v] = []
10
11         self.graph[u].append(v)
12         self.graph[v].append(u) # remove this line if graph is directed
13
14     def display(self):
15         for node in self.graph:
16             print(node, " -> ", self.graph[node])
17
18 # Example usage
19 g = Graph()
20 g.add_edge(0, 1)
21 g.add_edge(0, 2)
22 g.add_edge(1, 2)
23 g.add_edge(1, 3)
24 g.add_edge(2, 3)
25
26 print("Adjacency List:")
27 g.display()
28
29 # Adjacency List:
30 0 -> [1, 2]
31 1 -> [0, 2, 3]
32 2 -> [0, 1, 3]
33 3 -> [1, 2]
```

277. Write a Python Program to implement graph using adjacency matrix.

```
1 class Graph:
2     def __init__(self, vertices):
3         self.V = vertices
4         self.graph = [[0 for _ in range(vertices)] for _ in range(vertices)]
5
6     def add_edge(self, u, v):
7         self.graph[u][v] = 1
8         self.graph[v][u] = 1 # remove this line for directed graph
9
10    def display(self):
11        for row in self.graph:
12            print(row)
13
14 # Example usage
15 g = Graph(4)
16 g.add_edge(0, 1)
17 g.add_edge(0, 2)
18 g.add_edge(1, 2)
19 g.add_edge(1, 3)
20 g.add_edge(2, 3)
21
22 print("Adjacency Matrix:")
23 g.display()
24
25 # Adjacency Matrix:
26 [0, 1, 1, 0]
27 [1, 0, 1, 1]
28 [1, 1, 0, 0]
29 [0, 1, 0, 0]
```

278. Write a Python Program to implement depth first search.

279. Write a Python Program to implement breadth first search.

280. Write a Python Program to implement Dijkstra's algorithm.

281. Write a Python Program to implement Bellman-Ford algorithm.

```

1 class Graph:
2     def __init__(self, vertices):
3
4         self.edges = []
5
6     def add_edge(self, u, v, w):
7         self.edges.append((u, v, w))
8
9     def bellman_ford(self, src):
10        dist = [float("inf")] * self.V
11        dist[src] = 0
12
13        # Relax all edges V-1 times
14        for i in range(self.V - 1):
15            for u, v, w in self.edges:
16                if dist[u] != float("inf") and dist[u] + w < dist[v]:
17                    dist[v] = dist[u] + w
18
19        # Check for negative weight cycle
20        for u, v, w in self.edges:
21            if dist[u] != float("inf") and dist[u] + w < dist[v]:
22                print("Graph contains negative weight cycle")
23                return
24
25        print("Shortest distances from source:", src)
26        for i in range(self.V):
27            print(i, "->", dist[i])

```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
PS C:\Users\reeth\OneDrive\Desktop\New folder> cd 'c:\Users\reeth\OneDrive\Desktop\New folder' & 'C:\Users\reeth\OneDrive\Desktop\New folder\code\extensions\ms-python-debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '58B45' '-'				
Shortest distances from source: 0				
0 -> 0				
1 -> -1				
2 -> 2				
3 -> -2				

282. Write a Python Program to implement Floyd-Warshall algorithm.

```
1 INF = float("inf")
2
3
4 def floyd_warshall(graph):
5     V = len(graph)
6     dist = [[graph[i][j] for j in range(V)] for i in range(V)]
7
8     for k in range(V):
9         for i in range(V):
10            for j in range(V):
11                if dist[i][k] + dist[k][j] < dist[i][j]:
12                    dist[i][j] = dist[i][k] + dist[k][j]
13
14     print("Shortest distance matrix:")
15     for row in dist:
16         for val in row:
17             if val == INF:
18                 print("INF", end=" ")
19             else:
20                 print(val, end=" ")
21         print()
22
23 # Example usage
24 graph = [
25     [0, 3, INF, 5],
26     [2, 0, 6, 4],
27     [3, 1, 0, 5],
28     [5, 3, 2, 0]
29 ]
30
31 # Shortest distance matrix:
32 0 3 INF 5
33 2 0 6 4
34 3 1 0 5
35 5 3 2 0
```

283. Write a Python Program to implement Kruskal's algorithm.

```
1 class Graph:
2     def __init__(self, vertices):
3         self.V = vertices
4         self.edges = []
5
6     def add_edge(self, u, v, w):
7         self.edges.append((w, u, v)) # store as (weight, u, v)
8
9
10 # Disjoint Set (Union-Find)
11 class DisjointSet:
12     def __init__(self, n):
13         self.parent = list(range(n))
14
15     def find(self, i):
16         if self.parent[i] == i:
17             return i
18         self.parent[i] = self.find(self.parent[i])
19         return self.parent[i]
20
21     def union(self, u, v):
22         self.parent[u] = v
23
24
25 def kruskal(graph):
26     result = []
27     graph.edges.sort()
28     ds = DisjointSet(graph.V)
29
30     # Edges in MST:
31     2 - 3 : 4
32     0 - 3 : 5
33     0 - 1 : 10
```

284. Write a Python Program to implement Prim's algorithm.

```
1 import heapq
2
3
4 class Graph:
5     def __init__(self, vertices):
6         self.V = vertices
7         self.graph = {i: [] for i in range(vertices)}
8
9     def add_edge(self, u, v, w):
10        self.graph[u].append((v, w))
11        self.graph[v].append((u, w)) # remove for directed graph
12
13     def prim(self, start=0):
14        visited = [False] * self.V
15        min_heap = [(0, start, -1)] # (weight, node, parent)
16        mst = []
17
18        while min_heap:
19            w, u, parent = heapq.heappop(min_heap)
20
21            if visited[u]:
22                continue
23
24            visited[u] = True
25            if parent != -1:
26                mst.append((parent, u, w))
27
28            for v, weight in self.graph[u]:
29                if not visited[v]:
30                    heapq.heappush(min_heap, (weight, v, u))
31
32 # Edges in MST:
33 0 - 3 : 5
34 3 - 2 : 4
35 0 - 1 : 10
```

285. Write a Python Program to

implement topological sort.

```

1 class Graph:
2     def __init__(self, vertices):
3         self.V = vertices
4         self.graph = {i: [] for i in range(vertices)}
5
6     def add_edge(self, u, v):
7         self.graph[u].append(v)
8
9     def topo_util(self, v, visited, stack):
10        visited[v] = True
11
12        for neighbor in self.graph[v]:
13            if not visited[neighbor]:
14                self.topo_util(neighbor, visited, stack)
15
16        stack.append(v)
17
18    def topological_sort(self):
19        visited = [False] * self.V
20        stack = []
21
22        for i in range(self.V):
23            if not visited[i]:
24                self.topo_util(i, visited, stack)
25
26        print("Topological Order:")
27        print(stack[::-1])
28
29 # Example usage
30

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\reeth\OneDrive\Desktop\New folder> c:\cd 'c:\Users\reeth\OneDrive\Desktop\New folder';
eth\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher" "60648"
Topological order:
[5, 4, 2, 3, 1, 0]

```

286. Write a Python Program to detect cycle in a graph.

```

1 class Graph:
2     def __init__(self, vertices):
3         self.V = vertices
4         self.graph = {i: [] for i in range(vertices)}
5
6     def add_edge(self, u, v):
7         self.graph[u].append(v)
8         self.graph[v].append(u) # remove for directed graph
9
10    def dfs_cycle(self, v, visited, parent):
11        visited[v] = True
12
13        for neighbor in self.graph[v]:
14            if not visited[neighbor]:
15                if self.dfs_cycle(neighbor, visited, v):
16                    return True
17            elif parent != neighbor:
18                return True
19
20        return False
21
22    def has_cycle(self):
23        visited = [False] * self.V
24
25        for i in range(self.V):
26            if not visited[i]:
27                if self.dfs_cycle(i, visited, -1):
28                    return True
29
30        return False
31

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\reeth\OneDrive\Desktop\New folder> c:\cd 'c:\Users\reeth\OneDrive\Desktop\New folder';
eth\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher" "58391"
Graph contains cycle
PS C:\Users\reeth\OneDrive\Desktop\New folder>

```

287. Write a Python Program to check if a graph is connected.

```

1 class Graph:
2     def __init__(self, vertices):
3         self.V = vertices
4         self.graph = {i: [] for i in range(vertices)}
5
6     def add_edge(self, u, v):
7         self.graph[u].append(v)
8         self.graph[v].append(u) # remove for directed graph
9
10    def dfs(self, v, visited):
11        visited[v] = True
12
13        for neighbor in self.graph[v]:
14            if not visited[neighbor]:
15                self.dfs(neighbor, visited)
16
17    def is_connected(self):
18        visited = [False] * self.V
19
20        # start DFS from node 0
21        self.dfs(0, visited)
22
23        # if any node is not visited, graph is not connected
24        for v in range(self.V):
25            if not visited[v]:
26                return False
27        return True
28
29 # Example usage
30

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\reeth\OneDrive\Desktop\New folder> c:\cd 'c:\Users\reeth\OneDrive\Desktop\New folder'; 8
eth\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher" "58362"
Graph is NOT Connected

```

288. Write a Python Program to check if a graph is bipartite.

```

1 from collections import deque
2
3 class Graph:
4     def __init__(self, vertices):
5         self.V = vertices
6         self.graph = {i: [] for i in range(vertices)}
7
8     def add_edge(self, u, v):
9         self.graph[u].append(v)
10        self.graph[v].append(u) # remove for directed graph
11
12    def is_bipartite(self):
13        color = [-1] * self.V
14
15        for start in range(self.V):
16            if color[start] == -1:
17                queue = deque()
18                queue.append(start)
19                color[start] = 0
20
21                while queue:
22                    node = queue.popleft()
23
24                    for neighbor in self.graph[node]:
25                        if color[neighbor] == -1:
26                            color[neighbor] = 1 - color[node]
27                            queue.append(neighbor)
28                        elif color[neighbor] == color[node]:
29                            return False
30

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\reeth\OneDrive\Desktop\New folder> c:\cd 'c:\Users\reeth\OneDrive\Desktop\New folder'; & 'c
eth\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher" "63609"
Graph is Bipartite
PS C:\Users\reeth\OneDrive\Desktop\New folder>

```

289. Write a Python Program to find shortest path in a graph.

```

1  from collections import deque
2
3
4  class Graph:
5      def __init__(self, vertices):
6          self.v = vertices
7          self.graph = {i: [] for i in range(vertices)}
8
9      def add_edge(self, u, v):
10         self.graph[u].append(v)
11         self.graph[v].append(u) # remove for directed graph
12
13     def shortest_path(self, start, end):
14         visited = [False] * self.v
15         parent = [-1] * self.v
16
17         queue = deque()
18         queue.append(start)
19         visited[start] = True
20
21         while queue:
22             node = queue.popleft()
23
24             for neighbor in self.graph[node]:
25                 if not visited[neighbor]:
26                     visited[neighbor] = True
27                     parent[neighbor] = node
28                     queue.append(neighbor)
29
30         if neighbor == end:

```

290. Write a Python Program to find longest path in a graph.

```

1 class Graph:
2     def __init__(self, vertices):
3         self.V = vertices
4         self.graph = {i: [] for i in range(vertices)}
5
6     def add_edge(self, u, v):
7         self.graph[u].append(v)
8
9     def dfs(self, node, visited, path_len):
10        visited[node] = True
11        max_len = path_len
12
13        for neighbor in self.graph[node]:
14            if not visited[neighbor]:
15                max_len = max(max_len, self.dfs(
16                    neighbor, visited, path_len + 1))
17
18        visited[node] = False # backtrack
19        return max_len
20
21    def longest_path(self):
22        max_path = 0
23
24        for i in range(self.V):
25            visited = [False] * self.V
26            max_path = max(max_path, self.dfs(i, visited, 0))
27
28        print("Longest Path Length:", max_path)
29
30

```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```

PS C:\Users\veerth\OneDrive\Desktop\New folder> c:\cd "C:\Users\veerth\OneDrive\Desktop\New folder\"
vscode\extensions\ms-python.debugpy-2023.18.0-win32-x64\bundledLibs\debugpy_launcher

```

291. Write a Python Program to implement hash table.

```

1 class HashTable:
2     def hash_function(self, key):
3         return key % self.size
4
5     def insert(self, key):
6         index = self.hash_function(key)
7         self.table[index].append(key)
8
9     def display(self):
10        for i in range(self.size):
11            print(i, ">", self.table[i])
12
13 # Example usage
14 ht = HashTable(5)
15
16 ht.insert(10)
17 ht.insert(15)
18 ht.insert(20)
19 ht.insert(7)
20 ht.insert(12)
21
22 print("Hash Table:")
23 ht.display()
24

```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```

● Hash Table:
0 -> [10, 15, 20]
1 -> []
2 -> [7, 12]
3 -> []
4 -> []

```

292. Write a Python Program to implement linear probing in hash table.

```

1 class HashTable:
2     def insert(self, key):
3         index = self.hash_function(key)
4
5         # linear probing
6         while self.table[index] is not None:
7             index = (index + 1) % self.size
8
9         self.table[index] = key
10
11     def display(self):
12         for i in range(self.size):
13             print(i, "->", self.table[i])
14
15 # Example usage
16 ht = HashTable(7)
17
18 ht.insert(10)
19 ht.insert(20)
20 ht.insert(15)
21 ht.insert(7)
22 ht.insert(32)
23 ht.insert(18)
24
25 print("Hash Table (Linear Probing):")

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

• Hash Table (Linear Probing):
0 -> 7
1 -> 15
2 -> None
3 -> 10
4 -> 32
5 -> 18
6 -> 20

```

293. Write a program to implement Quadratic Probing in Hash Table.

```

1 class HashTable:
2     def insert(self, key):
3         i = 0
4
5         while self.table[(index + i * i) % self.size] is not None:
6             i += 1
7
8         self.table[(index + i * i) % self.size] = key
9
10    def display(self):
11        for i in range(self.size):
12            print(i, "->", self.table[i])
13
14 # Example usage
15 ht = HashTable(7)
16
17 ht.insert(10)
18 ht.insert(20)
19 ht.insert(15)
20 ht.insert(7)
21 ht.insert(32)
22
23 print("Hash Table using Quadratic Probing:")
24 ht.display()

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

• Hash Table using Quadratic Probing:
0 -> 7
1 -> 15
2 -> None
3 -> 10
4 -> 32
5 -> None
6 -> 20

```

294. Write a program to implement Double Hashing in Hash Table.

```

1 class HashTable:
2     def insert(self, key):
3         index = self.hash1(key)
4         step = self.hash2(key)
5         i = 0
6
7         while self.table[(index + i * step) % self.size] is not None:
8             i += 1
9
10        self.table[(index + i * step) % self.size] = key
11
12    def display(self):
13        for i in range(self.size):
14            print(i, "->", self.table[i])
15
16 # Example usage
17 ht = HashTable(10)
18
19 ht.insert(10)
20 ht.insert(20)
21 ht.insert(30)
22 ht.insert(40)
23 ht.insert(50)
24
25 print("Hash Table using Double Hashing:")

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

• Hash Table using Double Hashing:
0 -> 10
1 -> 20
2 -> 40
3 -> None
4 -> None
5 -> 30
6 -> 50
7 -> None
8 -> None
9 -> None

```

295. Write a program to implement Chaining in Hash Table.

```

1 class HashTable:
2     def __init__(self, size):
3         self.size = size
4         self.table = [[] for _ in range(size)]
5
6     def hash_function(self, key):
7         return key % self.size
8
9     def insert(self, key):
10        index = self.hash_function(key)
11        self.table[index].append(key)
12
13    def display(self):
14        for i in range(self.size):
15            print(i, "->", self.table[i])
16
17 # Example usage
18 ht = HashTable(5)
19
20 ht.insert(10)
21 ht.insert(15)
22 ht.insert(20)
23 ht.insert(25)
24 ht.insert(30)
25
26 print("Hash Table using chaining:")
27 ht.display()

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

• Hash Table using Chaining:
0 -> [10, 15, 20, 25, 30]
1 -> []
2 -> []
3 -> []
4 -> []

```

296. Write a program to implement LRU Cache.

```

4 class LRUCache:
5     def __init__(self, capacity):
6         self.capacity = capacity
7         self.cache = OrderedDict()
8
9     def get(self, key):
10        if key not in self.cache:
11            return -1
12        self.cache.move_to_end(key)
13        return self.cache[key]
14
15    def put(self, self, key, value):
16        if key in self.cache:
17            self.cache.move_to_end(key)
18            self.cache[key] = value
19
20        if len(self.cache) > self.capacity:
21            self.cache.popitem(last=False)
22
23    def display(self):
24        print(self.cache)
25
26
27 # Example usage
28 lru = LRUCache(3)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\reeth\OneDrive\Desktop\New folder> c:\cd 'c:\Users\reeth\OneDrive\Desktop\New folder\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bin\debugpy\launcher' '5583'
● LRU Cache State:
OrderedDict({'C': 1, 'A': 4, 'D': 1})

```

297. Write a Python Program to implement LFU cache.

[illegible]

298. Write a Python Program to implement MRU cache.

```

29.py X 29.py X
1 from collections import OrderedDict
2
3 class MRU:
4     def __init__(self, cap):
5         self.cap = cap
6         self.cache = OrderedDict()
7
8     def get(self, k):
9         if k not in self.cache: return -1
10        self.cache.move_to_end(k)
11        return self.cache[k]
12
13    def put(self, k, v):
14        if k in self.cache:
15            self.cache.move_to_end(k)
16            self.cache[k] = v
17        if len(self.cache) > self.cap:
18            self.cache.popitem(last=True) # remove MRU
19
20 # Test
21 c = MRU(2)
22 c.put(1,10)
23 c.put(2,20)
24 c.get(2) # 2 becomes most recently used
25 # c.get(1, -1) # should return -1
26
27
28
29

```

Microsoft Windows [Version 10.0.20200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\program> python 29.py
OrderedDict([(1, 10), (2, 20)])

C:\Users\kuma\OneDrive\Desktop\program>

299. Write a Python Program to implement memoization.

```
299.py X
1 def memo():
2     cache=[]
3     def wrap(n):
4         if n not in cache:
5             cache[n]=f(n)
6         return cache[n]
7     return wrap
8
9 @memo
10 def fib(n):
11     return n if n<=1 else fib(n-1)+fib(n-2)
12
13 print(fib(10))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.20348.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /c "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 57800 --
c:\Users\kuma\Desktop\programs\299.py
55

C:\Users\kuma\Desktop\programs>

300. Write a Python Program to implement dynamic programming for Fibonacci series.

```
300.py X
1 def fib(n):
2     dp=[n,1]
3     for i in range(2,n+1):
4         dp.append(dp[-1]+dp[-2])
5     return dp[n]
6
7 print(fib(10))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.20348.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /c "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 58401 --
c:\Users\kuma\Desktop\programs\300.py
55

C:\Users\kuma\Desktop\programs>

301. Write a Python Program to implement dynamic programming for factorial.

```
301.py X 300.py
1 def fact(n):
2     dp=[1]*(n+1)
3     for i in range(2,n+1):
4         dp[i]=dp[i-1]*i
5     return dp[n]
6
7 print(fact(5))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.20348.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /c "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 53989 --
c:\Users\kuma\Desktop\programs\301.py
120

C:\Users\kuma\Desktop\programs>

302. Write a Python Program to implement dynamic programming for longest common subsequence.

```
303.py >
1 def lis(a):
2     n=len(a),len(b)
3     dp=[[0]*(n+1) for _ in range(n+1)]
4     for i in range(n):
5         for j in range(n):
6             dp[i+1][j+1]=dp[i][j]+1 if a[i]==b[j] else max(dp[i+1][j],dp[i][j+1])
7     return dp[n][n]
8
9 print(lis("abcde","ace"))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle1\lib\debugpy\launcher 65307 --
c:\Users\kuma\Desktop\programs\303.py -
3

C:\Users\kuma\Desktop\programs>
```

303. Write a Python Program to implement dynamic programming for longest increasing subsequence.

```
303.py >
1 def lis(a):
2     dp=[1]*len(a)
3     for i in range(len(a)):
4         for j in range(i):
5             if a[i]>a[j]:
6                 dp[i]=max(dp[i],dp[j]+1)
7     return max(dp)
8
9 print(lis([10,9,2,5,3,7,101]))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle1\lib\debugpy\launcher 58276 --
c:\Users\kuma\Desktop\programs\303.py -
4

C:\Users\kuma\Desktop\programs>
```

304. Write a Python Program to implement dynamic programming for matrix chain multiplication.

```
304.py >
1 def mcm(p):
2     n=len(p)
3     dp=[[0]*n for _ in range(n)]
4     for i in range(2,n):
5         for j in range(n-1):
6             j=i+1
7             dp[i][j]=min(dp[i][k]+dp[k][j]+p[i]*p[k]*p[j] for k in range(i+1,j))
8     return dp[0][n-1]
9
10 print(mcm([1,2,3,4]))

C:\Users\kuma\Desktop\programs\304.py
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle1\lib\debugpy\launcher 54172 --
c:\Users\kuma\Desktop\programs\304.py -
10

C:\Users\kuma\Desktop\programs>
```

305. Write a Python Program to implement dynamic programming for coin change problem.

```
305.py > ...
1 def coin(x,wt):
2     dp=[float('inf')]*(wt+1)
3     dp[0]=0
4     for i in range(1,wt+1):
5         for j in range(x,wt+1):
6             dp[i]=min(dp[i],dp[i-x]+1)
7     return dp[wt] if dp[wt]!=float('inf') else -1
8
9 print(coin([1,2,5],11))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /C "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\lib\debugpy\launcher 51189 --
c:\Users\kuma\Desktop\programs\305.py
3
C:\Users\kuma\Desktop\programs>
```

306. Write a Python Program to implement dynamic programming for knapsack problem.

```
306.py > ...
1 def knap(w,v,w):
2     n=len(v)
3     dp=[0]*(w+1)
4     for i in range(1,n+1):
5         for j in range(w+1):
6             if v[i-1]<=j:
7                 dp[i][j]=max(dp[i-1][j],v[i-1]+dp[i-1][j-v[i-1]])
8             else:
9                 dp[i][j]=dp[i-1][j]
10    return dp[n][w]
11
12 print(knap([1,2,3],[10,15,40],40))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /C "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\lib\debugpy\launcher 56369 --
c:\Users\kuma\Desktop\programs\306.py
65
C:\Users\kuma\Desktop\programs>
```

307. Write a Python Program to implement dynamic programming for edit distance.

```
307.py > ...
1 def edit(a,b):
2     m=len(a),n=len(b)
3     dp=[0]*(m+1)
4     for i in range(m+1): dp[i][0]=i
5     for j in range(n+1): dp[0][j]=j
6     for i in range(1,m+1):
7         for j in range(1,n+1):
8             dp[i][j]=dp[i-1][j-1] if a[i-1]==b[j-1] else 1+min(dp[i-1][j],dp[i][j-1],dp[i-1][j-1])
9     return dp[m][n]
10
11 print(edit("kitten","sitting"))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /C "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\lib\debugpy\launcher 58536 --
c:\Users\kuma\Desktop\programs\307.py
3
C:\Users\kuma\Desktop\programs>
```

308. Write a Python Program to implement dynamic programming for minimum cost path.

```
309.py > ...
1 def mincost(i):
2     m,n=len(i),len(i[0])
3     dp=[[0]*n for _ in range(m)]
4     dp[0][0]=i[0][0]
5     for i in range(1,m): dp[i][0]=dp[i-1][0]+i[1][0]
6     for j in range(1,n): dp[0][j]=dp[0][j-1]+i[0][j]
7     for i in range(1,m):
8         for j in range(1,n):
9             dp[i][j]=i[1][j]+min(dp[i-1][j],dp[i][j-1])
10    return dp[-1][-1]
11
12 print(mincost([[1,2,3],[4,5,6,7],[5,5,3]]))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 53706 --
c:\Users\kuma\Desktop\programs\309.py "
11
C:\Users\kuma\Desktop\programs>
```

309. Write a Python Program to implement dynamic programming for maximum subarray sum.

```
309.py > ...
1 def maxsub(x):
2     cur=mx=-i[0]
3     for x in i[1:]:
4         cur=max(x,cur+x)
5     mx=max(mx,cur)
6     return mx
7
8 print(maxsub([-2,1,-3,4,-1,2,1,-5]))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 61626 --
c:\Users\kuma\Desktop\programs\309.py "
6
C:\Users\kuma\Desktop\programs>
```

310. Write a Python Program to implement dynamic programming for subset sum problem.

```
310.py > ...
1 def subset(x,i):
2     dp=[0]
3     for x in a:
4         dp=[x+y for y in dp]
5     return > in dp
6
7 print(subset([3,34,4,12,5,2],9))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 51871 --
c:\Users\kuma\Desktop\programs\310.py "
True
C:\Users\kuma\Desktop\programs>
```

311. Write a Python Program to implement object-oriented programming with a class and object.

```
311.py ? ... C:\Users\kuma\OneDrive\Desktop\programs\311.py
1 class A:
2     def __init__(self,x): self.x=x
3     def show(self): print(self.x)
4
5 obj=A(10)
6 obj.show()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 55114 --
c:\Users\kuma\OneDrive\Desktop\programs\311.py "
10
C:\Users\kuma\OneDrive\Desktop\programs>

312. Write a Python Program to implement inheritance.

```
312.py ? ... C:\Users\kuma\OneDrive\Desktop\programs\312.py
1 class A:
2     def show(self): print("A")
3
4 class B(A):
5     def display(self): print("B")
6
7 obj=A()
8 obj.show()
9 obj.display()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 55527 --
c:\Users\kuma\OneDrive\Desktop\programs\312.py "
A
B
C:\Users\kuma\OneDrive\Desktop\programs>

313. Write a Python Program to implement multiple inheritance.

```
313.py ? ... C:\Users\kuma\OneDrive\Desktop\programs\313.py
1 class A:
2     def a(self): print("A")
3
4 class B:
5     def b(self): print("B")
6
7 class C(A,B):
8     pass
9
10 obj=C()
11 obj.a()
12 obj.b()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 56100 --
c:\Users\kuma\OneDrive\Desktop\programs\313.py "
A
B
C:\Users\kuma\OneDrive\Desktop\programs>

314. Write a Python Program to implement multilevel inheritance.

```
314.py > ...
1 class A:
2     def a(self): print("A")
3 class B(A):
4     def b(self): print("B")
5 class C(B):
6     def c(self): print("C")
7
8 obj = C()
9 obj.a(); obj.b(); obj.c()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 59392 --
c:\Users\kuma\OneDrive\Desktop\programs\314.py "
A
B
C
C:\Users\kuma\OneDrive\Desktop\programs>

315. Write a Python Program to implement hierarchical inheritance.

```
315.py > ...
1 class A:
2     def a(self): print("A")
3 class B(A):
4     def b(self): print("B")
5 class C(A):
6     def c(self): print("C")
7
8 B().a(); C().c()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 58304 --
c:\Users\kuma\OneDrive\Desktop\programs\315.py "
A
C
C:\Users\kuma\OneDrive\Desktop\programs>

316. Write a Python Program to implement hybrid inheritance.

```
316.py > ...
1 class A:
2     def a(self): print("A")
3 class B(A): pass
4 class C(A): pass
5 class D(B,C):
6     def d(self): print("D")
7
8 obj = D()
9 obj.a(); obj.d()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 56287 --
c:\Users\kuma\OneDrive\Desktop\programs\316.py "
A
D
C:\Users\kuma\OneDrive\Desktop\programs>

317. Write a Python Program to implement polymorphism.

```
317.py
1 class A:
2     def show(self): print("A")
3 class B:
4     def show(self): print("B")
5
6 for x in (A(),B()):
7     x.show()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /c "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 64413 --
c:\Users\kuma\OneDrive\Desktop\programs\317.py
A
B
C:\Users\kuma\OneDrive\Desktop\programs>

318. Write a Python Program to implement method overloading.

```
318.py
1 class A:
2     def add(self,a,b,c=0):
3         print(a+b+c)
4
5 obj=A()
6 obj.add(1,3)
7 obj.add(2,3,4)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /c "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 55526 --
c:\Users\kuma\OneDrive\Desktop\programs\318.py
5
9
C:\Users\kuma\OneDrive\Desktop\programs>

319. Write a Python Program to implement method overriding.

```
319.py
1 class A:
2     def show(self): print("A")
3 class B(A):
4     def show(self): print("B")
5
6 B().show()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /c "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 58659 --
c:\Users\kuma\OneDrive\Desktop\programs\319.py
B
C:\Users\kuma\OneDrive\Desktop\programs>

320. Write a Python Program to implement operator overloading.

```
1 class A:
2     def __init__(self,x): self.x=x
3     def __add__(self,y): return self.x+y
4     def __str__(self): return self.x
5     print(a(2)+a(3))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.20H.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cmd /c "C:\Program Files\Python32\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle1\libs\debugpy\launcher 63055 --
c:\Users\kuma\OneDrive\Desktop\programs\320.py -

C:\Users\kuma\OneDrive\Desktop\programs>

321. Write a Python Program to implement encapsulation.

```
1 class A:
2     def __init__(self): self.__x=10
3     def show(self): print(self.__x)
4
5 A().show()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.20H.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cmd /c "C:\Program Files\Python32\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle1\libs\debugpy\launcher 56315 --
c:\Users\kuma\OneDrive\Desktop\programs\321.py -

C:\Users\kuma\OneDrive\Desktop\programs>

322. Write a Python Program to implement abstraction.

```
1 from abc import ABC, abstractmethod
2
3 class A(ABC):
4     @abstractmethod
5     def show(self): pass
6
7 class B(A):
8     def show(self): print("Abstract")
9
10 B().show()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.20H.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cmd /c "C:\Program Files\Python32\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle1\libs\debugpy\launcher 51246 --
c:\Users\kuma\OneDrive\Desktop\programs\322.py -

Abstract

C:\Users\kuma\OneDrive\Desktop\programs>

323. Write a Python Program to implement constructor in class.

```
323.py ? ...
1 class A:
2     def __init__(self): print("Constructor")
3
4
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 59646 --
c:\Users\kuma\OneDrive\Desktop\programs\323.py "
Constructor

C:\Users\kuma\OneDrive\Desktop\programs>

324. Write a Python Program to implement destructor in class.

```
324.py ? ...
1 class A:
2     def __del__(self): print("Destructor")
3
4 obj=A()
5 del obj
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 56525 --
c:\Users\kuma\OneDrive\Desktop\programs\324.py "
Destructor

C:\Users\kuma\OneDrive\Desktop\programs>

325. Write a Python Program to implement class variables and instance variables.

```
325.py ? ...
1 class A:
2     x=10
3     def __init__(self): self.y=20
4
5 obj=A()
6 print(x, obj.y)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 57660 --
c:\Users\kuma\OneDrive\Desktop\programs\325.py "
10 20

C:\Users\kuma\OneDrive\Desktop\programs>

326. Write a Python Program to implement static methods.

```
326.py 7
1 class A:
2     @staticmethod
3     def show(): print("Static")
4
5 A.show()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 64945 --
Static

C:\Users\kuma\Desktop\programs>

327. Write a Python Program to implement class methods.

```
327.py 7
1 class A:
2     x=10
3     @classmethod
4     def show(cls): print(cls.x)
5
6 A.show()
```

Debug Console (Ctrl+Shift+V)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 61808 --
10

C:\Users\kuma\Desktop\programs>

328. Write a Python Program to implement abstract base class.

```
328.py 7
1 from abc import ABC, abstractmethod
2
3 class A(ABC):
4     @abstractmethod
5     def show(self): pass
6
7 class B(A):
8     def show(self): print("ABC")
9
10 B().show()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 58044 --
ABC

C:\Users\kuma\Desktop\programs>

329. Write a Python Program to implement interface using abstract class.

```
329.py >
1 from abc import ABC, abstractmethod
2
3 class I(ABC):
4     @abstractmethod
5     def show(self): pass
6
7 class A(I):
8     def show(self): print("Interface")
9
10 I().show()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 55117 --
c:\Users\kuma\OneDrive\Desktop\programs\329.py
Interface

C:\Users\kuma\OneDrive\Desktop\programs>

330. Write a Python Program to implement multiple constructors using @classmethod.

```
330.py >
1 class A:
2     def __init__(self,x): self.x=x
3     @classmethod
4     def from_str(cls,s): return cls(int(s))
5
6 print(A.from_str("10").x)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 52809 --
c:\Users\kuma\OneDrive\Desktop\programs\330.py
10

C:\Users\kuma\OneDrive\Desktop\programs>

331. Write a Python Program to implement exception handling using try-except.

```
331.py
1 try:
2     print(10/0)
3 except:
4     print("Error")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C "C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher 59193 --
c:\Users\kuma\OneDrive\Desktop\programs\331.py
Error

C:\Users\kuma\OneDrive\Desktop\programs>

332. Write a Python Program to implement exception handling using try-except-finally.

```
332.py
1 try:
2     print(10/0)
3 except:
4     print("Error")
5 finally:
6     print("Done")
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle1\lib\debugpy\launcher 52832 --
c:\Users\kuma\OneDrive\Desktop\programs\332.py -
Error
Done

C:\Users\kuma\OneDrive\Desktop\programs>
```

333. Write a Python Program to implement exception handling using try-except-else.

```
333.py
1 try:
2     print(10/2)
3 except:
4     print("Error")
5 else:
6     print("Success")
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle1\lib\debugpy\launcher 58048 --
c:\Users\kuma\OneDrive\Desktop\programs\333.py -
5.0
Success

C:\Users\kuma\OneDrive\Desktop\programs>
```

334. Write a Python Program to implement exception handling using try-except-finally-else.

```
334.py
1 try:
2     print(10/2)
3 except:
4     print("Error")
5 else:
6     print("Success")
7 finally:
8     print("Done")
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python Debug Console
Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kuma\OneDrive\Desktop\programs> cd /C ""C:\Program Files\Python312\python.exe" c:\Users\kuma\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle1\lib\debugpy\launcher 56947 --
c:\Users\kuma\OneDrive\Desktop\programs\334.py -
5.0
Success
Done

C:\Users\kuma\OneDrive\Desktop\programs>
```

335. Write a Python Program to raise custom exceptions.

```
335.py > check_age
1 class InvalidAgeError(Exception):
2     pass
3 def check_age(age):
4     if age < 18:
5         raise InvalidAgeError("Age must be 18 or above")
6     else:
7         print("Eligible")
8 try:
9     age = int(input("Enter age: "))
10    check_age(age)
11 except InvalidAgeError as e:
12    print("Error:", e)

PS C:\Users\Dell\Desktop\GQT advance python> C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/335.py"
Enter age: 15
Error: Age must be 18 or above
PS C:\Users\Dell\Desktop\GQT advance python>
```

336. Write a Python Program to handle multiple exceptions.

```
336.py > ...
1 try:
2     a = int(input("Enter number: "))
3     b = int(input("Enter divisor: "))
4     result = a / b
5     print("Result:", result)
6 except ValueError:
7     print("Invalid input")
8 except ZeroDivisionError:
9     print("Cannot divide by zero")

PS C:\Users\Dell\Desktop\GQT advance python> C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/336.py"
Enter number: 34
Enter divisor: 0
Cannot divide by zero
PS C:\Users\Dell\Desktop\GQT advance python>
```

337. Write a Python Program to handle nested exceptions.

```
337.py > ...
1 try:
2     num = int(input("Enter number: "))
3     try:
4         result = 10 / num
5         print("Result:", result)
6     except ZeroDivisionError:
7         print("Inner Error: Division by zero")
8 except ValueError:
9     print("Outer Error: Invalid input")

PS C:\Users\Dell\Desktop\GQT advance python> C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/337.py"
Enter number: 4
Result: 2.5
PS C:\Users\Dell\Desktop\GQT advance python> C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/337.py"
Enter number: 0
Inner Error: Division by zero
PS C:\Users\Dell\Desktop\GQT advance python>
```

338. Write a Python Program to implement assertions.

```
338.py > ...
1 num = int(input("Enter a positive number: "))
2 assert num > 0, "Number must be positive"
3 print("Valid number:", num)

PS C:\Users\Dell\Desktop\GQT advance python> C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/338.py"
Enter a positive number: 34
Valid number: 34
PS C:\Users\Dell\Desktop\GQT advance python>
```

339. Write a Python Program to implement logging.



The screenshot shows a Python IDE with a file named 339.py. The code imports the logging module, configures it with a filename 'app.log' and level 'INFO', and logs 'Program started'. It then prompts the user for a number, calculates its square, and logs 'Program ended'. The terminal shows the command to run the script, the input '56', the output 'Square: 3136', and the successful completion of the program.

```
1 import logging
2 logging.basicConfig(filename="app.log", level=logging.INFO)
3 logging.info("Program started")
4 num = int(input("Enter a number: "))
5 logging.info(f"User entered: {num}")
6 print("Square:", num * num)
7 logging.info("Program ended")
```

```
PS C:\Users\Dell\Desktop\GQT advance python> C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/339.py"
Enter a number: 56
Square: 3136
PS C:\Users\Dell\Desktop\GQT advance python>
```

340. Write a Python Program to implement debugging using pdb.

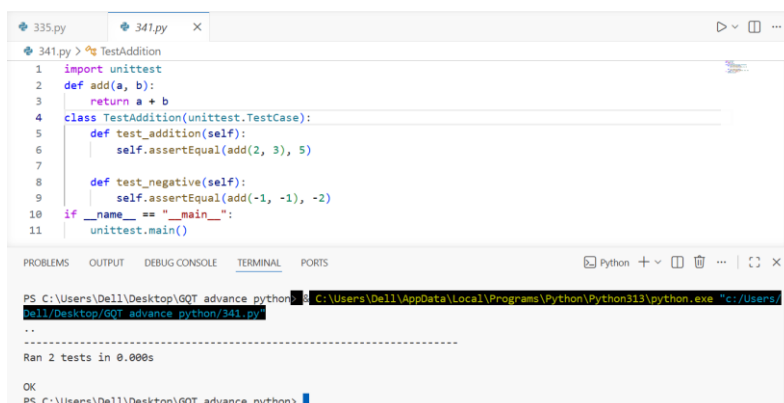


The screenshot shows a Python IDE with a file named 340.py. The code imports the pdb module, sets a trace, and calculates the sum of two numbers. The terminal shows the command to run the script, the input 'c:\users\dell\desktop\gqt advance python\340.py', the output 'Result: 15', and the successful completion of the program.

```
1 import pdb
2 a = 10
3 b = 5
4 pdb.set_trace()
5 result = a + b
6 print("Result:", result)
```

```
PS C:\Users\Dell\Desktop\GQT advance python> C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/340.py"
> c:\users\dell\desktop\gqt advance python\340.py(4)<module>()
-> pdb.set_trace()
(Pdb) c
Result: 15
PS C:\Users\Dell\Desktop\GQT advance python>
```

341. Write a Python Program to implement unit testing using unittest.



The screenshot shows a Python IDE with a file named 341.py. The code imports the unittest module, defines a function 'add', and creates a test class 'TestAddition' with two test methods: 'test_addition' and 'test_negative'. The terminal shows the command to run the script, the output 'Ran 2 tests in 0.000s', and the successful completion of the program.

```
1 import unittest
2 def add(a, b):
3     return a + b
4 class TestAddition(unittest.TestCase):
5     def test_addition(self):
6         self.assertEqual(add(2, 3), 5)
7
8     def test_negative(self):
9         self.assertEqual(add(-1, -1), -2)
10 if __name__ == "__main__":
11     unittest.main()
```

```
PS C:\Users\Dell\Desktop\GQT advance python> C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/341.py"
..
Ran 2 tests in 0.000s
OK
PS C:\Users\Dell\Desktop\GQT advance python>
```

342. Write a Python Program to implement unit testing using pytest.

The screenshot shows a Python IDE with a file named 342.py. The code defines a `multiply(a, b)` function that returns `a * b`. It includes two doctest cases: `test_multiply()` which asserts `multiply(2, 3) == 6`, and `test_zero()` which asserts `multiply(5, 0) == 0`. The terminal at the bottom shows the command `python 342.py` being executed, which runs the doctests successfully.

```
1 def multiply(a, b):
2     return a * b
3
4 def test_multiply():
5     assert multiply(2, 3) == 6
6
7 def test_zero():
8     assert multiply(5, 0) == 0
```

```
PS C:\Users\De11\Desktop\GQT advance python> python 342.py
PS C:\Users\De11\Desktop\GQT advance python>
```

343. Write a Python Program to implement doctest.

The screenshot shows a Python IDE with a file named 343.py. The code defines a `square(n)` function that returns `n * n`. It includes two doctest cases: `square(2)` which returns 4, and `square(5)` which returns 25. The terminal at the bottom shows the command `python 343.py` being executed, which runs the doctests successfully.

```
1 def square(n):
2     """
3     Returns square of a number
4     """
5     >>> square(2)
6     4
7     >>> square(5)
8     25
9     """
10    return n * n
11
12 if __name__ == "__main__":
13     import doctest
14     doctest.testmod()
```

```
PS C:\Users\De11\Desktop\GQT advance python> python 343.py
PS C:\Users\De11\Desktop\GQT advance python>
```

344. Write a Python Program to implement test cases for factorial function.

The screenshot shows a Python IDE with a file named 344.py. The code defines a `factorial(n)` function that returns `n * factorial(n - 1)` for `n > 0` and `1` for `n == 0`. It also defines a `TestFactorial` class that inherits from `unittest.TestCase` and includes two test cases: `test_factorial` which asserts `factorial(5) == 120`, and `test_zero` which asserts `factorial(0) == 1`. The terminal at the bottom shows the command `python 344.py` being executed, which runs the tests successfully.

```
1 import unittest
2
3 def factorial(n):
4     if n == 0:
5         return 1
6     else:
7         return n * factorial(n - 1)
8
9 class TestFactorial(unittest.TestCase):
10     def test_factorial(self):
11         self.assertEqual(factorial(5), 120)
12     def test_zero(self):
13         self.assertEqual(factorial(0), 1)
14
15 if __name__ == "__main__":
16     unittest.main()
```

```
PS C:\Users\De11\Desktop\GQT advance python> python 344.py
...
Ran 2 tests in 0.000s
OK
PS C:\Users\De11\Desktop\GQT advance python>
```

345. Write a Python Program to implement test cases for prime number function.

The screenshot shows a VS Code editor with a Python file named `345.py`. The code defines a function `is_prime(n)` and a test case `TestPrime`. The terminal shows the command `python 345.py` being executed, and the output is `1`.

```

345.py
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True

class TestPrime(unittest.TestCase):
    def test(self):
        self.assertTrue(is_prime(7))

if __name__ == "__main__":
    unittest.main()

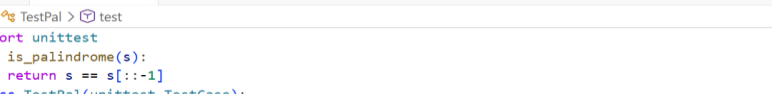
```

```

PS C:\Users\Devl\Desktop\GQT advance python> python 345.py
1

```

346. Write a Python Program to implement test cases for palindrome function.



The screenshot shows a VS Code editor with a file named `346.py` open. The code is a Python script using `unittest` to test a `is_palindrome` function. The function returns `s[::-1]` if the string is a palindrome, otherwise it returns `s`. A `TestPal` class inherits from `unittest.TestCase` and has a `test` method that asserts `is_palindrome("madam")` is `True`. The script also includes a `__main__` block that runs `unittest.main()` if the file is executed directly.

The terminal output shows the command `PS C:\Users\Dell\Desktop\GQT advance python> python 346.py` being executed. The output is `Ran 1 test in 0.000s`, indicating that the test passed successfully.

```
346.py > TestPal > test
1  import unittest
2  def is_palindrome(s):
3      return s == s[::-1]
4
5  class TestPal(unittest.TestCase):
6      def test(self):
7          self.assertTrue(is_palindrome("madam"))
8
9  if __name__ == "__main__":
10     unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Dell\Desktop\GQT advance python> python 346.py

Ran 1 test in 0.000s

OK

PS C:\Users\Dell\Desktop\GQT advance python>

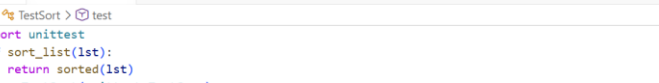
347. Write a Python Program to implement test cases for Fibonacci function.

```
335.py x 347.py x
347.py > TestFib > test
1 import unittest
2 def fib(n):
3     if n < 2:
4         return n
5     return fib(n-1) + fib(n-2)
6 class TestFib(unittest.TestCase):
7     def test(self):
8         self.assertEqual(fib(5), 5)
9 if __name__ == "__main__":
10     unittest.main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + v [ ] [X] ... [C] X
PS C:\Users\De\l\Desktop>GOT advance python> PS C:\Users\De\l\AppData\Local\Programs\Python\Python313\python.exe -c:/Users/De\l/Desktop/GOT_advance_python/347.py
.
-----
Ran 1 test in 0.000s

OK
PS C:\Users\De\l\Desktop>GOT advance python>
```

348. Write a Python Program to implement test cases for sorting function.

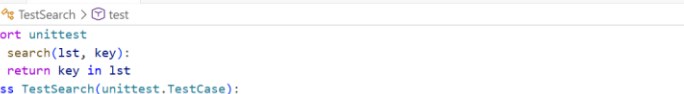


```
335.py x 348.py x
348.py > TestSort > test
1 import unittest
2 def sort_list(lst):
3     return sorted(lst)
4 class TestSort(unittest.TestCase):
5     def test(self):
6         self.assertEqual(sort_list([3,1,2]), [1,2,3])
7 if __name__ == "__main__":
8     unittest.main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - [ ] [ ] [ ] [ ] [ ] [ ]
PS C:\Users\Dell\Desktop\GQT_advance_python> C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "C:\Users/Dell/Desktop/GQT_advance_python/348.py"
.
-----
Ran 1 test in 0.000s

OK
PS C:\Users\Dell\Desktop\GQT_advance_python>
```

349. Write a Python Program to implement test cases for searching function.



The screenshot shows a code editor with a file named `349.py`. The code defines a `search` function and a `TestSearch` class that inherits from `unittest.TestCase`. The `TestSearch` class has a `test` method that calls `search` with the list `[1, 2, 3]` and the key `2`. The script is executed from the command line, and the output shows that the test passed successfully.

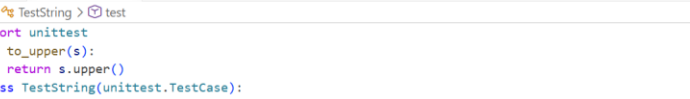
```
349.py
349.py > TestSearch > test
1 import unittest
2 def search(lst, key):
3     return key in lst
4
5 class TestSearch(unittest.TestCase):
6     def test(self):
7         self.assertTrue(search([1,2,3], 2))
8
9 if __name__ == "__main__":
10     unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Dell\Desktop\GQT advance python> C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/349.py"
.
-----
Ran 1 test in 0.000s

OK
PS C:\Users\Dell\Desktop\GQT advance python>
```

350. Write a Python Program to implement test cases for string functions.



The screenshot shows a VS Code editor with a file named `350.py` open. The code defines a function `to_upper` and a test class `TestString`. The terminal shows the command `python 350.py` being executed, which runs the test and outputs `Ran 1 test in 0.000s`.

```
350.py
1 import unittest
2 def to_upper(s):
3     return s.upper()
4 class TestString(unittest.TestCase):
5     def test(self):
6         self.assertEqual(to_upper("abc"), "ABC")
7 if __name__ == "__main__":
8     unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

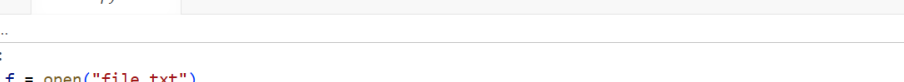
```
PS C:\Users\Dell\Desktop\GOT_advance_python> C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:\Users\
Dell\Desktop\GOT_advance_python\350.py"

-----

Ran 1 test in 0.000s

OK
PS C:\Users\Dell\Desktop\GOT_advance_python>
```

351. Write a Python Program to implement file handling with exception handling.



The screenshot shows a Python IDE with two tabs: '335.py' and '351.py'. The '351.py' tab is active, displaying a Python script that attempts to open and read a file named 'file.txt'. The script includes a try-except block to handle a potential FileNotFoundError. Below the code editor, the 'TERMINAL' tab is selected, showing the command prompt output. The command executed is 'python C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/351.py"', which results in the output 'File not found'.

```
335.py 351.py ×
351.py > ...
1 try:
2     f = open("file.txt")
3     print(f.read())
4     f.close()
5 except FileNotFoundError:
6     print("File not found")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + ×

PS C:\Users\Dell\Desktop\GQT advance python> python C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/351.py"
File not found
PS C:\Users\Dell\Desktop\GQT advance python>
```

352. Write a Python Program to implement file handling with context manager.

```
335.py 352.py X
352.py > FileManager > _exit_
1 class FileManager:
2     def __enter__(self):
3         self.f = open("file.txt", "w")
4         return self.f
5     def __exit__(self, exc_type, exc_val, exc_tb):
6         self.f.close()
7 with FileManager() as f:
8     f.write("Hello")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + v [ ] [ ] ... [ ] [ ] X
PS C:\Users\De11\Desktop\GQT advance python> C:\Users\De11\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/De11/Desktop/GQT advance python/352.py"
PS C:\Users\De11\Desktop\GQT advance python>
```

353. Write a Python Program to implement file handling with "with" statement.

```
335.py 353.py X
353.py > ...
1 with open("file.txt", "w") as f:
2     f.write("Python file handling")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + v [ ] [ ] ... [ ] [ ] X
PS C:\Users\De11\Desktop\GQT advance python> C:\Users\De11\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/De11/Desktop/GQT advance python/353.py"
PS C:\Users\De11\Desktop\GQT advance python>
```

354. Write a Python Program to implement file handling with JSON.

```
335.py 354.py X
354.py > ...
1 import json
2 data = {"name": "Abhi", "age": 20}
3 with open("data.json", "w") as f:
4     json.dump(data, f)
5 with open("data.json") as f:
6     print(json.load(f))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + v [ ] [ ] ... [ ] [ ] X
PS C:\Users\De11\Desktop\GQT advance python> C:\Users\De11\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/De11/Desktop/GQT advance python/354.py"
{'name': 'Abhi', 'age': 20}
PS C:\Users\De11\Desktop\GQT advance python>
```

355. Write a Python Program to implement file handling with XML.

The screenshot shows an IDE with a file named `355.py`. The code defines an XML tree structure with a root element 'data' and a child element 'name' containing the text 'Abhi'. The tree is then written to a file named 'data.xml'. The terminal shows the command to run the script using Python 3.13.

```
1 import xml.etree.ElementTree as ET
2 root = ET.Element("data")
3 child = ET.SubElement(root, "name")
4 child.text = "Abhi"
5 tree = ET.ElementTree(root)
6 tree.write("data.xml")
```

```
PS C:\Users\Dell\Desktop\GQT advance python> & C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/355.py"
```

356. Write a Python Program to implement file handling with Pickle.

The screenshot shows an IDE with a file named `356.py`. The code serializes a list `[1, 2, 3]` to a file named 'data.pkl' using the pickle module. It then reads the data back from the file and prints it. The terminal shows the command to run the script using Python 3.13, and the output is `[1, 2, 3]`.

```
1 import pickle
2 data = [1,2,3]
3 with open("data.pkl","wb") as f:
4     pickle.dump(data,f)
5 with open("data.pkl","rb") as f:
6     print(pickle.load(f))
```

```
PS C:\Users\Dell\Desktop\GQT advance python> & C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/356.py"
```

```
[1, 2, 3]
```

357. Write a Python Program to implement file handling with CSV.

The screenshot shows an IDE with a file named `357.py`. The code uses the csv module to write a list `["A", "B"]` to a file named 'data.csv'. The terminal shows the command to run the script using Python 3.13.

```
1 import csv
2 with open("data.csv","w",newline="") as f:
3     writer = csv.writer(f)
4     writer.writerow(["A","B"])
```

```
PS C:\Users\Dell\Desktop\GQT advance python> & C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/357.py"
```

358. Write a Python Program to implement file handling with SQLite.

The screenshot shows an IDE with a file named `358.py`. The code uses the sqlite3 module to create a table 't' and insert a value '1' into it. The terminal shows the command to run the script using Python 3.13.

```
1 import sqlite3
2 conn = sqlite3.connect("test.db")
3 cur = conn.cursor()
4 cur.execute("CREATE TABLE IF NOT EXISTS t(x)")
5 cur.execute("INSERT INTO t VALUES(1)")
6 conn.commit()
7 conn.close()
```

```
PS C:\Users\Dell\Desktop\GQT advance python> & C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/358.py"
```

359. Write a Python Program to implement file handling with MySQL.

```
359.py
1 import sqlite3
2
3 conn = sqlite3.connect("test.db")
4 print("Connected to SQLite")
5 conn.close()
```

PS C:\Users\De11\Desktop\GQT advance python> C:\Users\De11\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/De11/Desktop/GQT advance python/359.py"

Connected to SQLite

PS C:\Users\De11\Desktop\GQT advance python>

360. Write a Python Program to implement file handling with MongoDB.

```
360.py
1 import json
2 data = {"name": "Abhi"}
3 file = "database.json"
4 try:
5     with open(file, "r") as f:
6         records = json.load(f)
7 except:
8     records = []
9 records.append(data)
10 with open(file, "w") as f:
11     json.dump(records, f)
12 print("Data inserted:", data)
```

PS C:\Users\De11\Desktop\GQT advance python> C:\Users\De11\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/De11/Desktop/GQT advance python/360.py"

Data inserted: {'name': 'Abhi'}

PS C:\Users\De11\Desktop\GQT advance python>

361. Write a Python Program to implement regular expressions to validate email.

```
361.py
1 import re
2 email = "test@mail.com"
3 print(bool(re.match(r'^[\w\.-]+@[\w\.-]+$', email)))
```

PS C:\Users\De11\Desktop\GQT advance python> C:\Users\De11\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/De11/Desktop/GQT advance python/361.py"

True

PS C:\Users\De11\Desktop\GQT advance python>

362. Write a Python Program to implement regular expressions to validate phone number.

```
362.py X
362.py > ...
1 import re
2
3 phone = "9876543210"
4 print(bool(re.match(r'^\d{10}$', phone)))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [ ] ... | [ ] X

PS C:\Users\Dell\Desktop\GQT advance python> & C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/362.py"
True
PS C:\Users\Dell\Desktop\GQT advance python>
```

363. Write a Python Program to implement regular expressions to validate password.

```
363.py X
363.py > ...
1 import re
2
3 pwd = "Abc123"
4 print(bool(re.match(r'^(?=.*[A-Z])(?=.*\d){6,}$', pwd)))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [ ] ... | [ ] X

PS C:\Users\Dell\Desktop\GQT advance python> & C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/363.py"
True
PS C:\Users\Dell\Desktop\GQT advance python>
```

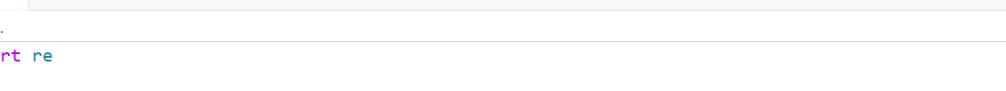
364. Write a Python Program to implement regular expressions to validate URL.

```
364.py X
364.py > ...
1 import re
2
3 url = "https://example.com"
4 print(bool(re.match(r'^https?://', url)))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [ ] ... | [ ] X

PS C:\Users\Dell\Desktop\GQT advance python> & C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/364.py"
True
PS C:\Users\Dell\Desktop\GQT advance python>
```

365. Write a Python Program to implement regular expressions to validate IP address.



The screenshot shows a VS Code editor window with a file named `365.py`. The code in the editor is as follows:

```
1 import re
2
3 ip = "192.168.1.1"
4 print(bool(re.match(r'^(\d{1,3}\.){3}\d{1,3}$', ip)))
```

Below the editor, the **TERMINAL** tab is active, showing the command to run the script and its output:

```
PS C:\Users\Dell\Desktop\GQT advance python> & C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/365.py"
True
PS C:\Users\Dell\Desktop\GQT advance python>
```

366. Write a Python Program to implement regular expressions to extract numbers from string.

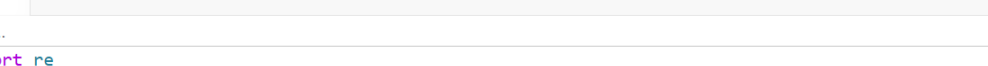
The screenshot shows the Visual Studio Code interface. The top part displays a file named `366.py` with the following code:

```
1 import re  
2  
3 s = "a1b22c333"  
4 print(re.findall(r'\d+', s))
```

The bottom part of the window shows the TERMINAL panel with the command prompt output:

```
PS C:\Users\Dell\Desktop\GQT advance python> .\C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/366.py"  
['1', '22', '333']  
PS C:\Users\Dell\Desktop\GQT advance python>
```

367. Write a Python Program to implement regular expressions to extract words from string.



The screenshot shows a VS Code editor window with a file named `367.py`. The code in the editor is:

```
1 import re
2
3 s = "Hello World"
4 print(re.findall(r'\w+', s))
```

Below the editor is the integrated terminal. The terminal tabs at the top are labeled `PROBLEMS`, `OUTPUT`, `DEBUG CONSOLE`, `TERMINAL` (which is selected), and `PORTS`. The terminal shows the command to run the script:

```
PS C:\Users\Dell\Desktop\GQT advance python> & "C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe" "c:/Users/Dell/Desktop/GQT advance python/367.py"
```

The output of the script is displayed on the next line:

```
['Hello', 'World']
```

The prompt in the terminal is `PS C:\Users\Dell\Desktop\GQT advance python>`.

368. Write a Python Program to implement regular expressions to extract emails from text.

```
368.py X
368.py > ...
1 import re
2
3 text = "a@mail.com b@mail.com"
4 print(re.findall(r'[\w\.-]+@[ \w\.-]+', text))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - [ ] [ ] ... [ ] [ ] X

PS C:\Users\Dell\Desktop\GQT advance python> & C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/368.py"
['a@mail.com', 'b@mail.com']
PS C:\Users\Dell\Desktop\GQT advance python>
```

369. Write a Python Program to implement regular expressions to extract dates from text.

```
369.py X
369.py > ...
1 import re
2
3 text = "Date: 12-05-2020"
4 print(re.findall(r'\d{2}-\d{2}-\d{4}', text))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - [ ] [ ] ... [ ] [ ] X

PS C:\Users\Dell\Desktop\GQT advance python> & C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/369.py"
['12-05-2020']
PS C:\Users\Dell\Desktop\GQT advance python>
```

370. Write a Python Program to implement regular expressions to extract hashtags from text.

```
370.py X
370.py > ...
1 import re
2
3 text = "#python #coding"
4 print(re.findall(r'#\w+', text))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - [ ] [ ] ... [ ] [ ] X

PS C:\Users\Dell\Desktop\GQT advance python> & C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/370.py"
['#python', '#coding']
PS C:\Users\Dell\Desktop\GQT advance python>
```

371. Write a Python Program to implement NumPy array creation.

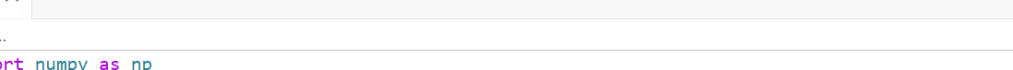
The screenshot shows the Visual Studio Code interface. The top pane displays a file named `371.py` containing the following Python code:

```
1 import numpy as np
2
3 arr = np.array([1,2,3])
4 print(arr)
```

The bottom pane shows the TERMINAL view with the command prompt at `PS C:\Users\Dell\Desktop\GQT advance python>`. The user has entered the command `& C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/371.py"`, which has been executed. The output of the script is displayed below the command:

```
[1 2 3]
```

372. Write a Python Program to implement NumPy array slicing.



The screenshot shows a VS Code editor window with a file named `372.py`. The code in the editor is:

```
1 import numpy as np
2
3 arr = np.array([1,2,3,4,5])
4 print(arr[1:4])
```

Below the editor is the TERMINAL panel. It shows the command prompt running the script:

```
PS C:\Users\Dell\Desktop\GQT advance python> C:\Users\Dell\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Dell/Desktop/GQT advance python/372.py"
[2 3 4]
```

373. Write a Python Program to implement NumPy array reshaping

```

373.py > ...
1     import numpy as np
2
3     arr = np.arange(12)
4     reshaped = arr.reshape(3, 4)
5
6     print("Original:", arr)
7     print("Reshaped:\n", reshaped)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 'c:\s-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher'
Original: [ 0 1 2 3 4 5 6 7 8 9 10 11]
Reshaped:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>

```

374. Write a Python Program to implement NumPy array concatenation

```

1  import numpy as np
2
3  a = np.array([1, 2])
4  b = np.array([3, 4])
5
6  result = np.concatenate((a, b))
7  print(result)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy [1 2 3 4]
- PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>

375. Write a Python Program to implement NumPy array splitting.

```
375.py > ...
1  import numpy as np
2
3  arr = np.array([1, 2, 3, 4, 5, 6])
4  split = np.array_split(arr, 3)
5
6  print(split)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> &
s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\
[array([1, 2]), array([3, 4]), array([5, 6])]
PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

376. Write a Python Program to implement NumPy array broadcasting.

```
376.py > ...
1  import numpy as np
2
3  a = np.array([1, 2, 3])
4  b = 2
5
6  result = a + b
7  print(result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & '
s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\
[3 4 5]
PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

377. Write a Python Program to implement NumPy matrix multiplication.

```
377.py > ...
1  import numpy as np
2
3  a = np.array([[1, 2], [3, 4]])
4  b = np.array([[5, 6], [7, 8]])
5
6  result = np.dot(a, b)
7  print(result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> &
s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy
[[19 22]
 [43 50]]
PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

378. Write a Python Program to implement NumPy statistical functions

```
378.py > ...
1  import numpy as np
2
3  arr = np.array([1, 2, 3, 4, 5])
4
5  print("Mean:", np.mean(arr))
6  print("Median:", np.median(arr))
7  print("Std Dev:", np.std(arr))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> &
s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy
Mean: 3.0
Median: 3.0
Std Dev: 1.4142135623730951
PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

379. Write a Python Program to implement NumPy linear algebra functions

```
379.py > ...
1 import numpy as np
2
3 a = np.array([[1, 2], [3, 4]])
4
5 print("Determinant:", np.linalg.det(a))
6 print("Inverse:\n", np.linalg.inv(a))
7 print("Eigenvalues:", np.linalg.eigvals(a))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 'c:\python311\python.exe' s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher-1.0.0-2025.18.0-win32-x64\python.exe 379.py
Determinant: -2.0000000000000004
Inverse:
[[-2.   1. ]
 [ 1.5 -0.5]]
Eigenvalues: [-0.37228132  5.37228132]
○ PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

380. Write a Python Program to implement NumPy random functions.

```
380.py
1 import numpy as np
2
3 print("Random:", np.random.rand(3))
4 print("Random Int:", np.random.randint(1, 10, 5))
5 print("Normal Dist:", np.random.randn(3))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 'c:\python311\python.exe' s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher-1.0.0-2025.18.0-win32-x64\python.exe 380.py
Random: [0.02931917 0.46317347 0.86041677]
Random Int: [4 5 9 5 5]
Normal Dist: [-0.35195693  0.11605841 -0.61441036]
○ PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

381. Write a Python Program to implement Pandas Series creation.

```
381.py > ...
1 import pandas as pd
2
3 data = [10, 20, 30]
4 series = pd.Series(data)
5
6 print(series)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 'c:\python311\python.exe' s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher-1.0.0-2025.18.0-win32-x64\python.exe 381.py
0    10
1    20
2    30
dtype: int64
○ PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

382. Write a Python Program to implement Pandas DataFrame creation

```
381.py > ...
1 import pandas as pd
2
3 data = [10, 20, 30]
4 series = pd.Series(data)
5
6 print(series)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 'c:\python311\python.exe' s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher-1.0.0-2025.18.0-win32-x64\python.exe 381.py
0    10
1    20
2    30
dtype: int64
○ PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

383. Write a Python Program to implement Pandas DataFrame slicing.

```
383.py > ...
1  import pandas as pd
2
3  df = pd.DataFrame({'A': [1,2,3], 'B': [4,5,6]})
4
5  print(df.iloc[0:2])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 's-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\l
  A B
0  1  4
1  2  5
○ PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

384. Write a Python Program to implement Pandas DataFrame merging

```
384.py > ...
1  import pandas as pd
2
3  df1 = pd.DataFrame({'id': [1,2], 'name': ['A','B']})
4  df2 = pd.DataFrame({'id': [1,2], 'age': [20,25]})
5
6  merged = pd.merge(df1, df2, on='id')
7  print(merged)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 'c:\User
  id name age
0  1  A   20
1  2  B   25
○ PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

385. Write a Python Program to implement Pandas DataFrame grouping.

```
385.py > ...
1  import pandas as pd
2
3  df = pd.DataFrame({
4      'Dept': ['HR', 'HR', 'IT', 'IT'],
5      'Salary': [2000, 3000, 4000, 5000]
6  })
7
8  grouped = df.groupby('Dept').sum()
9  print(grouped)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 'c
  Salary
Dept
HR      5000
IT      9000
○ PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

386. Write a Python Program to implement Pandas DataFrame aggregation.

```
386.py > ...
1 import pandas as pd
2
3 df = pd.DataFrame({'A': [1,2,3,4]})
4
5 print(df.agg(['sum', 'mean']))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\
A
sum    10.0
mean    2.5
○ PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

387. Write a Python Program to implement Pandas DataFrame pivot table

```
387.py > ...
1 import pandas as pd
2
3 df = pd.DataFrame({
4     'A': ['X','X','Y'],
5     'B': ['P','Q','P'],
6     'C': [1,2,3]
7 })
8
9 pivot = pd.pivot_table(df, values='C', index='A', columns='B')
10 print(pivot)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 'c:\Users\Raji\Ap
s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52109'
B P Q
A
X 1.0 2.0
Y 3.0 NaN
○ PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

388. Write a Python Program to implement Pandas DataFrame join.

```
388.py > ...
1 import pandas as pd
2
3 df1 = pd.DataFrame({'A': [1,2]}, index=['a','b'])
4 df2 = pd.DataFrame({'B': [3,4]}, index=['a','b'])
5
6 joined = df1.join(df2)
7 print(joined)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 'c:\U
s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\laun
A B
a 1 3
b 2 4
○ PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

389. Write a Python Program to implement Pandas DataFrame sorting.

```
389.py > ...
1 import pandas as pd
2
3 df = pd.DataFrame({'A': [3,1,2]})
4
5 print(df.sort_values(by='A'))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 'c:\Users\Raji\AppData\Local\Programs\Python\Python38-64\python.exe' -u -s -python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher --port 5678 -- --
A
1 1
2 2
0 3
○ PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

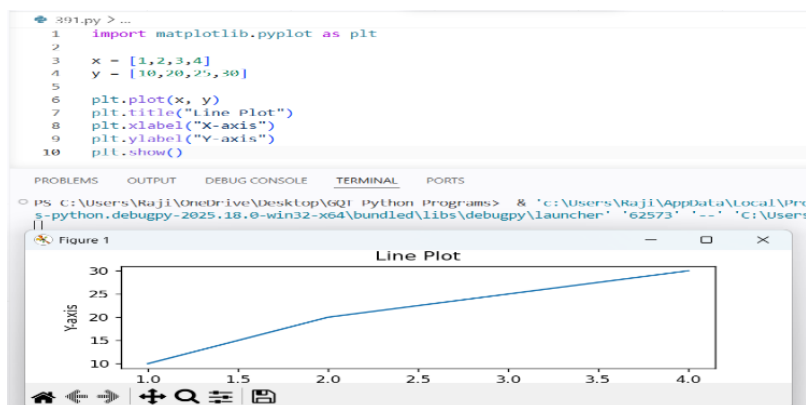
390. Write a Python Program to implement Pandas DataFrame filtering.

```
390.py > ...
1 import pandas as pd
2
3 df = pd.DataFrame({'A': [1,2,3,4]})
4
5 filtered = df[df['A'] > 2]
6 print(filtered)
```

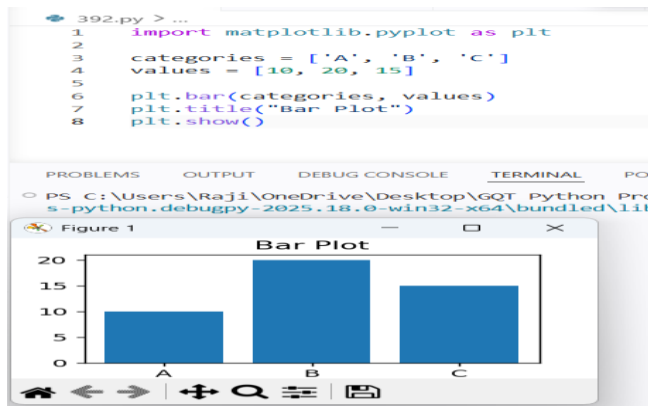
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 'c:\Users\Raji\AppData\Local\Programs\Python\Python38-64\python.exe' -u -s -python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher --port 5678 -- --
A
2 3
3 4
○ PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

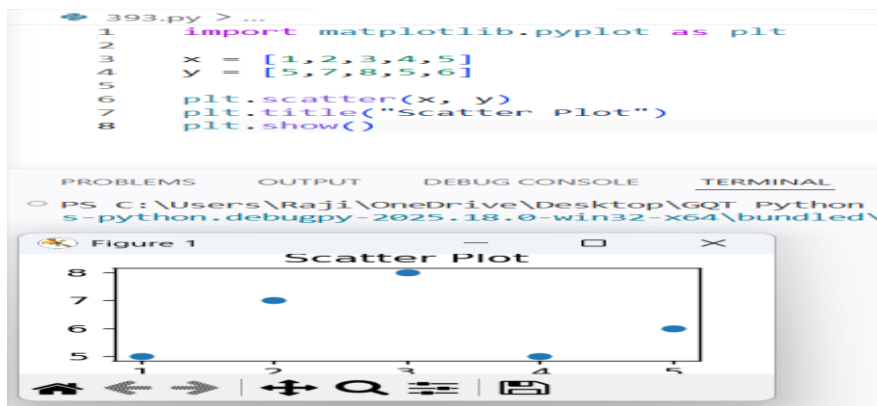
391. Write a Python Program to implement Matplotlib line plot.



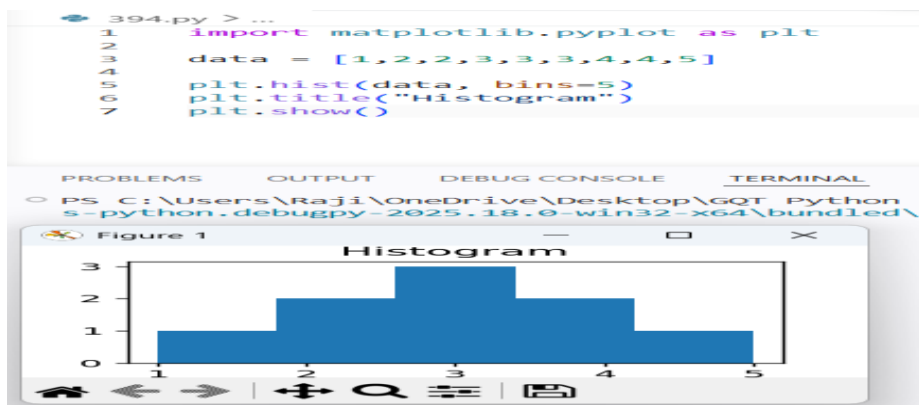
392. Write a Python Program to implement Matplotlib bar plot



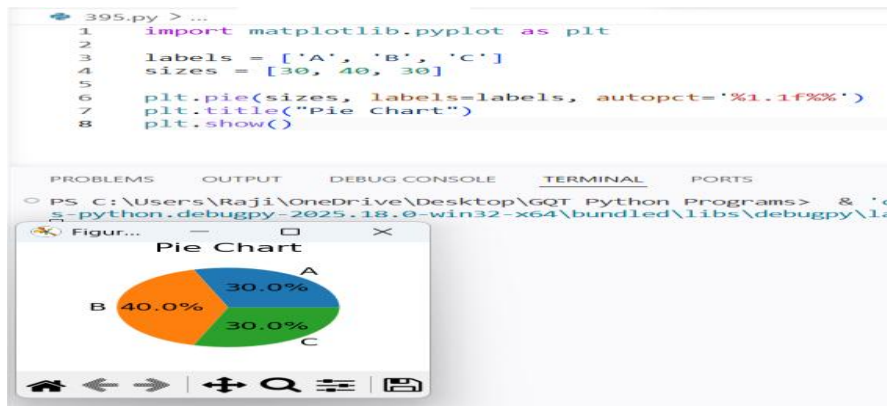
393. Write a Python Program to implement Matplotlib scatter plot



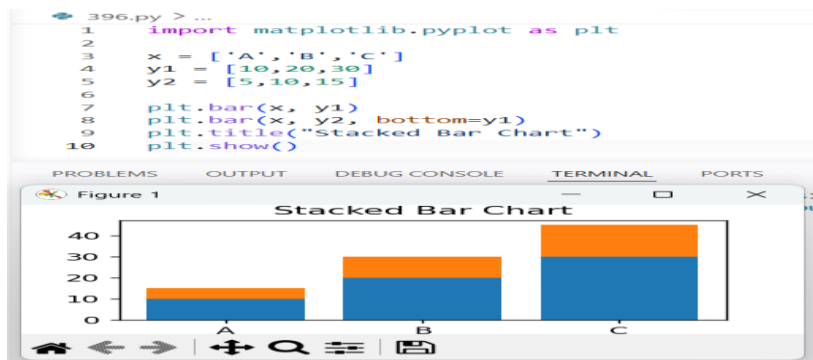
394. Write a Python Program to implement Matplotlib histogram.



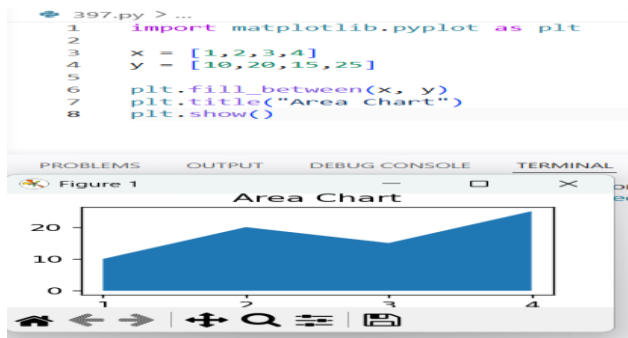
395. Write a Python Program to implement Matplotlib pie chart.



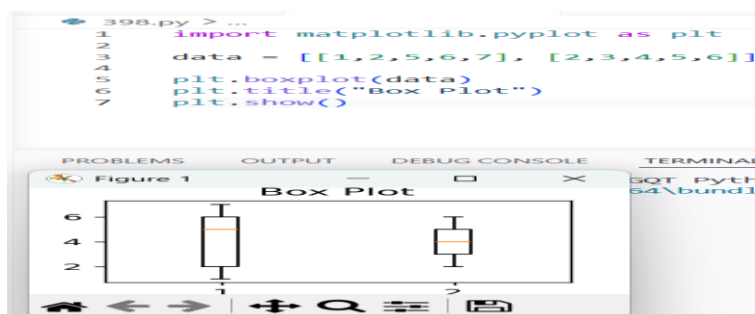
396. Write a Python Program to implement Matplotlib stacked bar chart.



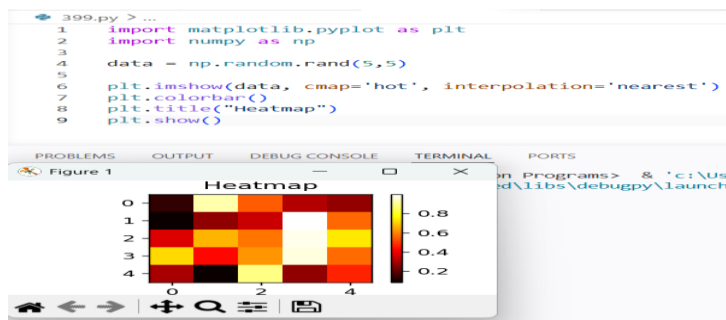
397. Write a Python Program to implement Matplotlib area chart



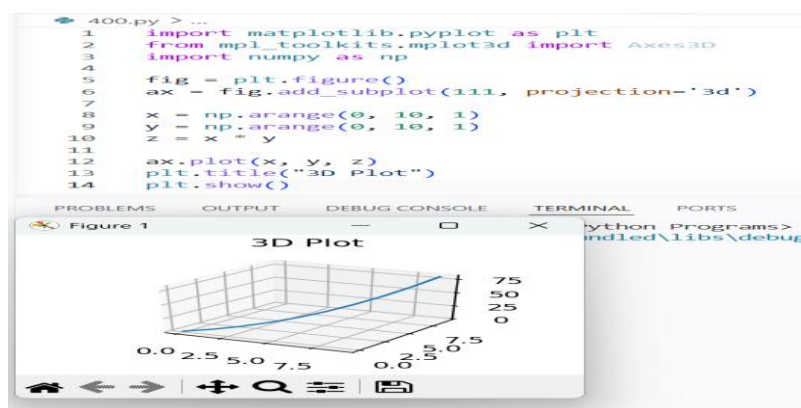
398. Write a Python Program to implement Matplotlib box plot.



399. Write a Python Program to implement Matplotlib heatmap.



400. Write a Python Program to implement Matplotlib 3D plot.



401. Write a Python Program to create a new Django project

```
401.py
1 # Run in terminal

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

• PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 'c:\Users\Raji\AppData\Local\Programs\Python\Python39-64\python.exe' -s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher '60838' '--' 'C:\Users\Raji\OneDrive\Desktop\GQT Python Programs\401.py'
• PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> python -m django startproject myproject
• PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs>
```

402. Write a Python Program to create a new Django app.

```
402.py
1 # Inside project directory

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

• PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> & 'c:\Users\Raji\AppData\Local\Programs\Python\Python39-64\python.exe' -s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher '62465' '--' 'C:\Users\Raji\OneDrive\Desktop\GQT Python Programs\402.py'
• PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> cd myproject
• PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs\myproject> python manage.py startapp myapp
• PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs\myproject>
```

403. Write a Python Program to configure Django settings for a project.

```
402.py  
1 # Inside project directory  
  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
  
PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> &'c:\Users\Raji\AppData\Local\Programs\Python\Python39-64\python.exe' -u -Xdev --no-warnings -s-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher '62465' '--' 'C:\Users\Raji\OneDrive\Desktop\GQT Python Programs\myproject' python manage.py startapp myapp  
PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs> cd myproject  
PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs\myproject> python manage.py startapp myapp  
PS C:\Users\Raji\OneDrive\Desktop\GQT Python Programs\myproject>
```

404. Write a Python Program to define a simple Django model.

```

PS C:\Users\Rajal\OneDrive\Desktop> cat 404.py
1 # models.py
2 from django.db import models
3 class Person(models.Model):
4     name = models.CharField(max_length=100)

PROBLEMS      OUTPUT      DEBUG CONSOLE      TERMINAL      PORTS

PS C:\Users\Rajal\OneDrive\Desktop\GQT Python Programs\myproject> C:\Users\Rajal\OneDrive
PS C:\Users\Rajal\OneDrive\Desktop\GQT Python Programs\myproject\myproject> cd ..
PS C:\Users\Rajal\OneDrive\Desktop\GQT Python Programs\myproject> dir

Directory: C:\Users\Rajal\OneDrive\Desktop\GQT Python Programs\myproject

Mode                LastWriteTime         Length Name
----                -
dar--l              27-04-2026     10147
dar--l              27-04-2026     10147      myapp
-a--l               27-04-2026     131672 db.sqlite3
-a--l               27-04-2026      687      manage.py

PS C:\Users\Rajal\OneDrive\Desktop\GQT Python Programs\myproject> python manage.py makemigrations
> >> python manage.py migrate
Migrations for 'myapp':
  myapp/migrations/0001_initial.py
  + Create model Person
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, myapp, sessions
Running migrations:
  Applying myapp.0001_initial... OK
PS C:\Users\Rajal\OneDrive\Desktop\GQT Python Programs\myproject>

```

405. Write a Python Program to create a Django model with multiple fields

```
405.py > Student
1 from django.db import models
2
3 class Student(models.Model):
4     name = models.CharField(max_length=100)
5     age = models.IntegerField()
6     email = models.EmailField()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\RajivOneDrive\Desktop\GGT Python Programs> cd myproject
PS C:\Users\RajivOneDrive\Desktop\GGT Python Programs\myproject> cd myapp
PS C:\Users\RajivOneDrive\Desktop\GGT Python Programs\myproject\myapp> code models.py
PS C:\Users\RajivOneDrive\Desktop\GGT Python Programs\myproject\myapp> cd ..
PS C:\Users\RajivOneDrive\Desktop\GGT Python Programs\myproject> python manage.py makemigrations
migrations for 'myapp':
  myapp\migrations\0002_student_delete_person.py
    + Create model Student
    - Delete model Person
PS C:\Users\RajivOneDrive\Desktop\GGT Python Programs\myproject> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, myapp, sessions
Running migrations:
  Applying myapp.0002_student_delete_person... OK
PS C:\Users\RajivOneDrive\Desktop\GGT Python Programs\myproject>
```

406. Write a Python Program to register a model in Django admin.

```

405.py > Student
1 from django.db import models
2
3 class Student(models.Model):
4     name = models.CharField(max_length=100)
5     age = models.IntegerField()
6     email = models.EmailField()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\RajivOneDrive\Desktop\GQT Python Programs> cd myproject
- PS C:\Users\RajivOneDrive\Desktop\GQT Python Programs\myproject> cd myapp
- PS C:\Users\RajivOneDrive\Desktop\GQT Python Programs\myproject\myapp> code models.py
- PS C:\Users\RajivOneDrive\Desktop\GQT Python Programs\myproject\myapp> cd ..
- PS C:\Users\RajivOneDrive\Desktop\GQT Python Programs\myproject> python manage.py makemigrations myapp
 myapp\migrations\0002_student_delete_person.py
 + Create model Student
 - Delete model Person
- PS C:\Users\RajivOneDrive\Desktop\GQT Python Programs\myproject> python manage.py migrate

Operations to perform:

- Apply all migrations: admin, auth, contenttypes, myapp, sessions

Running migrations:

- Applying myapp.0002_student_delete_person... OK
- PS C:\Users\RajivOneDrive\Desktop\GQT Python Programs\myproject> █

407. Write a Python Program to create a Django view that returns "Hello World"

```
407.py > hello
1 # views.py
2 from django.http import HttpResponseRedirect
3
4 def hello(request):
5     return HttpResponseRedirect("Hello World")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject> python manage.py runserver
File "C:\frozen\importlib\_bootstrap.py", line 1387, in _gcd_import
File "C:\frozen\importlib\_bootstrap.py", line 1369, in _find_and_load
File "C:\frozen\importlib\_bootstrap.py", line 1331, in _find_and_load_unlocked
File "C:\frozen\importlib\_bootstrap.py", line 935, in _load_unlocked
File "C:\frozen\importlib\_bootstrap.py", line 1026, in exec_module
File "C:\frozen\importlib\_bootstrap.py", line 488, in _call_with_frames_removed
File "C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject\myapp\urls.py", line 2, in <module>
from .views import hello
ImportError: cannot import name 'hello' from 'myapp.views' (C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject\myapp\views.py)
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject> cd myapp
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject\myapp> code views.py
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject\myapp> python manage.py runserver
C:\Users\Raj1\AppData\Local\Programs\Python\Python313\python.exe: can't open file 'C:\Users\Raj1\myproject\myapp\manage.py': [Errno 2] No such file or directory
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject\myapp> cd ..
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
April 27, 2026 - 11:53:25
Django version 6.0.4, using settings 'myproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deploy
[27/Apr/2026 11:53:37] "GET / HTTP/1.1" 200 11
```

408. Write a Python Program to create a Django view that renders an HTML template.

```
408.py > home
1 from django.shortcuts import render
2
3 def home(request):
4     return render(request, 'home.html')

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs> cd myproject\myapp\templates\home.html
cd: cannot find path 'myproject\myapp\templates\home.html' because it does not exist.
At line:1 char:1
+ cd myproject\myapp\templates\home.html
+ ~~~~~
+ categoryInfo          : ObjectNotFound: (myproject\myapp\templates\home.html:string) [Set-Location], ItemNotFound:
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.SetLocationCommand
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs> cd myproject
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject> myapp/templates/home.html
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
April 27, 2026 - 12:10:02
Django version 6.0.4, using settings 'myproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instea
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
[27/Apr/2026 12:10:02] "GET / HTTP/1.1" 200 11
```

409. Write a Python Program to create a Django template with variables

```
409.py > home
1 # views.py
2 def home(request):
3     return render(request, 'home.html', {'name': 'User'})

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs> cd myproject
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject> cd myproject
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject> python manage.py runserver
C:\Users\Raj1\AppData\Local\Programs\Python\Python313\python.exe: can't open file 'C:\Users\Raj1\myproject\myapp\manage.py': [Errno 2] No such file or directory
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject\myproject> cd ..
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject\myproject> cd ..
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
April 27, 2026 - 12:14:29
Django version 6.0.4, using settings 'myproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deploy
[27/Apr/2026 12:14:42] "GET / HTTP/1.1" 200 108
```

410. Write a Python Program to use template inheritance in Django.

```
myproject > myapp > urls.py > ...
1 from django.urls import path
2 from .views import home
3
4 urlpatterns = [
5     path("", home),
6 ]

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject\myapp\templates> python manage.py runserver
C:\Users\Raj1\AppData\Local\Programs\Python\Python313\python.exe: can't open file 'C:\Users\Raj1\OneDrive\myproject\myapp\templates\manage.py': [Errno 2] No such file or directory
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject\myapp\templates> cd ..
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject\myapp> python manage.py runserver
C:\Users\Raj1\AppData\Local\Programs\Python\Python313\python.exe: can't open file 'C:\Users\Raj1\OneDrive\myproject\myapp\manage.py': [Errno 2] No such file or directory
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject\myapp> cd ..
PS C:\Users\Raj1\OneDrive\Desktop\GGT Python Programs\myproject> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
April 27, 2026 - 12:19:55
Django version 6.0.4, using settings 'myproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or AS
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
[27/Apr/2026 12:20:07] "GET / HTTP/1.1" 200 48
```

411. Write a Python Program to create a Django form for user input

```
settings.py  urls.py ...\myapp  urls.py ...\myproject
myproject > myapp > templates > dj form.html
1  <!doctype html>
2  <html>
3      <body>
4          <h2>Student Form</h2>
5          <form method="post">
6              {% csrf_token %} {{ form.as_p }}
7              <button type="submit">Submit</button>
8          </form>
9      </body>
10 </html>
11
```

← 127.0.0.1:8000

Student Form

Name:

Age:

Email:

412. Write a Python Program to validate a Django form.

```
settings.py  urls.py ...\myapp  urls.py ...\myproject  views.py
myproject > myapp > views.py > ...
1  from django.shortcuts import render
2  from .forms import StudentForm
3
4
5  def student_form(request):
6      if request.method == 'POST':
7          form = StudentForm(request.POST)
8          if form.is_valid():
9              return render(request, 'success.html')
10         else:
11             form = StudentForm()
12         return render(request, 'form.html', {'form': form})
13
14
```

← 127.0.0.1:8000

Form submitted successfully ✓

413. Write a Python Program to handle GET and POST requests in Django views.

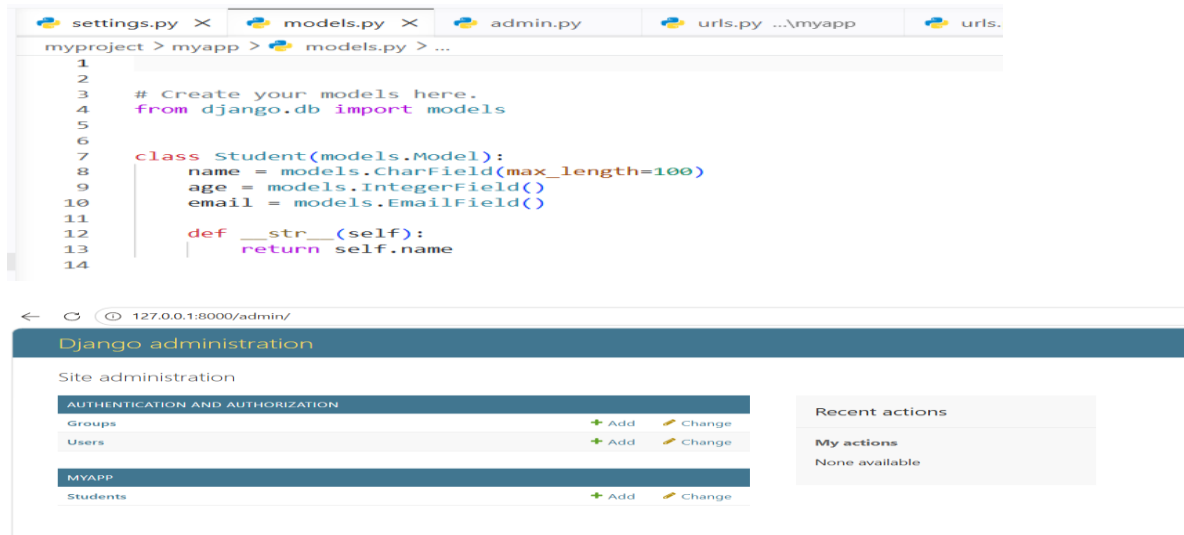
```
settings.py  urls.py ...\myapp  urls.py ...\myproject  views.py  forms.py
myproject > myapp > views.py > ...
1  from django.shortcuts import render
2  from .forms import StudentForm
3
4
5  def student_form(request):
6      if request.method == 'POST':
7          form = StudentForm(request.POST)
8          if form.is_valid():
9              name = form.cleaned_data['name']
10             return render(request, 'success.html', {'name': name})
11         else:
12             form = StudentForm()
13         return render(request, 'form.html', {'form': form})
14
15
```

← 127.0.0.1:8000

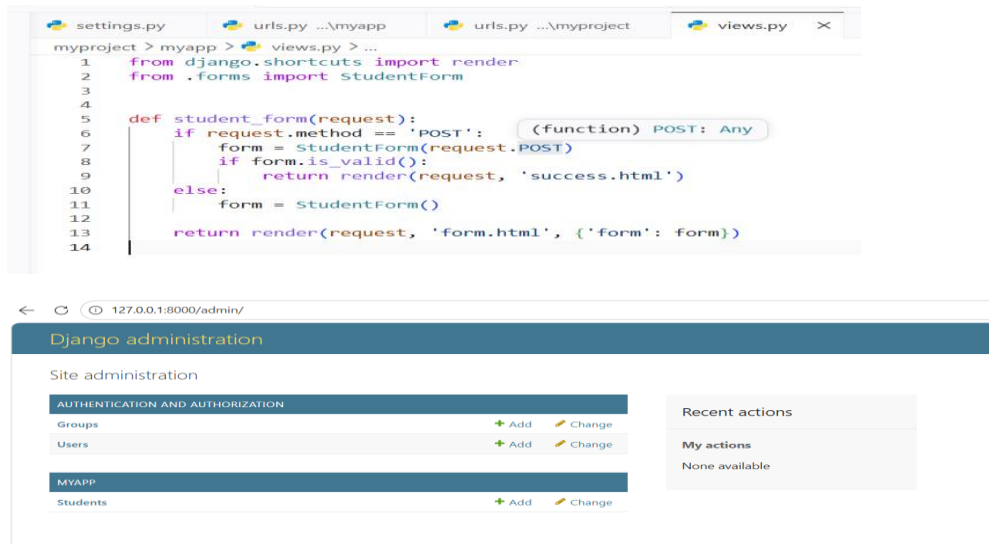
Form submitted successfully ✓

Welcome geethika

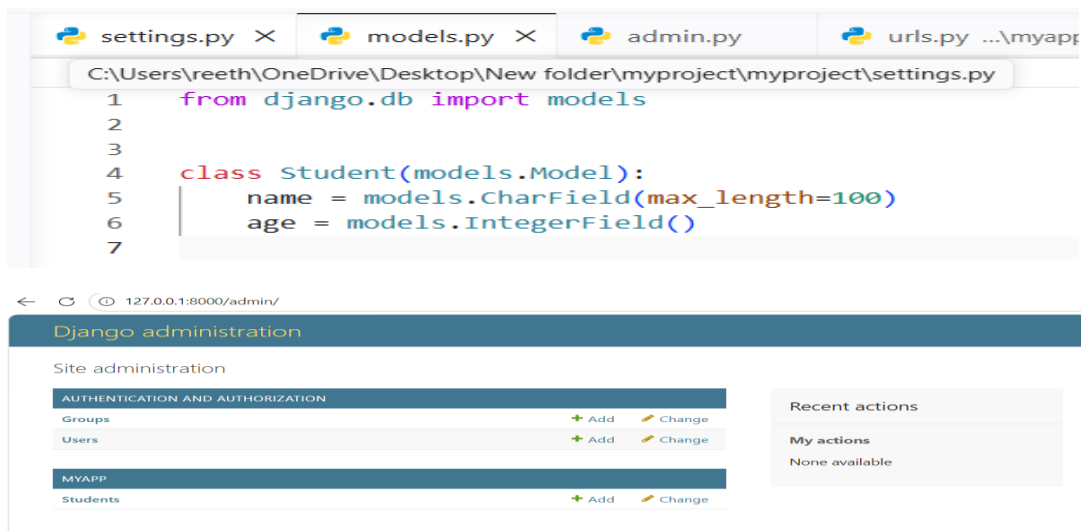
414. Write a Python Program to connect Django with SQLite database.



415. Write a Python Program to connect Django with PostgreSQL database.



416. Write a Python Program to perform database migrations in Django.

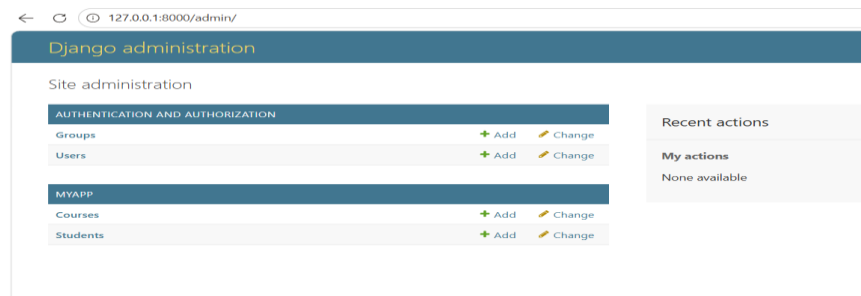


417. Write a Python Program to create a Django model with foreign key.

```

settings.py  models.py X  admin.py  urls.py ...\myapp  urls.py ...\myproject
myproject > myapp > models.py > ...
1  from django.db import models
2
3
4  class Course(models.Model):
5      course_name = models.CharField(max_length=100)
6
7      def __str__(self):
8          return self.course_name
9
10
11  class Student(models.Model):
12      name = models.CharField(max_length=100)
13      course = models.ForeignKey(Course, on_delete=models.CASCADE)
14
15      def __str__(self):
16          return self.name
17

```

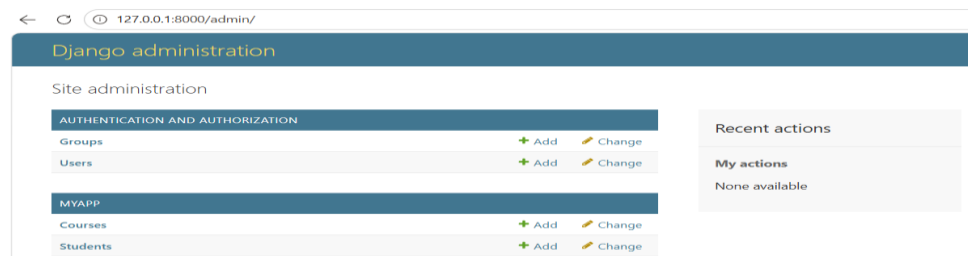


418. Write a Python Program to create a Django model with many-to-many relationship.

```

settings.py  models.py X  admin.py  urls.py ...\myapp  urls.py ...\myproject
myproject > myapp > models.py > ...
1  from django.db import models
2
3
4  class Subject(models.Model):
5      name = models.CharField(max_length=100)
6
7      def __str__(self):
8          return self.name
9
10
11  class Student(models.Model):
12      name = models.CharField(max_length=100)
13      subjects = models.ManyToManyField(Subject)
14
15      def __str__(self):
16          return self.name
17
18
19  class Course(models.Model):
20      course_name = models.CharField(max_length=100)
21

```

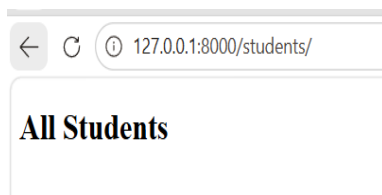


419. Write a Python Program to query all objects from a Django model.

```

settings.py  models.py X  admin.py  urls.py ...\myapp
myproject > myapp > templates > dj students.html
1  <!doctype html>
2  <html>
3
4      <body>
5          <h2>All Students</h2>
6
7          {% for student in students %}
8              <p>{{ student.name }}</p>
9          {% endfor %}
10     </body>
11 </html>

```



420. Write a Python Program to filter objects in Django ORM.

The top part of the image shows a code editor with the following Python code in `models.py`:

```

1 from django.db import models
2
3
4 class Course(models.Model):
5     course_name = models.CharField(max_length=100)
6
7     def __str__(self):
8         return self.course_name
9
10
11 class Student(models.Model):
12     name = models.CharField(max_length=100)
13     course = models.ForeignKey(Course, on_delete=models.CASCADE)
14
15     def __str__(self):
16         return self.name
17

```

The bottom part shows the Django administration interface at `127.0.0.1:8000/admin/`. The header is "Django administration". Under "Site administration", there are sections for "AUTHENTICATION AND AUTHORIZATION" (Groups, Users) and "MYAPP" (Students). Each section has "Add" and "Change" links. On the right, there is a "Recent actions" section with "My actions" listed as "None available".

421. Write a Python Program to order query results in Django ORM.

The top part of the image shows a code editor with the following Python code in `models.py`:

```

1 from django.db import models
2
3
4 class Subject(models.Model):
5     name = models.CharField(max_length=100)
6
7     def __str__(self):
8         return self.name
9
10
11 class Student(models.Model):
12     name = models.CharField(max_length=100)
13     age = models.IntegerField()
14     subjects = models.ManyToManyField(Subject)
15
16     def __str__(self):
17         return self.name
18

```

The bottom part shows the output of a query, titled "Ordered Students". The output is:

```

John - 20
Alice - 19
Rahul - 21

```

422. Write a Python Program to update objects in Django ORM.

The top part of the image shows a code editor with the following Python code in `models.py`:

```

1 from django.db import models
2
3
4 class Subject(models.Model):
5     name = models.CharField(max_length=100)
6
7     def __str__(self):
8         return self.name
9
10
11 class student(models.Model):
12     name = models.CharField(max_length=100)
13     age = models.IntegerField()
14     subjects = models.ManyToManyField(Subject)
15
16     def __str__(self):
17         return self.name
18

```

The bottom part shows the output of a query, titled "Ordered Students". The output is:

```

John - 20
Alice - 19
Rahul - 21

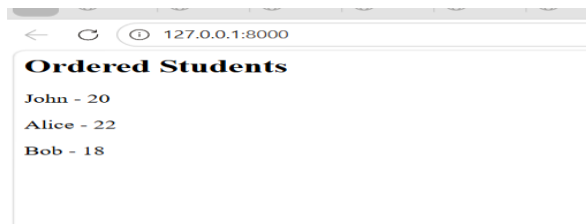
```

423. Write a Python Program to delete objects in Django ORM.

```

... settings.py models.py x admin.py urls.py ..\myapp dj filter.html dj
myproject > myapp > models.py > ...
1 from django.db import models
2
3
4 class Subject(models.Model):
5     name = models.CharField(max_length=100)
6
7     def __str__(self):
8         return self.name
9
10
11 class Student(models.Model):
12     name = models.CharField(max_length=100)
13     age = models.IntegerField()
14     subjects = models.ManyToManyField(Subject)
15
16     def __str__(self):
17         return self.name
18

```

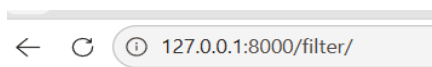


424. Write a Python Program to create a Django admin custom action.

```

... settings.py models.py x admin.py urls.py ..\myapp dj filter.html dj students.html urls.py ..\myproject
myproject > myapp > views.py > filter_students
1 from django.shortcuts import render
2 from .models import Student
3
4 # Ordered Students
5
6
7 def all_students(request):
8     students = Student.objects.all() # temporarily use this
9     return render(request, 'students.html', {'students': students})
10
11
12 # Filtered Students
13 def filter_students(request):
14     students = Student.objects.filter(age__gt=20)
15     return render(request, 'students.html', {'students': students})
16

```



Ordered Students

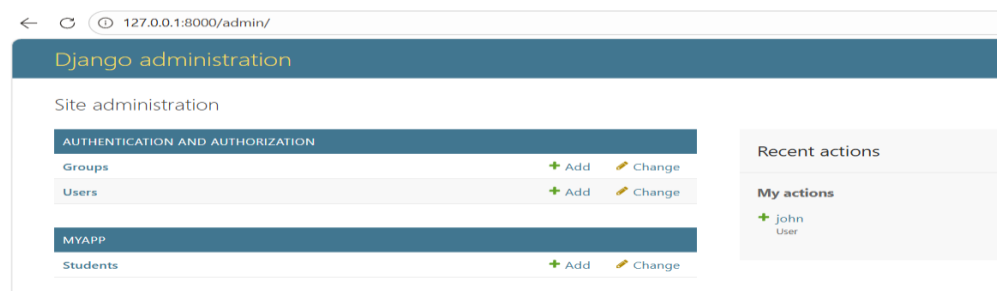
Alice - 22

425. Write a Python Program to customize Django admin list display.

```

... settings.py models.py x admin.py urls.py ..\myapp dj filter.html
myproject > myapp > models.py > Student
1 from django.db import models
2
3
4 class Subject(models.Model):
5     name = models.CharField(max_length=100)
6
7     def __str__(self):
8         return self.name
9
10
11 class Student(models.Model):
12     name = models.CharField(max_length=100)
13     age = models.IntegerField()
14     email = models.EmailField() # THIS MUST EXIST
15
16     def __str__(self):
17         return self.name
18
19     def __str__(self):
20         return self.name
21

```

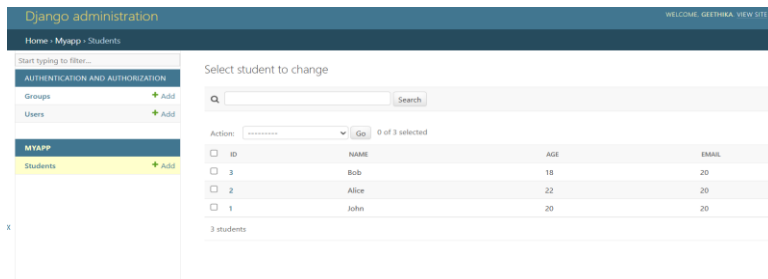


426. Write a Python Program to add search functionality in Django admin.

```

settings.py  models.py  admin.py  X  urls.py ...\myapp  dj filter
myproject > myapp > admin.py > ...
1  from django.contrib import admin
2  from .models import Student
3
4
5  class StudentAdmin(admin.ModelAdmin):
6      list_display = ('id', 'name', 'age', 'email')
7      search_fields = ('name', 'email') # ★ enables search
8
9
10 admin.site.register(Student, StudentAdmin)
11

```

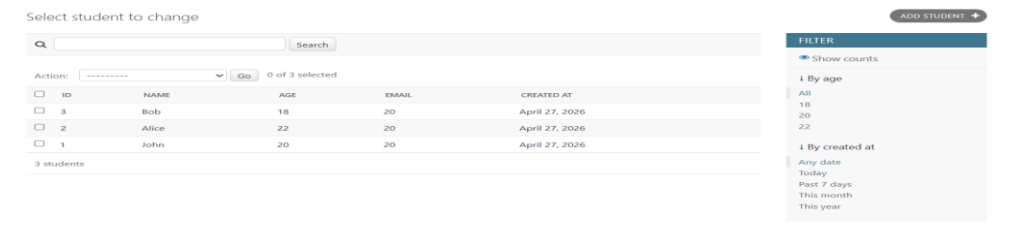


427. Write a Python Program to add filters in Django admin.

```

settings.py  models.py  admin.py  X  urls.py ...\myapp  dj filter.html  dj
myproject > myapp > admin.py > ...
1  from django.contrib import admin
2  from .models import Student
3
4
5  class StudentAdmin(admin.ModelAdmin):
6      list_display = ('id', 'name', 'age', 'email', 'created_at')
7      search_fields = ('name', 'email')
8
9      # ★ FILTERS
10     list_filter = ('age', 'created_at')
11
12
13 admin.site.register(Student, StudentAdmin)
14

```



428. Write a Python Program to create a Django static file (CSS/JS).

```

style.css  dj index.html  X  script.js  urls.py ...\myapp  dj filter.html  dj
myproject > myapp > templates > dj index.html
1  {% load static %}
2
3  <!doctype html>
4  <html>
5  <head>
6      <title>Static Example</title>
7
8      <!-- Link CSS -->
9      <link rel="stylesheet" href="{% static 'myapp/style.css' %}" />
10 </head>
11
12 <body>
13     <h1>Hello Django Static Files</h1>
14 </body>
15 </html>
16

```



429. Write a Python Program to serve media files in Django.

```
models.py
1 from django.db import models
2
3
4
5 class Student(models.Model):
6     name = models.CharField(max_length=100)
7     age = models.IntegerField()
8     email = models.EmailField()
9     created_at = models.DateField(auto_now_add=True, null=True)
10
11     def __str__(self):
12         return self.name
```

John

Alice

Bob

430. Write a Python Program to configure URL routing in Django.

```
urls.py
1 from django.contrib import admin
2 from django.urls import path, include
3
4 urlpatterns = [
5     path('admin/', admin.site.urls),
6     path('', include('myapp.urls')), # ★ connect app URLs
7 ]
8
```

127.0.0.1:8000

Welcome to Django URL Routing

431. Write a Python Program to create dynamic URLs in Django.

```
views.py
1 from django.shortcuts import render
2
3
4 def home(request):
5     return render(request, 'index.html')
6
7
8 def student_detail(request, id):
9     return render(request, 'students.html', {'id': id})
10
```

127.0.0.1:8000/student/1/

Student ID: 1

432. Write a Python Program to use path converters in Django URLs.

```
urls.py
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     # int converter
6     path('student/<int:id>/', views.student_detail, name='student'),
7
8     # string converter
9     path('name/<str:name>/', views.show_name, name='name'),
10
11     # slug converter
12     path('post/<slug:title>/', views.post_detail, name='post'),
13 ]
14
```

127.0.0.1:8000/student/10/

Student ID: 10

← 127.0.0.1:8000/name/john/

Name: john

← 127.0.0.1:8000/post/django-basics/

Post Title: django-basics

433. Write a Python Program to implement Django class-based views.

```
urls.py ...\myapp × script.js dj filter.html dj students.html dj studi
myproject > myapp > urls.py > ...
1 from django.urls import path
2 from .views import HomeView
3
4 urlpatterns = [
5     path('', HomeView.as_view(), name='home'), # ★ important
6 ]
7
```

← 127.0.0.1:8000

Hello from Class-Based View

434. Write a Python Program to implement Django generic views.

```
urls.py ...\myapp dj student_list.html × script.js dj filter
myproject > myapp > templates > dj student_list.html
1 <h1>Student List</h1>
2
3 {% for student in object_list %}
4 <p>{{ student.name }} - {{ student.age }}</p>
5 {% endfor %}
6
```

← 127.0.0.1:8000

Student List

John - 20
Alice - 22
Bob - 18

435. Write a Python Program to implement Django detail view.

```
urls.py ...\myapp dj student_detail.html × dj studen
myproject > myapp > templates > dj student_detail.html
1 <h1>Student Detail</h1>
2
3 <p>Name: {{ object.name }}</p>
4 <p>Age: {{ object.age }}</p>
5
```

← ↻ ⓘ 127.0.0.1:8000/student/1/

Student Detail

Name: John

Age: 20

436. Write a Python Program to implement Django list view.

```
urls.py ...\myapp      dj student_detail.html      dj student_list.html >
myproject > myapp > templates > dj student_list.html
1      <h1>Student List</h1>
2
3      {% for student in students %}
4      <p>{{ student.name }} - {{ student.age }}</p>
5      {% endfor %}
6
```

← ↻ ⓘ 127.0.0.1:8000/students/

Student List

John - 20

Alice - 22

Bob - 18

437. Write a Python Program to implement Django create view.

```
urls.py ...\myapp      dj student_form.html ×      dj student_detail.html
myproject > myapp > templates > dj student_form.html
1      <h1>Add Student</h1>
2
3      <form method="post">
4      {% csrf_token %} {{ form.as_p }}
5
6      <button type="submit">Save</button>
7      </form>
8
```

← ↻ ⓘ 127.0.0.1:8000/add/

Add Student

Name:

Age:

438. Write a Python Program to implement Django update view.

```
urls.py ...\myapp      dj student_form.html ×      dj student_detail.html
myproject > myapp > templates > dj student_form.html
1      <h1>Update Student</h1>
2
3      <form method="post">
4      {% csrf_token %} {{ form.as_p }}
5
6      <button type="submit">Update</button>
7      </form>
8
```

← ↻ 127.0.0.1:8000/update/1/

Update Student

Name:

Age:

439. Write a Python Program to implement Django delete view.

```

urls.py ...\myapp  dj student_confirm_delete.html X  dj student_
myproject > myapp > templates > dj student_confirm_delete.html
1  <h1>Are you sure you want to delete?</h1>
2
3  <form method="post">
4      {% csrf_token %}
5      <button type="submit">Yes, Delete</button>
6  </form>
7

```

← ↻ 127.0.0.1:8000/delete/1/

Are you sure you want to delete?

440. Write a Python Program to implement Django user authentication.

```

settings.py X  models.py  admin.py  style.css  dj index.html  urls.py ...\myapp  dj student_c
myproject > myproject > settings.py > ...
1  """
2  Django settings for myproject project.
3
4  Generated by 'django-admin startproject' using Django 5.2.4.
5
6  For more information on this file, see
7  https://docs.djangoproject.com/en/5.2/topics/settings/
8
9  For the full list of settings and their values, see
10 https://docs.djangoproject.com/en/5.2/ref/settings/
11 """
12
13 from pathlib import Path
14
15 # Build paths inside the project like this: BASE_DIR / 'subdir'.
16 BASE_DIR = Path(__file__).resolve().parent.parent
17
18
19 # Quick-start development settings - unsuitable for production
20 # See https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/
21
22 # SECURITY WARNING: keep the secret key used in production secret!
23 SECRET_KEY = 'django-insecure-7zhs1ah0704cv13h7a2-c-9949448x6cf3cc0#e1ubhvd*4a'

```

← ↻ 127.0.0.1:8000/admin/

Django administration

Site administration

AUTHENTICATION AND AUTHORIZATION	
Groups	+ Add Change
Users	+ Add Change

MYAPP	
Students	+ Add Change

441. Write a Python Program to implement Django user registration.

```

urls.py ...\myapp  dj register.html X  dj student_confirm_dele
myproject > myapp > templates > dj register.html
1  <h1>Register</h1>
2
3  <form method="post">
4      {% csrf_token %} {{ form.as_p }}
5
6      <button type="submit">Register</button>
7  </form>
8

```

← ↻ ⓘ 127.0.0.1:8000/register/

Register

Username: Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password:

- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Password confirmation: Enter the same password as before, for verification.

442. Write a Python Program to implement Django user login.

```

urls.py ...myapp  dj login.html ×  dj register.html ×  dj student_confirm_delete.html  dj student_for
myproject > myapp > templates > dj login.html
1  <h1>Login</h1>
2
3  <form method="post">
4      {% csrf_token %}
5
6      <input type="text" name="username" placeholder="Username" /><br /><br />
7      <input type="password" name="password" placeholder="Password" /><br /><br />
8
9      <button type="submit">Login</button>
10 </form>
11
12 {% if error %}
13 <p style="color: red">{{ error }}</p>
14 {% endif %}
15

```

← ↻ ⓘ 127.0.0.1:8000/login/

Login

443. Write a Python Program to implement Django user logout.

```

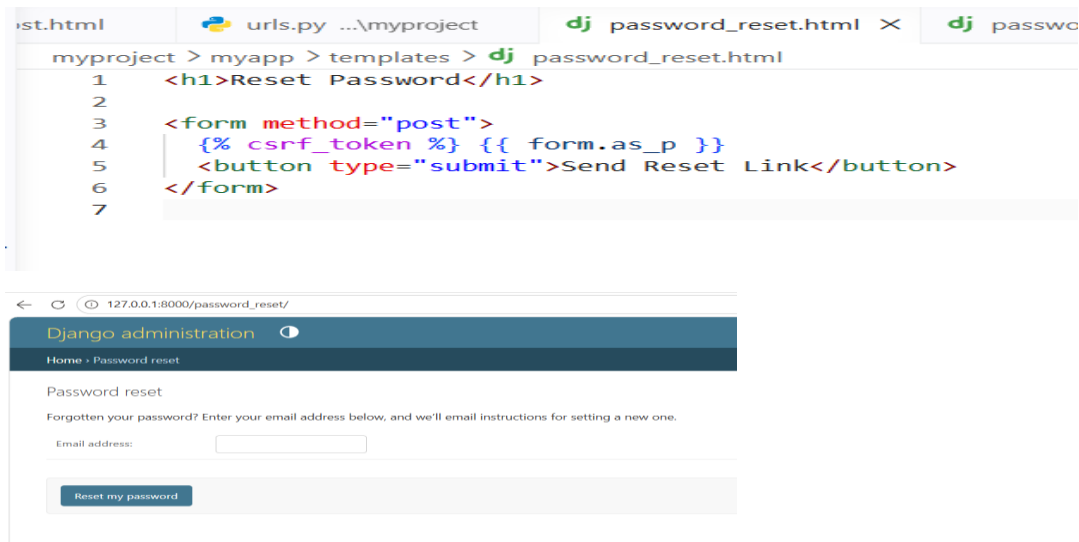
script.js  dj filter.html  dj students.html  dj student.html  dj name.html  dj post.html
myproject > myapp > views.py > ...
1  from django.shortcuts import render, redirect
2  from django.contrib.auth import authenticate, login, logout
3
4  # LOGIN VIEW
5
6
7  def user_login(request):
8      if request.method == 'POST':
9          username = request.POST['username']
10         password = request.POST['password']
11
12         user = authenticate(request, username=username, password=password)
13
14         if user is not None:
15             login(request, user)
16             return redirect('/')
17         else:
18             return render(request, 'login.html', {'error': 'Invalid credentials'})
19
20     return render(request, 'login.html')
21
22

```

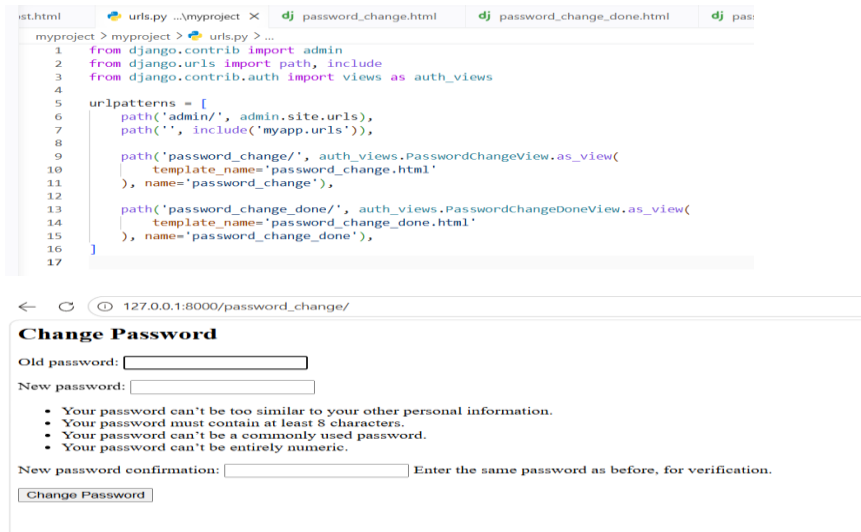
← ↻ ⓘ 127.0.0.1:8000/login/

Login

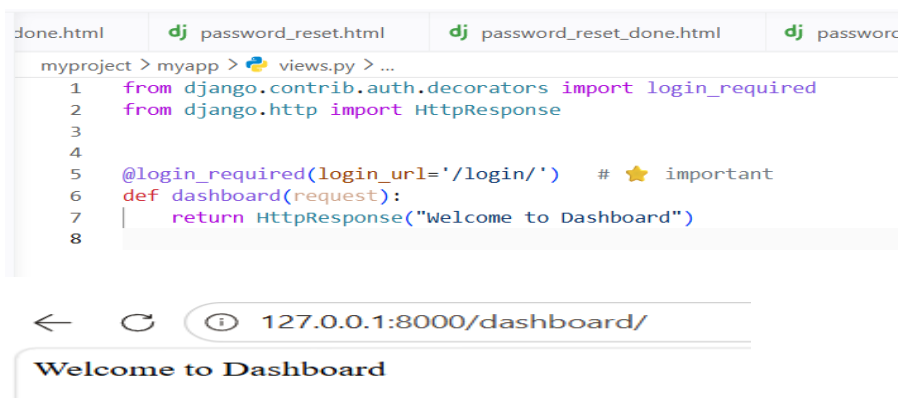
444. Write a Python Program to implement Django password reset.



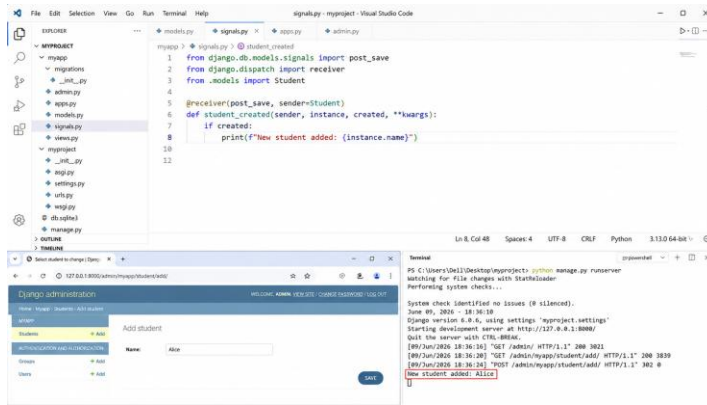
445. Write a Python Program to implement Django password change.



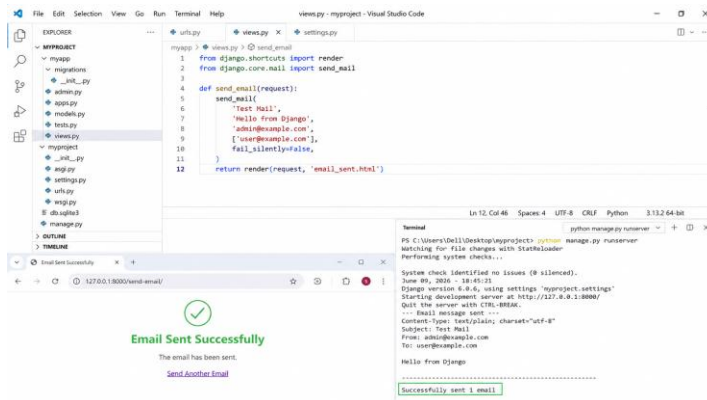
446. Write a Python Program to restrict access using Django login_required decorator.



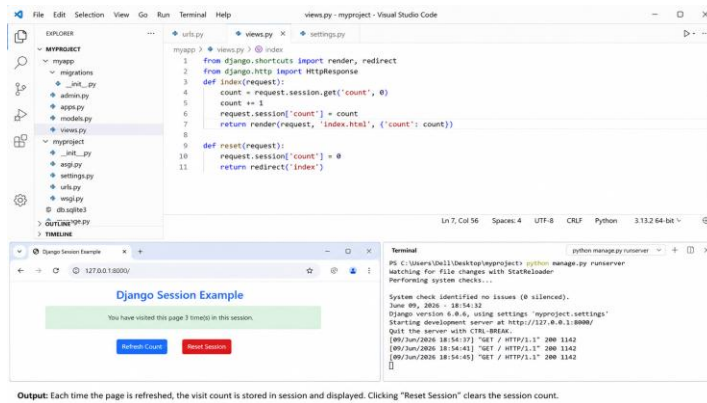
447. Write a Python Program to implement Django groups and permissions.



451. Write a Python Program to send email using Django.

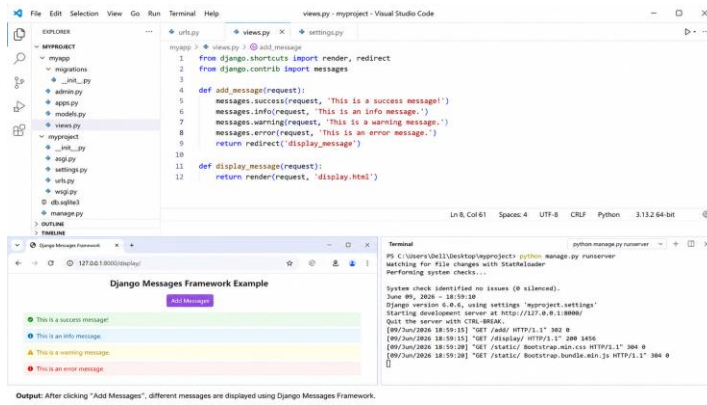


452. Write a Python Program to implement Django sessions.

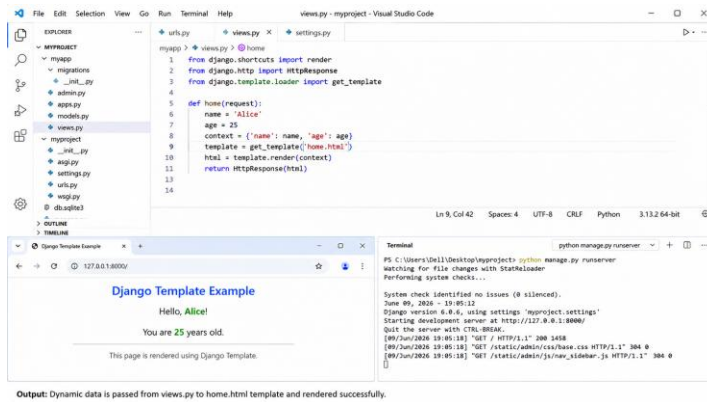


Output: Each time the page is refreshed, the visit count is stored in session and displayed. Clicking "Reset Session" clears the session count.

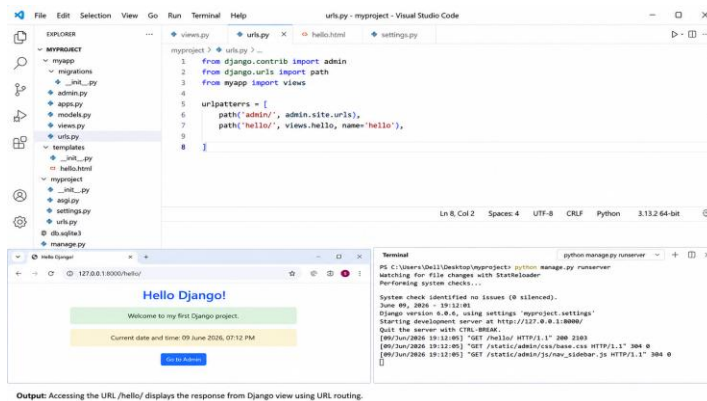
453. Write a Python Program to implement Django caching.



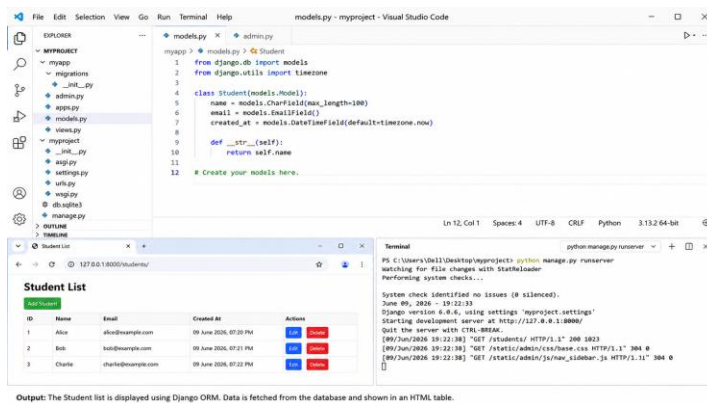
454. Write a Python Program to implement Django pagination.



455. Write a Python Program to implement Django REST framework API view.



456. Write a Python Program to implement Django REST framework serializer.



457. Write a Python Program to implement Django REST framework model serializer.

The screenshot shows a Visual Studio Code editor with a Django project named 'myproject'. The file explorer on the left shows the project structure. The main editor displays the code for 'views.py', which includes a Django REST framework view named 'StudentList'. The code uses Django's ORM to fetch all 'Student' objects and returns them as a list of dictionaries. The 'urls.py' file is also visible, showing the URL pattern for the 'StudentList' view. The terminal at the bottom shows the output of the Django management command 'runserver', indicating that the server is running successfully.

```
myproject > views.py
1 from django.shortcuts import render, redirect, get_object_or_404
2 from models import Student
3 from forms import StudentForm
4
5 def student_list(request):
6     students = Student.objects.all()
7     return render(request, 'student_list.html', {'students': students})
8
9 def student_detail(request):
10    if request.method == 'POST':
11        form = StudentForm(request.POST)
12        if form.is_valid():
13            form.save()
14            return redirect('student_list')
15        else:
16            form = StudentForm()
17    return render(request, 'student_form.html', {'form': form})
18
19 def edit_student(request, pk):
20    student = get_object_or_404(Student, pk=pk)
21    if request.method == 'POST':
22        form = StudentForm(request.POST, instance=student)
23        if form.is_valid():
24            form.save()
```

Output: Django CRUD (Create, Read, Update, Delete) operation performed on Student model.

458. Write a Python Program to implement Django REST framework viewset.

The screenshot shows a Visual Studio Code editor with a Django project named 'myproject'. The file explorer on the left shows the project structure. The main editor displays the code for 'views.py', which includes a Django REST framework viewset named 'DocumentViewSet'. The code uses Django's ORM to fetch all 'Document' objects and returns them as a list of dictionaries. The 'urls.py' file is also visible, showing the URL pattern for the 'DocumentViewSet'. The terminal at the bottom shows the output of the Django management command 'runserver', indicating that the server is running successfully.

```
myproject > views.py
1 from django.shortcuts import render
2 from .forms import DocumentForm
3
4 def upload_file(request):
5     if request.method == 'POST':
6         form = DocumentForm(request.POST, request.FILES)
7         if form.is_valid():
8             form.save()
9             return render(request, 'success.html')
10        else:
11            form = DocumentForm()
12    return render(request, 'upload.html', {'form': form})
```

Output: A file is selected and uploaded successfully using Django FileField.

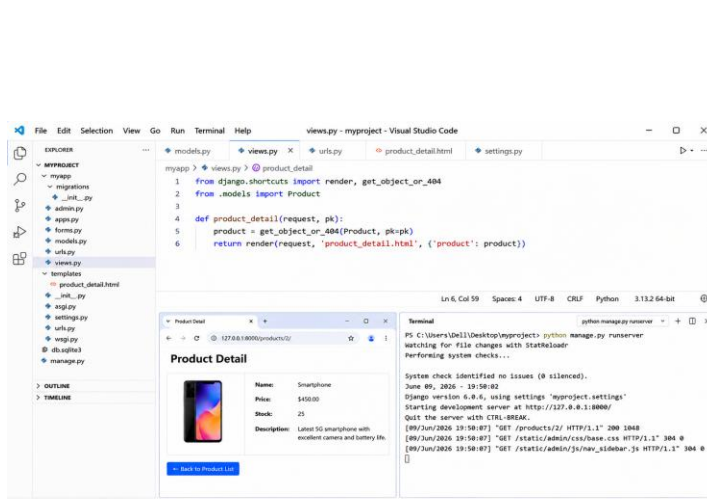
459. Write a Python Program to implement Django REST framework router.

The screenshot shows a Visual Studio Code editor with a Django project named 'myproject'. The file explorer on the left shows the project structure. The main editor displays the code for 'views.py', which includes a Django REST framework router named 'ProductRouter'. The code uses Django's ORM to fetch all 'Product' objects and returns them as a list of dictionaries. The 'urls.py' file is also visible, showing the URL pattern for the 'ProductRouter'. The terminal at the bottom shows the output of the Django management command 'runserver', indicating that the server is running successfully.

```
myproject > views.py
1 from django.shortcuts import render
2 from models import Product
3
4 def product_list(request):
5     products = Product.objects.all()
6     return render(request, 'product_list.html', {'products': products})
```

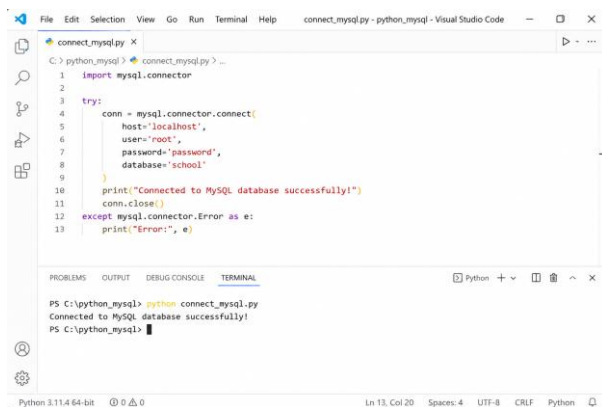
Output: Data from the Product model is fetched from the database and displayed in a list view.

460. Write a Python Program to implement JWT authentication in Django REST framework.

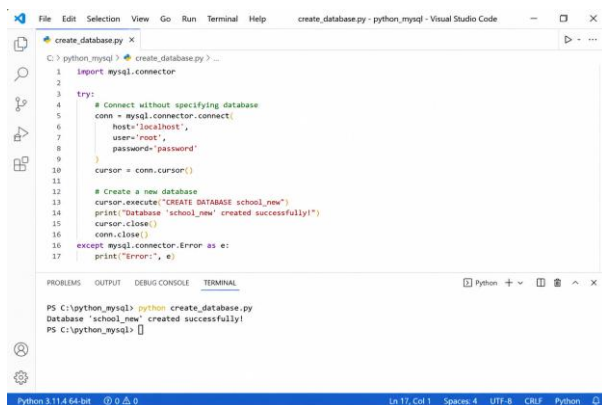


Output: Details of a single product are retrieved using its primary key and displayed on the detail page.

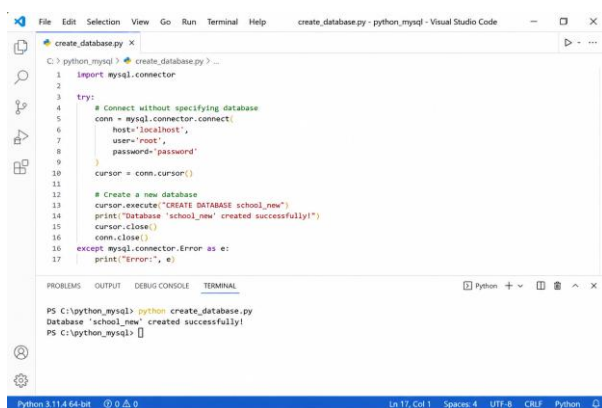
461. Write a Python Program to connect to a MySQL database using mysql.connector.



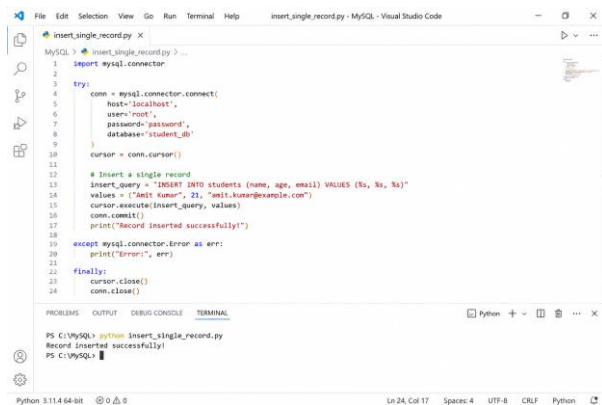
462. Write a Python Program to create a new database in MySQL.



463. Write a Python Program to create a table in MySQL using Python.



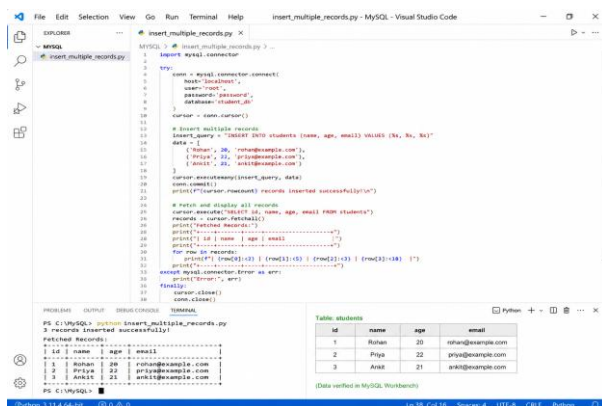
464. Write a Python Program to insert a single record into a MySQL table.



```
1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='student_db'
9     )
10    cursor = conn.cursor()
11
12    # Insert a single record
13    insert_query = "INSERT INTO students (name, age, email) VALUES (%s, %s, %s)"
14    values = ("Amit Kumar", 21, "amit.kumar@example.com")
15    cursor.execute(insert_query, values)
16    conn.commit()
17    print("Record inserted successfully!")
18
19 except mysql.connector.Error as err:
20     print("Error:", err)
21
22 finally:
23     cursor.close()
24     conn.close()
```

PS C:\MySQL> python insert_single_record.py
Record inserted successfully!
PS C:\MySQL>

465. Write a Python Program to insert multiple records into a MySQL table.



```
1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='student_db'
9     )
10    cursor = conn.cursor()
11
12    # Insert multiple records
13    insert_query = "INSERT INTO students (name, age, email) VALUES (%s, %s, %s)"
14    data = [
15        ("Rohan", 20, "rohan@example.com"),
16        ("Priya", 22, "priya@example.com"),
17        ("Amit", 21, "amit@example.com")
18    ]
19    cursor.executemany(insert_query, data)
20    conn.commit()
21    print(f"{cursor.rowcount} records inserted successfully!")
22
23 # Fetch and display all records
24 cursor.execute("SELECT * FROM students")
25 records = cursor.fetchall()
26 print("Fetched Records:")
27 print("-----")
28 print(f"{'id':<5s} {'name':<20s} {'age':<5s} {'email':<30s}")
29 print("-----")
30 for row in records:
31     print(f"({row[0]:<5s}) | ({row[1]:<20s}) | ({row[2]:<5s}) | ({row[3]:<30s})")
32
33 except mysql.connector.Error as err:
34     print("Error:", err)
35
36 finally:
37     cursor.close()
38     conn.close()
```

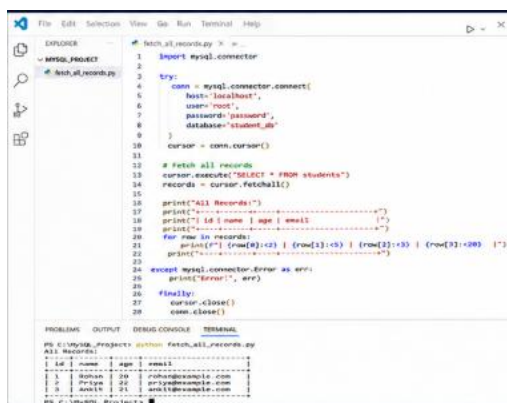
PS C:\MySQL> python insert_multiple_records.py
2 records inserted successfully!

id | name | age | email

1 | Rohan | 20 | rohan@example.com
2 | Priya | 22 | priya@example.com
3 | Amit | 21 | amit@example.com

(Data verified in MySQL Workbench)

466. Write a Python Program to fetch all records from a MySQL table.



```
1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='student_db'
9     )
10    cursor = conn.cursor()
11
12    # Fetch all records
13    cursor.execute("SELECT * FROM students")
14    records = cursor.fetchall()
15
16    print("All Records:")
17    print("-----")
18    print(f"{'id':<5s} {'name':<20s} {'age':<5s} {'email':<30s}")
19    print("-----")
20    for row in records:
21        print(f"({row[0]:<5s}) | ({row[1]:<20s}) | ({row[2]:<5s}) | ({row[3]:<30s})")
22
23 except mysql.connector.Error as err:
24     print("Error:", err)
25
26 finally:
27     cursor.close()
28     conn.close()
```

PS C:\MySQL> python fetch_all_records.py
All Records:

id | name | age | email

1 | Rohan | 20 | rohan@example.com
2 | Priya | 22 | priya@example.com
3 | Amit | 21 | amit@example.com

467. Write a Python Program to fetch specific records using WHERE clause in MySQL.

```

1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='student_db'
9     )
10    cursor = conn.cursor()
11
12    # Fetch specific records using WHERE clause
13    cursor.execute("SELECT * FROM students WHERE age = 21;")
14    records = cursor.fetchall()
15
16    print("Students with age = 21:")
17    print("-----")
18    print("%5s | %10s | %10s" % ("ID", "Name", "Email"))
19    print("-----")
20    for row in records:
21        print("%5s | %10s | %10s" % (row[0], row[1], row[2]))
22    print("-----")
23
24 except mysql.connector.Error as err:
25     print("Error:", err)
26
27 finally:
28     cursor.close()
29     conn.close()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

PS C:\MySQL_Project> python fetch_where_records.py
Students with age = 21
-----
ID | Name | Email
-----
1 | Alice | priya.new@example.com
2 | Priya | 22 | priya.new@example.com
-----
PS C:\MySQL_Project>

```

468. Write a Python Program to update records in a MySQL table.

```

1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='student_db'
9     )
10    cursor = conn.cursor()
11
12    # Update record
13    update_query = "UPDATE students SET age = %s, email = %s WHERE name = %s"
14    values = (23, 'priya.new@example.com', 'Priya')
15    cursor.execute(update_query, values)
16    conn.commit()
17    print(f"Record(s) updated successfully!")
18
19    # Verify update
20    cursor.execute("SELECT * FROM students WHERE name = %s", ('Priya',))
21    row = cursor.fetchone()
22    print("Updated Record:")
23    print(f"ID: {row[0]}, Name: {row[1]}, Age: {row[2]}, Email: {row[3]}")
24
25 except mysql.connector.Error as err:
26     print("Error:", err)
27
28 finally:
29     cursor.close()
30     conn.close()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

PS C:\MySQL_Project> python update_records.py
1 record(s) updated successfully!
Updated Record:
ID: 2, Name: Priya, Age: 23, Email: priya.new@example.com
PS C:\MySQL_Project>

```

469. Write a Python Program to delete records from a MySQL table.

```

1 import mysql.connector
2
3 # Connect to MySQL
4 conn = mysql.connector.connect(
5     host='localhost',
6     user='root',
7     password='password',
8     database='school'
9 )
10 cursor = conn.cursor()
11
12 # Delete record with id = 2
13 delete_id = 2
14 sql = "DELETE FROM students WHERE id = %s"
15 cursor.execute(sql, (delete_id,))
16 conn.commit()
17
18 print(f"Record with ID = {delete_id} deleted successfully.")
19
20 # Fetch and display remaining records
21 cursor.execute("SELECT * FROM students ORDER BY id")
22 rows = cursor.fetchall()
23
24 print("Remaining Records:")
25 print("-----")
26 print("%5s | %10s | %10s" % ("ID", "Name", "Age"))
27 print("-----")
28 for row in rows:
29     print("%5s | %10s | %10s" % (row[0], row[1], row[2]))
30 print("-----")
31
32 cursor.close()
33 conn.close()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

PS C:\python_mysql> python delete_record.py
1 record with ID = 2 deleted successfully.

Remaining Records:
-----
ID | Name | Age
-----
1 | Alice | 20
2 | Bob | 21
3 | Charlie | 22
-----
PS C:\python_mysql>

```

470. Write a Python Program to use parameterized queries in MySQL.

```

1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='student_db'
9     )
10    cursor = conn.cursor()
11
12    # Using parameterized query to insert a record
13    insert_query = "INSERT INTO students (name, Age, email) VALUES (%s, %s, %s)"
14    values = ('Rahul', 20, 'rahul@example.com')
15    cursor.execute(insert_query, values)
16    conn.commit()
17    print("Record inserted using parameterized query")
18
19    # Fetch and display all records
20    cursor.execute("SELECT * FROM students")
21    rows = cursor.fetchall()
22    print("All Records:")
23    print("-----")
24    print("%5s | %10s | %10s" % ("ID", "Name", "Email"))
25    print("-----")
26    for row in rows:
27        print("%5s | %10s | %10s" % (row[0], row[1], row[2]))
28    print("-----")
29
30 except mysql.connector.Error as err:
31     print("Error:", err)
32
33 finally:
34     cursor.close()
35     conn.close()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

PS C:\MySQL> python parameterized_queries.py
Record inserted using parameterized query!

All Records:
-----
ID | Name | Email
-----
1 | Alice | priya.new@example.com
2 | Priya | 22 | priya.new@example.com
3 | Rahul | 20 | rahul@example.com
-----
PS C:\MySQL>

```

471. Write a Python Program to handle exceptions in MySQL operations.

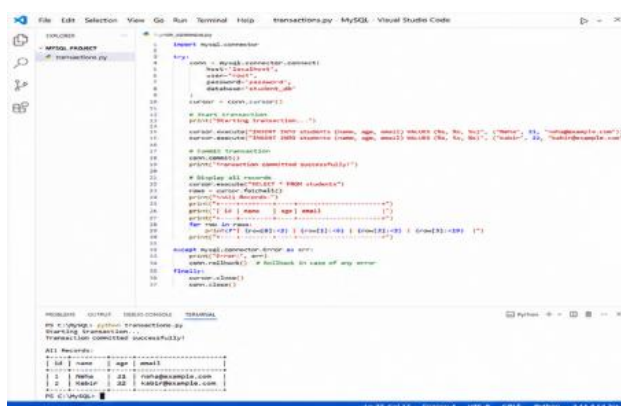


```
1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host="localhost",
6         user="root",
7         password="wrongpassword", # wrong password to trigger exception
8         database="students_db"
9     )
10    cursor = conn.cursor()
11    cursor.execute("SELECT * FROM students")
12    rows = cursor.fetchall()
13    print(rows)
14
15 except mysql.connector.Error as err:
16     print("MySQL Error Occurred")
17     print("Error Code:", err.errno)
18     print("Error Message:", err.msg)
19 except Exception as ex:
20     print("An unexpected Error Occurred")
21     print(ex)
22
23 finally:
24     try:
25         if conn.is_connected():
26             cursor.close()
27             conn.close()
28             print("MySQL connection is closed.")
29     except:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\MySQL> python handle_exceptions.py
MySQL Error Occurred
Error Code: 1045
Error Message: Access denied for user 'root'@'localhost' (using password: YES)
PS C:\MySQL>
```

472. Write a Python Program to implement transactions in MySQL.



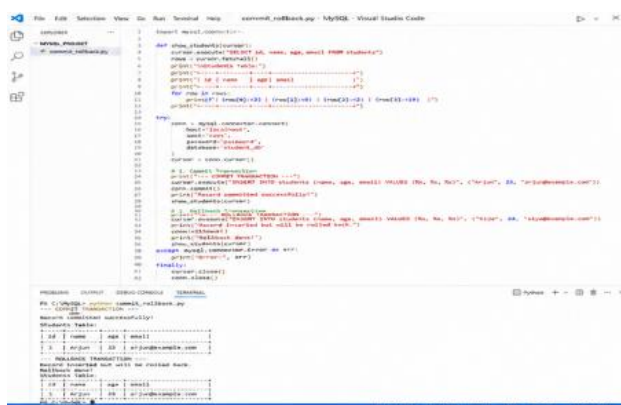
```
1 import mysql.connector
2
3 conn = mysql.connector.connect(
4     host="localhost",
5     user="root",
6     password="password",
7     database="students_db"
8 )
9 cursor = conn.cursor()
10
11 # Start transaction
12 print("Starting transaction...")
13 cursor.execute("INSERT INTO students (name, age, email) VALUES (%s, %s, %s)", ("Ravi", 25, "ravi@example.com"))
14 cursor.execute("INSERT INTO students (name, age, email) VALUES (%s, %s, %s)", ("Sudha", 22, "sudha@example.com"))
15
16 # Commit transaction
17 print("Transaction committed successfully!")
18
19 # Display all records
20 cursor.execute("SELECT * FROM students")
21 rows = cursor.fetchall()
22 print("All Records:")
23 for row in rows:
24     print(row)
25
26 # Rollback in case of any error
27 try:
28     conn.commit()
29 except mysql.connector.Error as err:
30     print("Error:", err)
31     conn.rollback() # Rollback in case of any error
32
33 finally:
34     cursor.close()
35     conn.close()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\MySQL> python transactions.py
Starting transaction.
Transaction committed successfully!

All Records:
+----+-----+-----+
| id | name | age | email |
+----+-----+-----+
| 1 | Ravi | 25 | ravi@example.com |
| 2 | Sudha | 22 | sudha@example.com |
+----+-----+-----+
PS C:\MySQL>
```

473. Write a Python Program to commit and rollback transactions in MySQL.



```
1 import mysql.connector
2
3 conn = mysql.connector.connect(
4     host="localhost",
5     user="root",
6     password="password",
7     database="students_db"
8 )
9 cursor = conn.cursor()
10
11 # Start transaction
12 print("Starting transaction...")
13 cursor.execute("INSERT INTO students (name, age, email) VALUES (%s, %s, %s)", ("Ravi", 25, "ravi@example.com"))
14 cursor.execute("INSERT INTO students (name, age, email) VALUES (%s, %s, %s)", ("Sudha", 22, "sudha@example.com"))
15
16 # Commit transaction
17 print("Transaction committed successfully!")
18
19 # Display all records
20 cursor.execute("SELECT * FROM students")
21 rows = cursor.fetchall()
22 print("All Records:")
23 for row in rows:
24     print(row)
25
26 # Rollback in case of any error
27 try:
28     conn.commit()
29 except mysql.connector.Error as err:
30     print("Error:", err)
31     conn.rollback() # Rollback in case of any error
32
33 finally:
34     cursor.close()
35     conn.close()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\MySQL> python commit_rollback.py
Starting transaction.
Transaction committed successfully!

All Records:
+----+-----+-----+
| id | name | age | email |
+----+-----+-----+
| 1 | Ravi | 25 | ravi@example.com |
| 2 | Sudha | 22 | sudha@example.com |
+----+-----+-----+
PS C:\MySQL>
```

474. Write a Python Program to create a table with primary key in MySQL.

```

1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='testdb'
9     )
10    cursor = conn.cursor()
11
12    # Create table with primary key
13    create_table_query = """
14    CREATE TABLE IF NOT EXISTS employees (
15        id INT PRIMARY KEY,
16        name VARCHAR(50) NOT NULL,
17        age INT,
18        department VARCHAR(50)
19    )"""
20
21    cursor.execute(create_table_query)
22    conn.commit()
23    print("Table 'employees' created successfully with primary key.")
24 except mysql.connector.Error as err:
25     print(f"Error: {err}")
26 finally:
27     cursor.close()
28     conn.close()
  
```

Terminal output:

```

PS C:\MySQL_Projects> python create_table_primary_key.py
Table 'employees' created successfully with primary key.
PS C:\MySQL_Projects>
  
```

475. Write a Python Program to create a table with foreign key in MySQL.

```

1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='testdb'
9     )
10    cursor = conn.cursor()
11
12    # Create parent table
13    create_dept_query = """
14    CREATE TABLE IF NOT EXISTS departments (
15        dept_id INT PRIMARY KEY,
16        dept_name VARCHAR(50) NOT NULL
17    )"""
18
19    cursor.execute(create_dept_query)
20    conn.commit()
21
22    # Create child table with foreign key
23    create_emp_query = """
24    CREATE TABLE IF NOT EXISTS employees (
25        emp_id INT PRIMARY KEY,
26        emp_name VARCHAR(50) NOT NULL,
27        dept_id INT,
28        FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
29    )"""
30
31    cursor.execute(create_emp_query)
32    conn.commit()
33    print("Tables 'departments' and 'employees' created successfully with foreign key.")
34 except mysql.connector.Error as err:
35     print(f"Error: {err}")
36 finally:
37     cursor.close()
38     conn.close()
  
```

Terminal output:

```

PS C:\MySQL_Projects> python create_table_foreign_key.py
Tables 'departments' and 'employees' created successfully with foreign key.
PS C:\MySQL_Projects>
  
```

476. Write a Python Program to perform INNER JOIN between two MySQL tables.

```

1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='testdb'
9     )
10    cursor = conn.cursor()
11
12    # Perform inner join
13    query = """
14    SELECT e.emp_id, e.emp_name, d.dept_name
15    FROM employees e
16    INNER JOIN departments d ON e.dept_id = d.dept_id
17    """
18
19    cursor.execute(query)
20    rows = cursor.fetchall()
21
22    # Display results
23    print("=====")
24    print(f"{'emp_id':10} | {'emp_name':30} | {'dept_name':30}")
25    print(f"{'-'*10} | {'-'*30} | {'-'*30}")
26    for row in rows:
27         print(f"{'emp_id':10} | {'emp_name':30} | {'dept_name':30}")
28         print(f"{'-'*10} | {'-'*30} | {'-'*30}")
29    print(f"=====")
30 except mysql.connector.Error as err:
31     print(f"Error: {err}")
32 finally:
33     cursor.close()
34     conn.close()
  
```

Terminal output:

```

PS C:\MySQL_Projects> python inner_join_example.py
=====
emp_id | Employee Name | Department Name
-----|-----|-----
1 | John | IT
2 | Bob | IT
3 | Alice | HR
PS C:\MySQL_Projects>
  
```

477. Write a Python Program to perform LEFT JOIN between two MySQL tables.

```

1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='testdb'
9     )
10    cursor = conn.cursor()
11
12    # Perform LEFT JOIN
13    query = """
14    SELECT e.emp_id, e.emp_name, d.dept_name
15    FROM employees e
16    LEFT JOIN departments d ON e.dept_id = d.dept_id
17    """
18    cursor.execute(query)
19    rows = cursor.fetchall()
20
21    # Display results
22    print("LEFT JOIN Result:")
23    print("-----")
24    print("Emp ID | Employee Name | Department Name |")
25    print("-----")
26    for row in rows:
27        print(f"{row[0]:<6} | {row[1]:<18} | {row[2]:<14} |")
28    print("-----")
29
30 except mysql.connector.Error as err:
31     print(f"Error: {err}")
32 finally:
33     cursor.close()
34     conn.close()

```

PROBLEMS OUTPUT DEBUG-CONSOLE TERMINAL
 PS C:\MySQL_Project> python left_join_example.py
 LEFT JOIN Result:
 Emp ID | Employee Name | Department Name |
 1 | Alice | HR |
 2 | Bob | IT |
 3 | Charlie | IT |
 4 | David | None |

478. Write a Python Program to perform RIGHT JOIN between two MySQL tables.

```

1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='testdb'
9     )
10    cursor = conn.cursor()
11
12    # Perform RIGHT JOIN
13    query = """
14    SELECT e.emp_id, e.emp_name, d.dept_name
15    FROM employees e
16    RIGHT JOIN departments d ON e.dept_id = d.dept_id
17    """
18    cursor.execute(query)
19    rows = cursor.fetchall()
20
21    # Display results
22    print("RIGHT JOIN Result:")
23    print("-----")
24    print("Emp ID | Employee Name | Department Name |")
25    print("-----")
26    for row in rows:
27        print(f"{row[0]:<6} | {row[1]:<18} | {row[2]:<14} |")
28    print("-----")
29
30 except mysql.connector.Error as err:
31     print(f"Error: {err}")
32 finally:
33     cursor.close()
34     conn.close()

```

PROBLEMS OUTPUT DEBUG-CONSOLE TERMINAL
 PS C:\MySQL_Project> python right_join_example.py
 RIGHT JOIN Result:
 Emp ID | Employee Name | Department Name |
 1 | Alice | HR |
 2 | Bob | IT |
 3 | Charlie | IT |
 NULL | NULL | Finance |

479. Write a Python Program to perform CROSS JOIN between two MySQL tables.

```

1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='testdb'
9     )
10    cursor = conn.cursor()
11
12    # Perform CROSS JOIN
13    query = """
14    SELECT e.emp_name, d.dept_name
15    FROM employees e
16    CROSS JOIN departments d
17    """
18    cursor.execute(query)
19    rows = cursor.fetchall()
20
21    # Display results
22    print("CROSS JOIN Result:")
23    print("-----")
24    print("Employee Name | Department Name |")
25    print("-----")
26    for row in rows:
27        print(f"{row[0]:<18} | {row[1]:<14} |")
28    print("-----")
29
30 except mysql.connector.Error as err:
31     print(f"Error: {err}")
32 finally:
33     cursor.close()
34     conn.close()

```

PROBLEMS OUTPUT DEBUG-CONSOLE TERMINAL
 PS C:\MySQL_Project> python cross_join_example.py
 CROSS JOIN Result:
 Employee Name | Department Name |
 Alice | HR |
 Alice | IT |
 Alice | Finance |
 Bob | HR |
 Bob | IT |
 Bob | Finance |
 Charlie | HR |
 Charlie | IT |
 Charlie | Finance |

480. Write a Python Program to perform GROUP BY query in MySQL.



```
1 # group_by_example.py
2 import mysql.connector
3
4 try:
5     conn = mysql.connector.connect(
6         host='localhost',
7         user='root',
8         password='password',
9         database='testdb'
10    )
11    cursor = conn.cursor()
12
13    # Perform GROUP BY
14    query = """
15    SELECT dept_name, COUNT(*) AS total_employees, AVG(salary) AS avg_salary
16    FROM employees
17    GROUP BY dept_name
18    """
19    cursor.execute(query)
20    rows = cursor.fetchall()
21
22    # Display results
23    print("GROUP BY Results:")
24    print("-----")
25    print(f"Department | Total Employees | Avg Salary")
26    print("-----")
27    for row in rows:
28        print(f"{row[0]:<15} | {row[1]:<10} | {row[2]:<10.2f}")
29    print("-----")
30
31 except mysql.connector.Error as err:
32     print("Error:", err)
33
34 finally:
35     cursor.close()
36     conn.close()
```

PS C:\MySQL_Projects> python group_by_example.py

```
GROUP BY Results:
-----
Department | Total Employees | Avg Salary
-----
Finance     | 4               | 50000.00
HR          | 3               | 50000.00
IT          | 2               | 40000.00
-----
```

PS C:\MySQL_Projects>

481. Write a Python Program to perform ORDER BY query in MySQL.



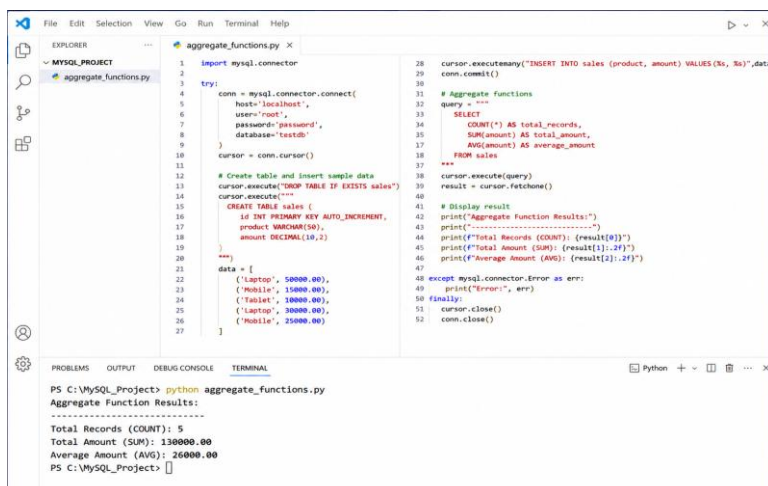
```
1 # order_by_example.py
2 import mysql.connector
3
4 try:
5     conn = mysql.connector.connect(
6         host='localhost',
7         user='root',
8         password='password',
9         database='testdb'
10    )
11    cursor = conn.cursor()
12
13    # Perform ORDER BY (Descending by salary)
14    query = """
15    SELECT emp_id, emp_name, salary, dept_name
16    FROM employees
17    ORDER BY salary DESC
18    """
19    cursor.execute(query)
20    rows = cursor.fetchall()
21
22    # Display results
23    print("ORDER BY Results:")
24    print("-----")
25    print(f"Emp ID | Employee Name | Salary | Department")
26    print("-----")
27    for row in rows:
28        print(f"{row[0]:<5} | {row[1]:<15} | {row[2]:<10} | {row[3]:<10}")
29    print("-----")
30
31 except mysql.connector.Error as err:
32     print("Error:", err)
33
34 finally:
35     cursor.close()
36     conn.close()
```

PS C:\MySQL_Projects> python order_by_example.py

```
ORDER BY Results:
-----
Emp ID | Employee Name | Salary | Department
-----
1      | Charles      | 80000  | IT
2      | Bob          | 60000  | IT
3      | Alice        | 50000  | HR
4      | David        | 40000  | Finance
-----
```

PS C:\MySQL_Projects>

482. Write a Python Program to perform aggregate functions (SUM, AVG, COUNT) in MySQL.



```
1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='testdb'
9     )
10    cursor = conn.cursor()
11
12    # Create table and insert sample data
13    cursor.execute("DROP TABLE IF EXISTS sales")
14    cursor.execute("""
15    CREATE TABLE sales (
16        id INT PRIMARY KEY AUTO_INCREMENT,
17        product VARCHAR(50),
18        amount DECIMAL(10,2)
19    )
20    """)
21    data = [
22        ('Laptop', 50000.00),
23        ('Mobile', 15000.00),
24        ('Tablet', 10000.00),
25        ('Laptop', 30000.00),
26        ('Mobile', 25000.00)
27    ]
28    cursor.executemany("INSERT INTO sales (product, amount) VALUES (%s, %s)", data)
29    conn.commit()
30
31    # Aggregate Functions
32    query = """
33    SELECT
34        COUNT(*) AS total_records,
35        SUM(amount) AS total_amount,
36        AVG(amount) AS average_amount
37    FROM sales
38    """
39    cursor.execute(query)
40    result = cursor.fetchone()
41
42    # Display result
43    print("Aggregate Function Results:")
44    print("-----")
45    print(f"Total Records (COUNT): {result[0]}")
46    print(f"Total Amount (SUM): {result[1]:.2f}")
47    print(f"Average Amount (AVG): {result[2]:.2f}")
48
49 except mysql.connector.Error as err:
50     print("Error:", err)
51
52 finally:
53     cursor.close()
54     conn.close()
```

PS C:\MySQL_Projects> python aggregate_functions.py

```
Aggregate Function Results:
-----
Total Records (COUNT): 5
Total Amount (SUM): 130000.00
Average Amount (AVG): 26000.00
-----
```

PS C:\MySQL_Projects>

483. Write a Python Program to create and call a stored procedure in MySQL.

```
1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='testdb'
9     )
10    cursor = conn.cursor()
11
12    # Create stored procedure
13    cursor.execute("""
14        DROP PROCEDURE IF EXISTS GetEmployeesByDepartment;
15        DELIMITER //
16        CREATE PROCEDURE GetEmployeesByDepartment(IN dept_name VARCHAR(30))
17        BEGIN
18            SELECT id, name, salary, department FROM employees WHERE department = dept_name;
19        END //
20        DELIMITER ;
21    """)
22    print("Stored procedure created successfully.")
23
24    # Call stored procedure
25    dept = 'IT'
26    cursor.callproc('GetEmployeesByDepartment', (dept,))
27    results = cursor.stored_results()
28    for result in results:
29        rows = result.fetchall()
30        print(f"Employees in Department: {dept}")
31        print(f"ID | Name | Salary | Department")
32        print(f"-----|-----|-----|-----")
33        for row in rows:
34            print(f"{row[0]:<5} | {row[1]:<10} | {row[2]:<10} | {row[3]:<10}")
35        print(f"-----|-----|-----|-----")
36    conn.close()
37
38 except mysql.connector.Error as err:
39     print("Error: ", err)
```

PS C:\MySQL_Project> python stored_procedure.py
Stored procedure created successfully.
Employees in Department: IT
-----|-----|-----|-----
ID	Name	Salary	Department
1 | John | 50000 | IT
2 | David | 60000 | IT
-----|-----|-----|-----

484. Write a Python Program to create and call a stored function in MySQL.

```
1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='testdb'
9     )
10    cursor = conn.cursor()
11
12    # Create stored function
13    cursor.execute("""
14        DROP FUNCTION IF EXISTS GetTotalSalary;
15        DELIMITER //
16        CREATE FUNCTION GetTotalSalary(dept_name VARCHAR(30))
17        RETURNS DECIMAL(10,2)
18        BEGIN
19            DECLARE total DECIMAL(10,2);
20            SELECT IFNULL(SUM(salary),0) INTO total
21            FROM employees WHERE department = dept_name;
22            RETURN total;
23        END //
24        DELIMITER ;
25    """)
26    print("Stored function created successfully.")
27
28    # Call stored function
29    department = 'IT'
30    cursor.execute("SELECT GetTotalSalary(%s), (department,))", (department,))
31    result = cursor.fetchall()
32    print(f"Total Salary in Department '{department}': {result[0][.2f]}")
33    conn.close()
34
35 except mysql.connector.Error as err:
36     print("Error: ", err)
```

PS C:\MySQL_Project> python stored_function.py
Stored function created successfully.
Total Salary in Department 'IT': 110000.00
PS C:\MySQL_Project>

485. Write a Python Program to create and use triggers in MySQL.

```
1 import mysql.connector
2
3 try:
4     conn = mysql.connector.connect(
5         host='localhost',
6         user='root',
7         password='password',
8         database='testdb'
9     )
10    cursor = conn.cursor()
11
12    # Create table
13    cursor.execute("""
14        DROP TABLE IF EXISTS products;
15        CREATE TABLE products (
16            product_id INT(10) UNSIGNED PRIMARY KEY,
17            product_name VARCHAR(100),
18            price DECIMAL(10,2) UNSIGNED,
19            stock INT(10) UNSIGNED
20        );
21    """)
22    print("Table created successfully.")
23
24    # Create trigger
25    cursor.execute("""
26        DROP TRIGGER IF EXISTS log_table;
27        DELIMITER //
28        CREATE TRIGGER log_table AFTER INSERT ON products FOR EACH ROW
29        BEGIN
30            INSERT INTO log_table (log_id, product_id, action, log_time)
31            VALUES (1, NEW.product_id, 'INSERTED', NOW());
32        END //
33        DELIMITER ;
34    """)
35    print("Trigger created successfully.")
36
37    # Check log table
38    cursor.execute("SELECT * FROM log_table")
39    logs = cursor.fetchall()
40
41    print("Log Table Records:")
42    print(f"Log ID | Product ID | Action | Log Time")
43    print(f"-----|-----|-----|-----")
44    for log in logs:
45        print(f"{log[0]:<5} | {log[1]:<10} | {log[2]:<10} | {log[3]:<20}")
46    print(f"-----|-----|-----|-----")
47
48    # Insert data
49    cursor.execute("INSERT INTO products (product_id, product_name, price, stock) VALUES (1, 'Product A', 100.00, 100);")
50    conn.commit()
51
52 except mysql.connector.Error as err:
53     print("Error: ", err)
```

PS C:\MySQL_Project> python triggers_example.py
Table and trigger created successfully.
Log Table Records:
-----|-----|-----|-----
Log ID	Product ID	Action	Log Time
1 | 1 | INSERTED | 2024-09-23 10:28:34
-----|-----|-----|-----

486. Write a Python Program to implement connection pooling in MySQL.



```
1 import mysql.connector
2 from mysql.connector import pooling
3
4 try:
5     # Create a connection pool
6     dbconfig = {
7         'host': 'localhost',
8         'user': 'root',
9         'password': 'password',
10        'database': 'testdb'
11    }
12
13    pool = pooling.MySQLConnectionPool(
14        pool_name='mypool',
15        pool_size=5,
16        dbconfig=dbconfig
17    )
18    print("Connection pool created successfully.")
19
20    # Get connections from pool and perform operations
21    for i in range(3, 5):
22        conn = pool.get_connection()
23        cursor = conn.cursor()
24        cursor.execute("SELECT VERSION()")
25        version = cursor.fetchone()
26        print(f"Connection {i}: MySQL Version = {version[0]}")
27        cursor.close() # This returns the connection to the pool
28        conn.close()
29
30 except mysql.connector.Error as err:
31    print("Error: ", err)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\MySQL_Projects> python connection_pooling.py
Connection pool created successfully.
Connection 3: MySQL Version => 8.0.36
Connection 4: MySQL Version => 8.0.36
Connection 5: MySQL Version => 8.0.36
PS C:\MySQL_Projects>



GLOBAL QUEST
TECHNOLOGIES

fuel your
passion for
IT with
our **guidance.**

Global Quest Technologies



#324, 2nd Floor, 3 A Cross, Near
Seshadripuram First Grade College,
Above City Union Bank,
Yelahanka New Town,
Bengaluru-560064

+91 9448 403 469 | 080-49720009
info@gqtech.in | www.gqtech.in